Artificial Neural Networks
&
Deep Learning

Dr. Majed Elwardy
22/09/2026

Acknowledgement
These slides are reused **as-is from previous years**, prepared by Dr. Huseyin Kusetogullari and Dr. Sadi Alawadi, DIDA, BTH.

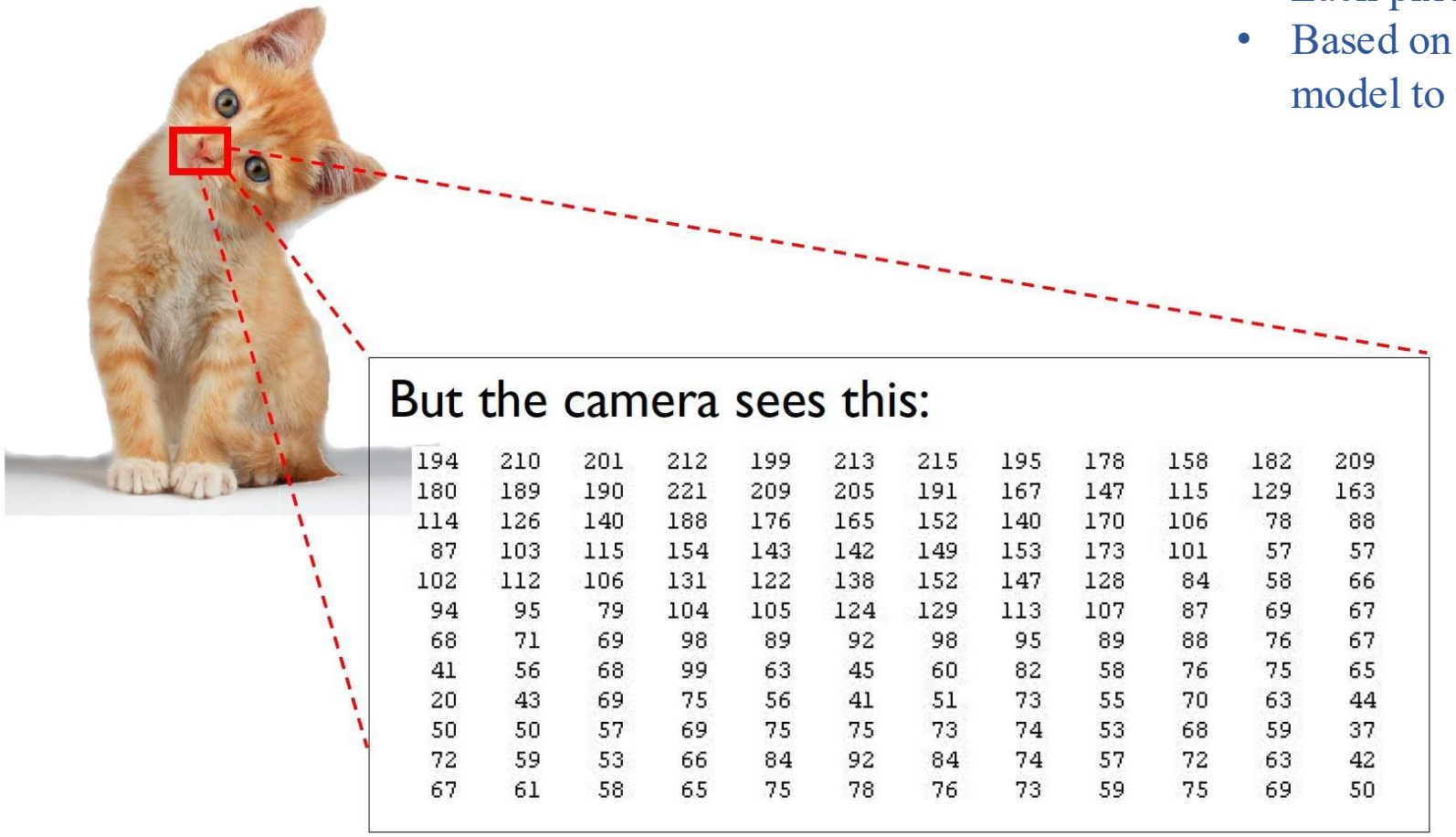
Outlines

- Artificial Neural Networks (ANNs)
- Gradient descents
- Multi Layers Networks
- Deep Learning
- Deep Learning timeline

Artificial Neural Networks (ANNs)

Artificial Neural Networks & Deep learning

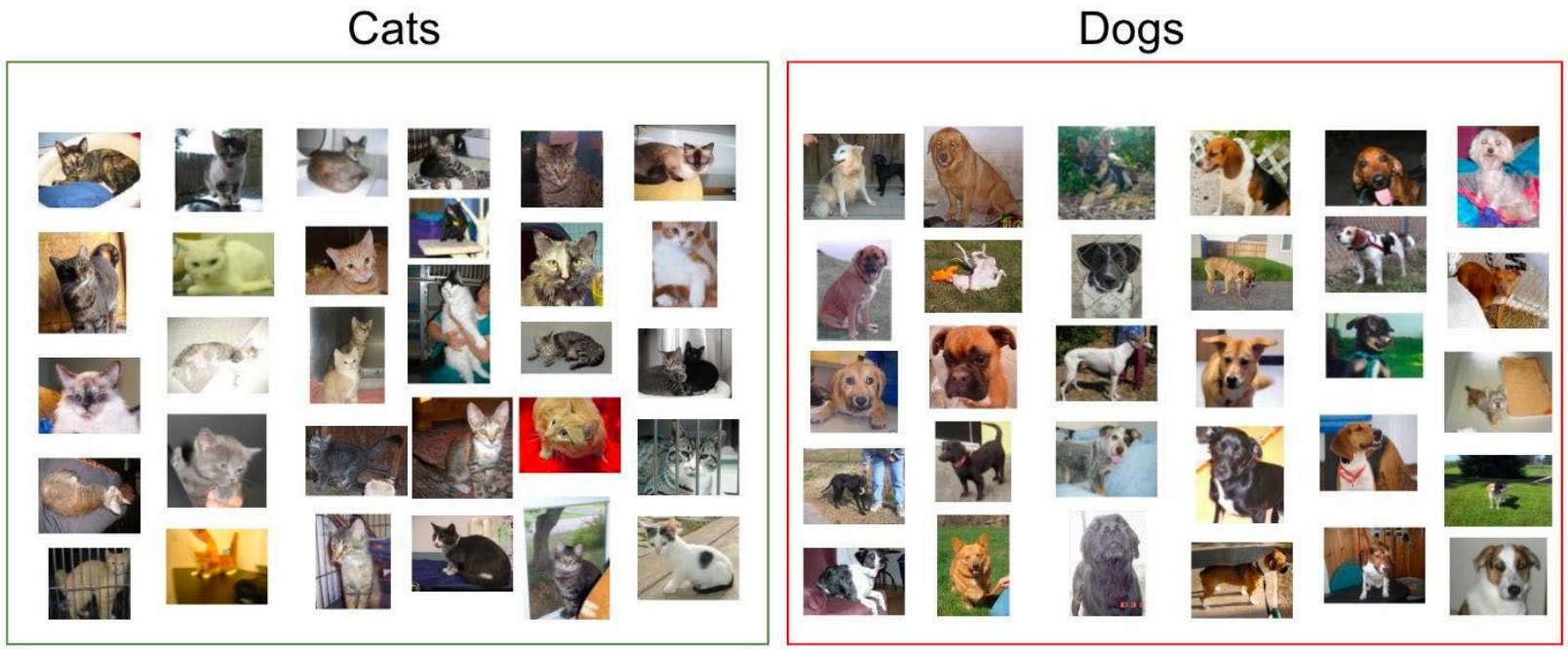
Example: what is this?



- Each pixel can be seen as a feature.
- Based on these pixel values, you want the ML model to tell you that this is a “nose of a cat”

Artificial Neural Networks & Deep learning

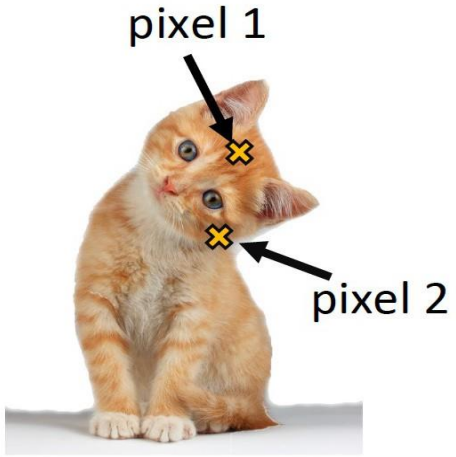
Large Dataset *d*

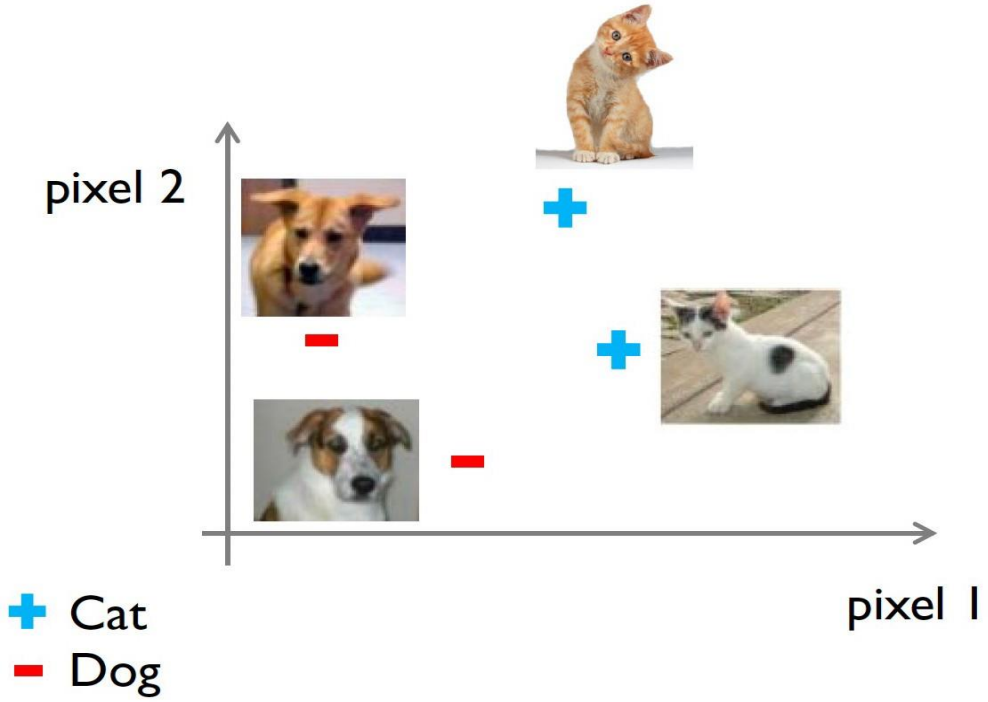


Testing:
What is this?

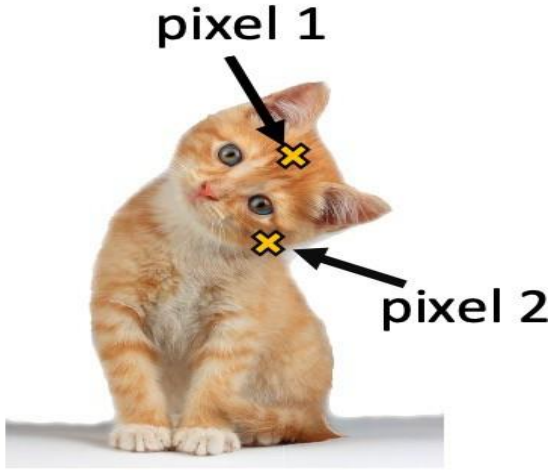


Large Dataset *d*

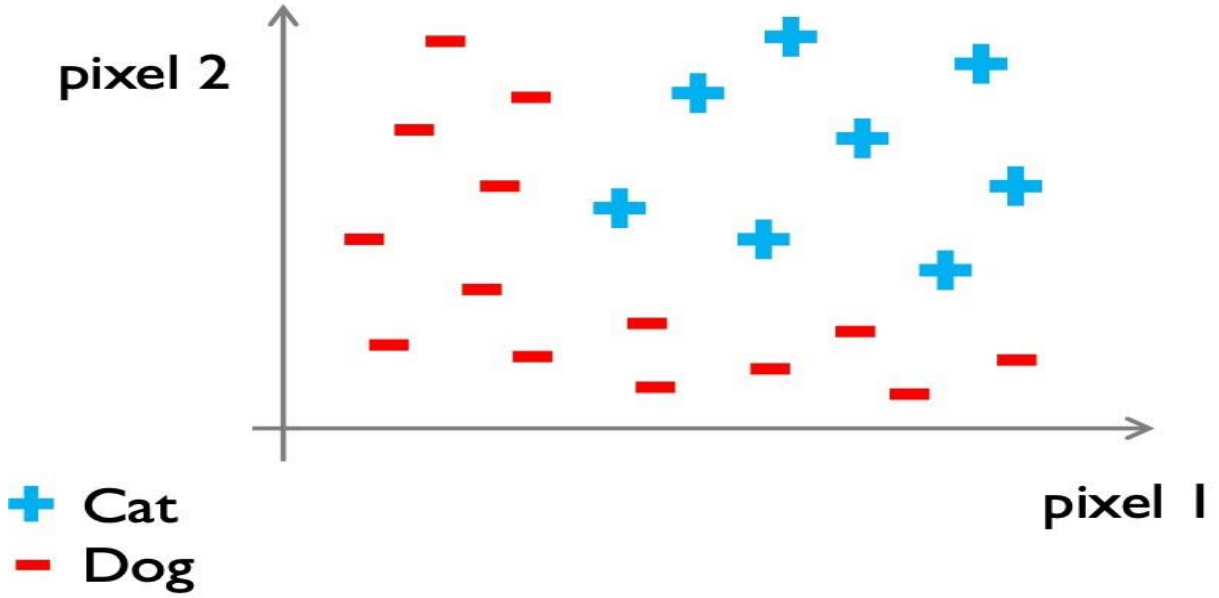




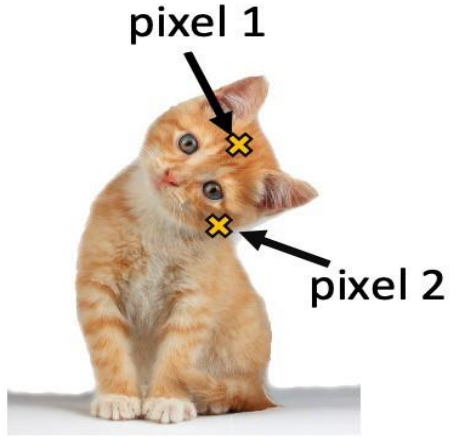
Large Dataset *d*



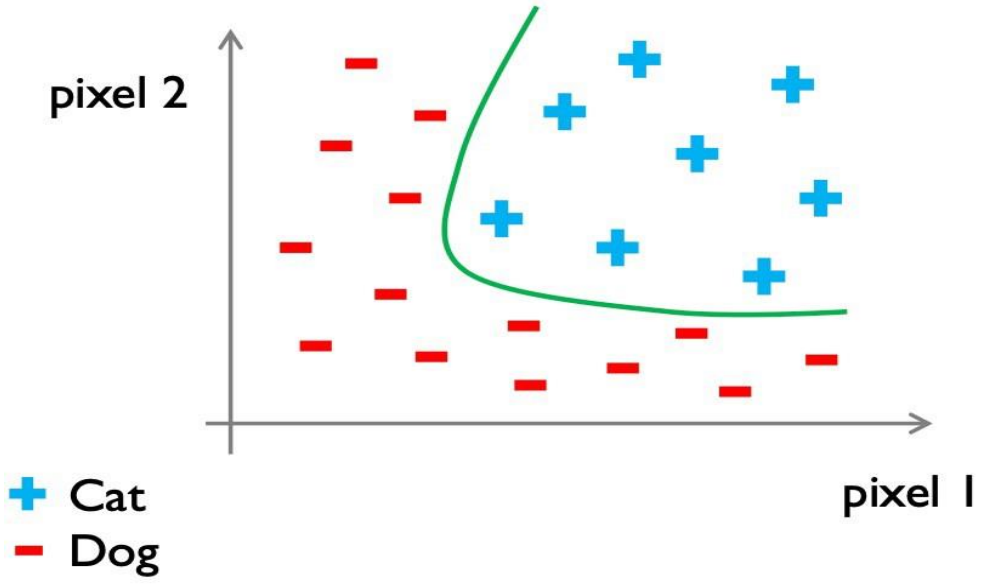
Learning Algorithm



Large Dataset *d*



Learning Algorithm



50 × 50 pixel images→ 2500 pixels
$d = 2500$ (7500 if RGB)

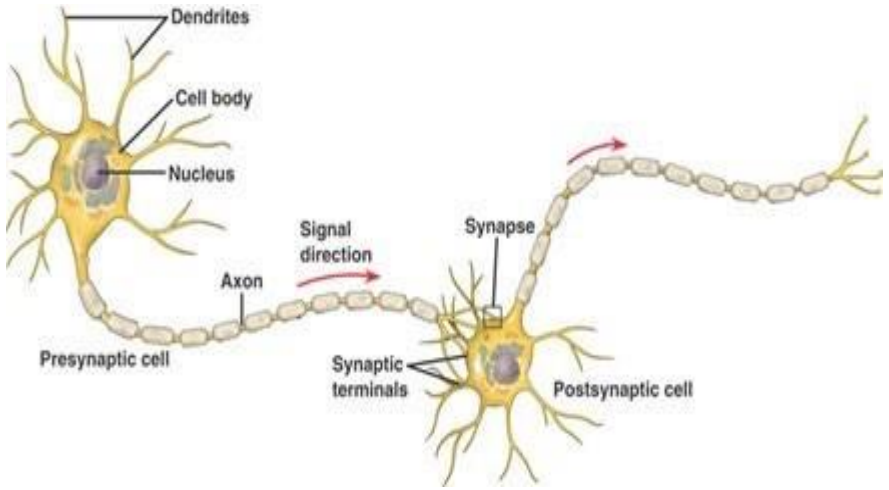
$$x = \begin{bmatrix} \text{pixel 1 intensity} \\ \text{pixel 2 intensity} \\ \vdots \\ \text{pixel 2500 intensity} \end{bmatrix}$$

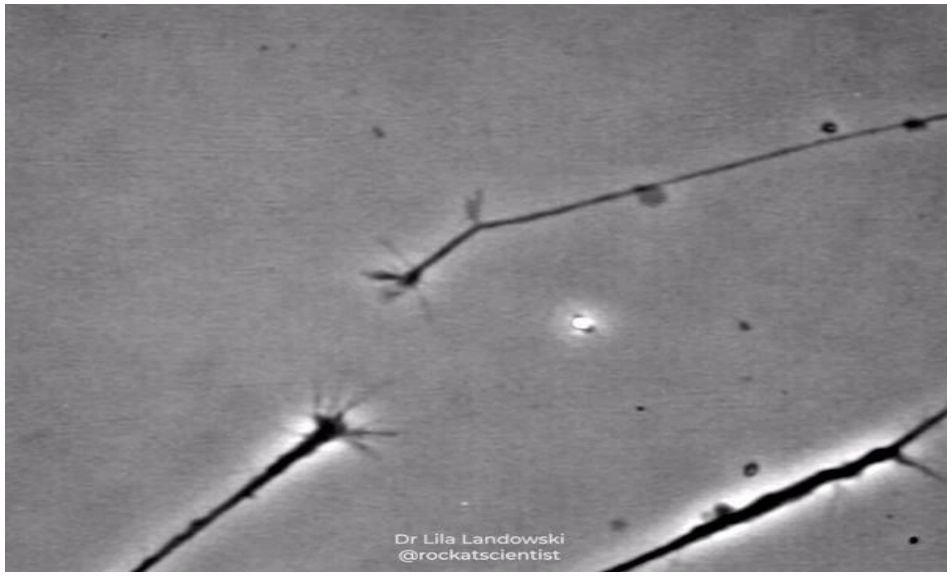
Quadratic features ($x_i \times x_j$): ≈3 million features

Artificial Neural Networks & Deep learning

Definition: Artificial Neural Networks (ANN)

- An artificial network consists of a pool of simple processing units which communicate by sending signals to each other over a large number of weighted connections.
- ANN is ***a mathematical model*** that tries to ***simulate*** the structure and functionalities of biological neural networks in the brain.
- Basic building block of an ANN is artificial neuron, that is a simple mathematical model (function).



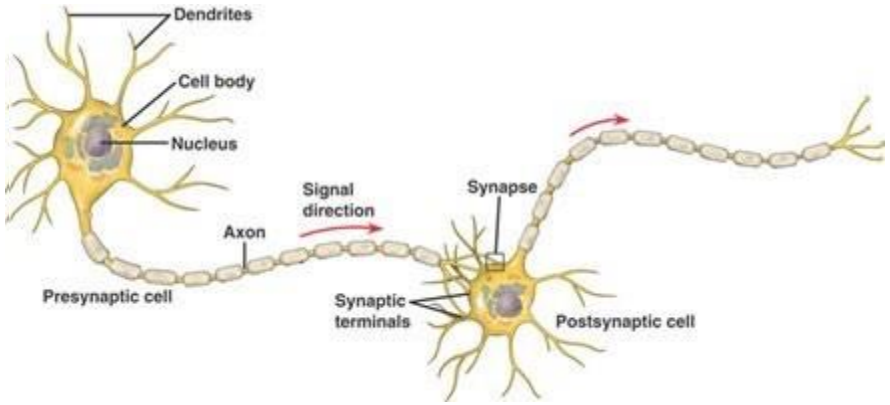


Two neurons sensing one another and connecting.
This is a video taken of **a neuron in a culture dish,** and the video has been sped up! This was actually the growth that occurred over the space of **20 minutes**. * This video belongs to Dr Lila Landowski.

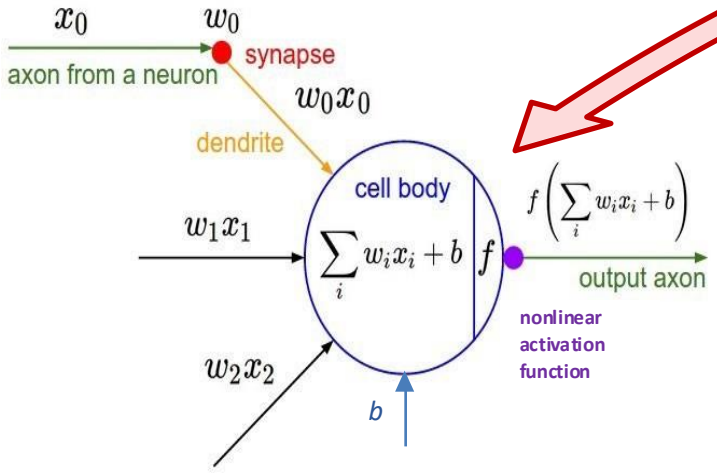
https://www.youtube.com/shorts/9ebifjoFtJs

Artificial Neural Networks & Deep learning

- The brain has **~100 billion** neurons and **~1000 trillion** synaptic connections.
- A neuron can **send electrical** pulses, which then **excite** the connected neuron. If this neuron gets **sufficiently excited**, it will further **transfer** electrical pulses to the **next** one.

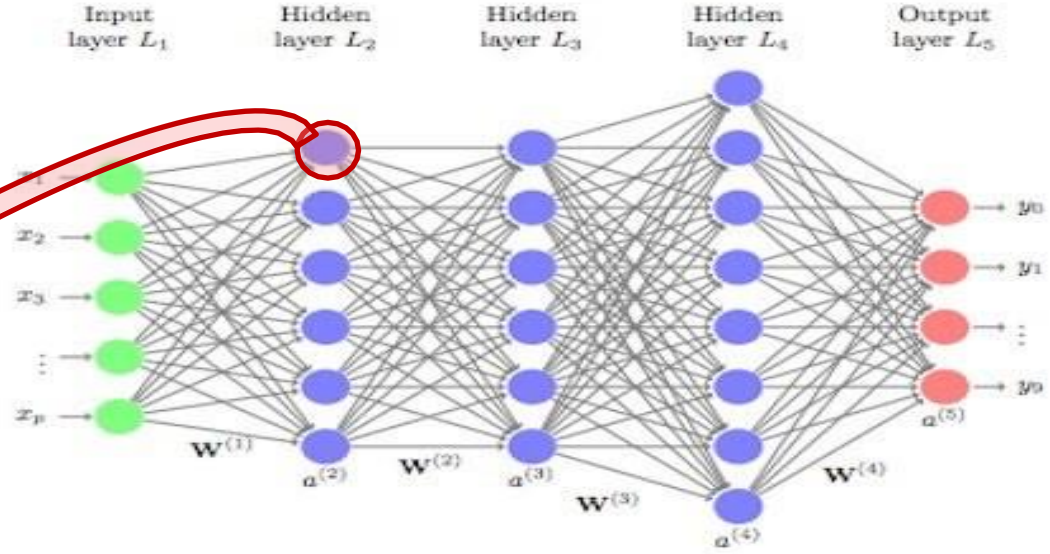


Biological Neuron



Artificial Neuron

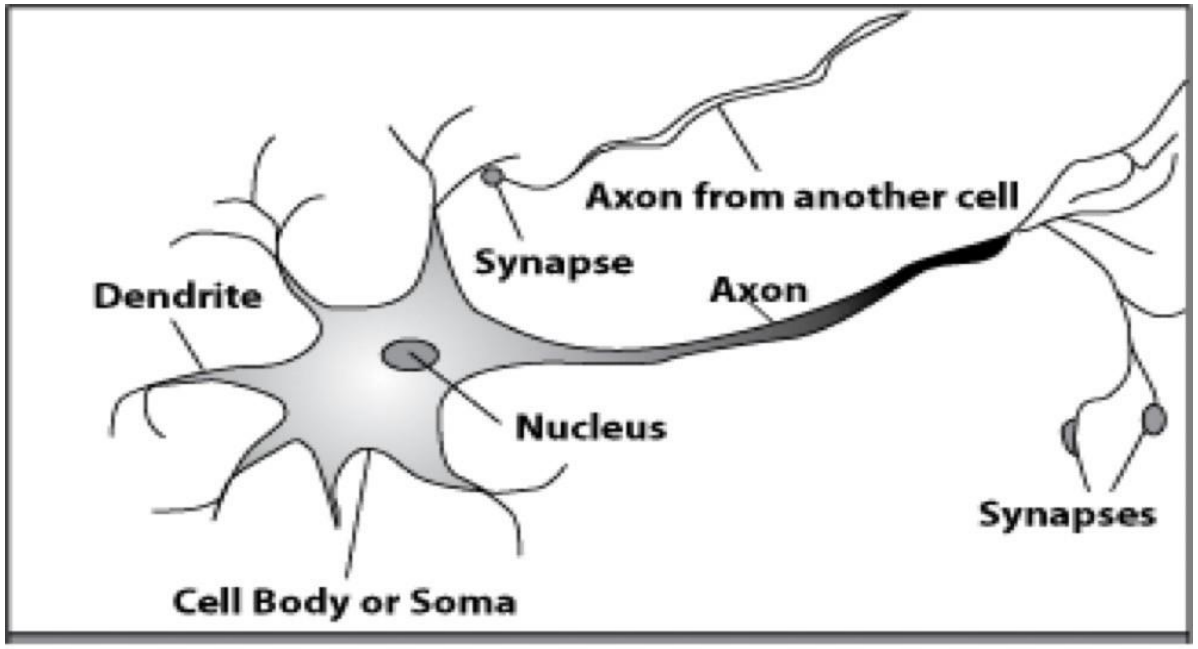
Input can be in any arbitrary length



Artificial Neural Network

Artificial Neural Networks

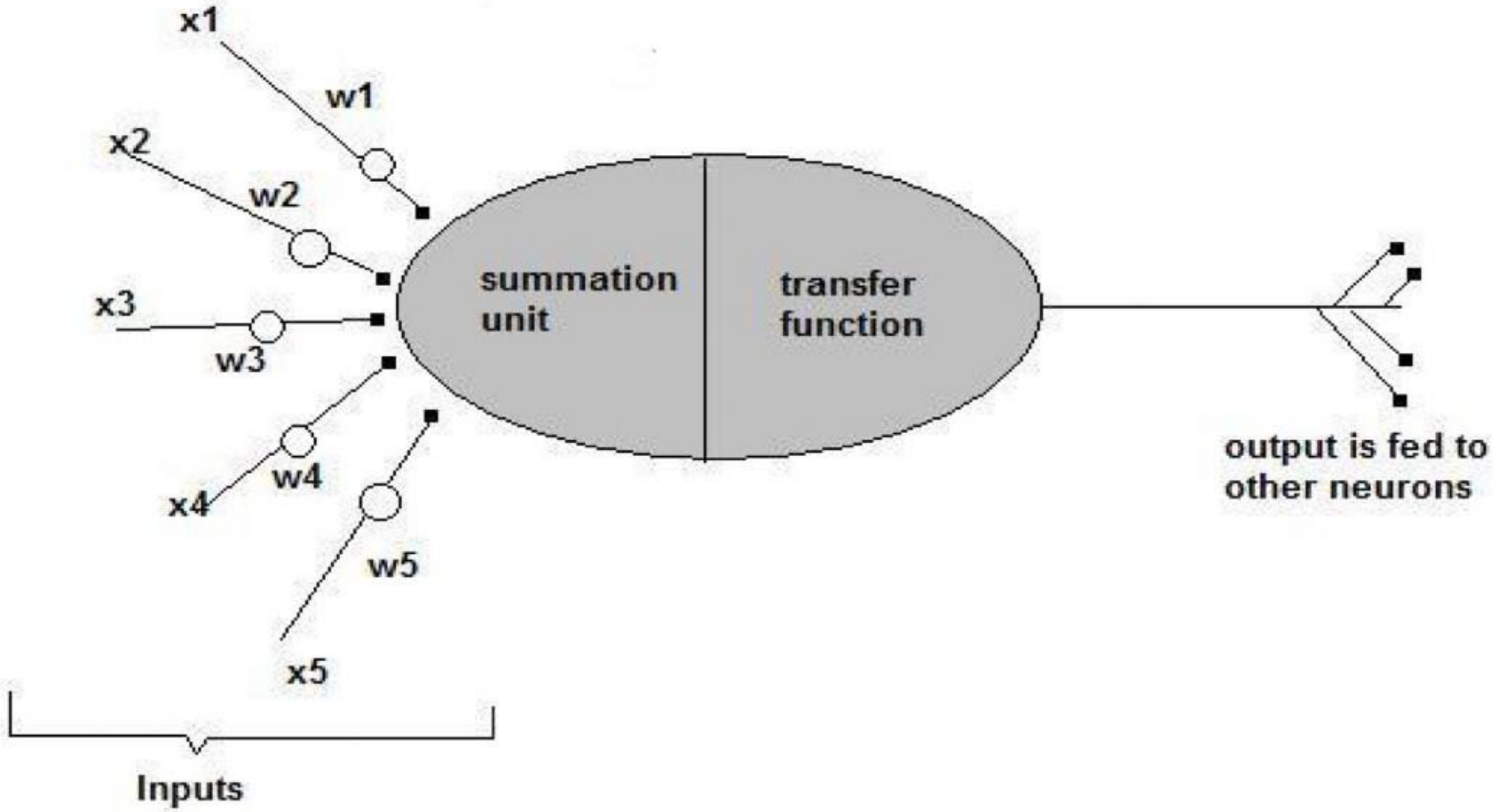
- Information is transmitted as a series of electric **impulses,** so-called **spikes**.
- The **frequency** and phase of these **spikes** encode the information.
- The neurons are connected by **weighted** links passing signals from one neuron to another.
- Each neuron receives a **number of input signals** through its connections; however, it never produces more than a **single output signal.**



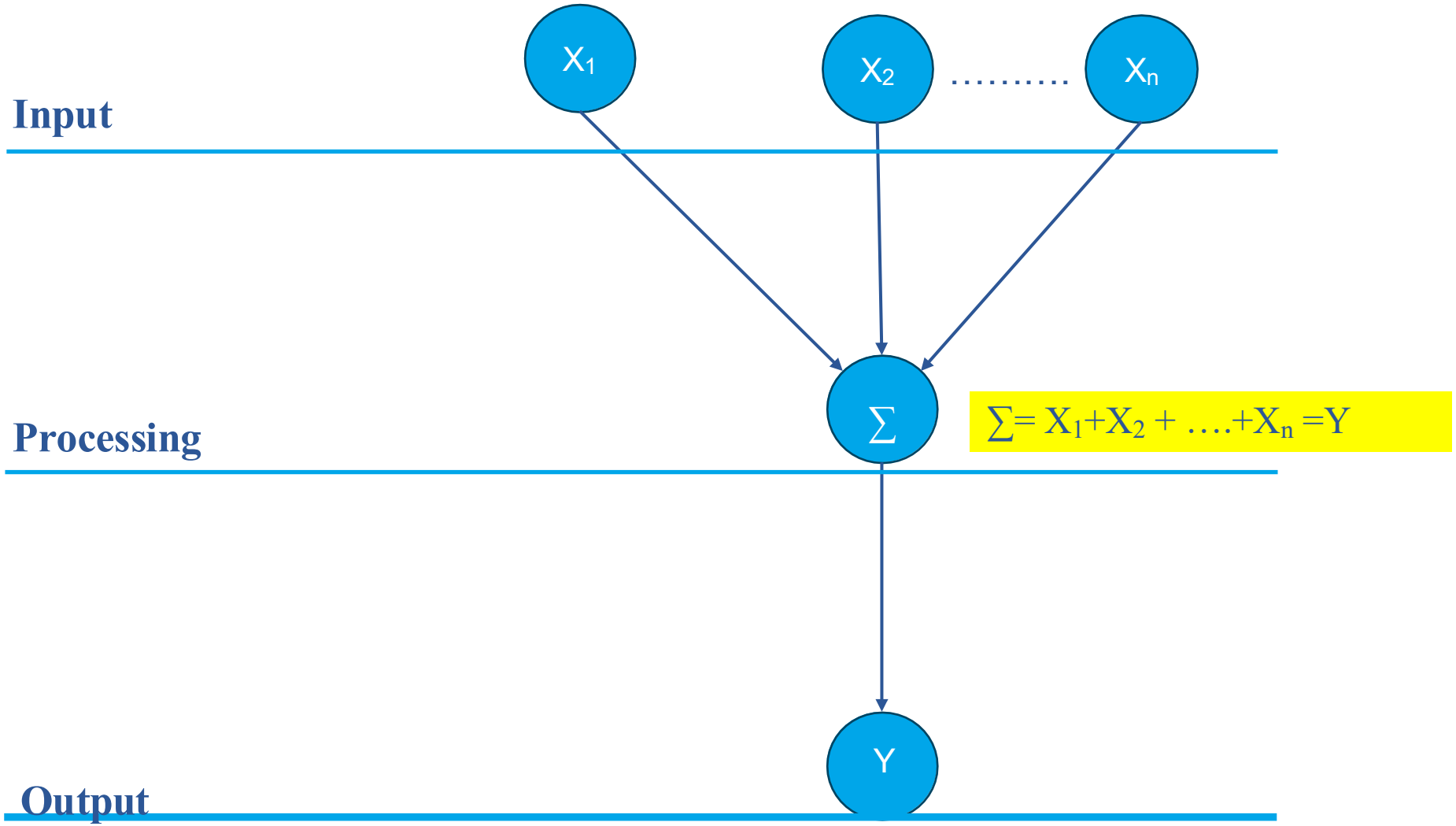
Biological neural network	Artificial neural network
Soma	Neuron
Dendrite	Input
Axon	Output
Synapse	Weight

Artificial Neural Networks

A Single Neuron

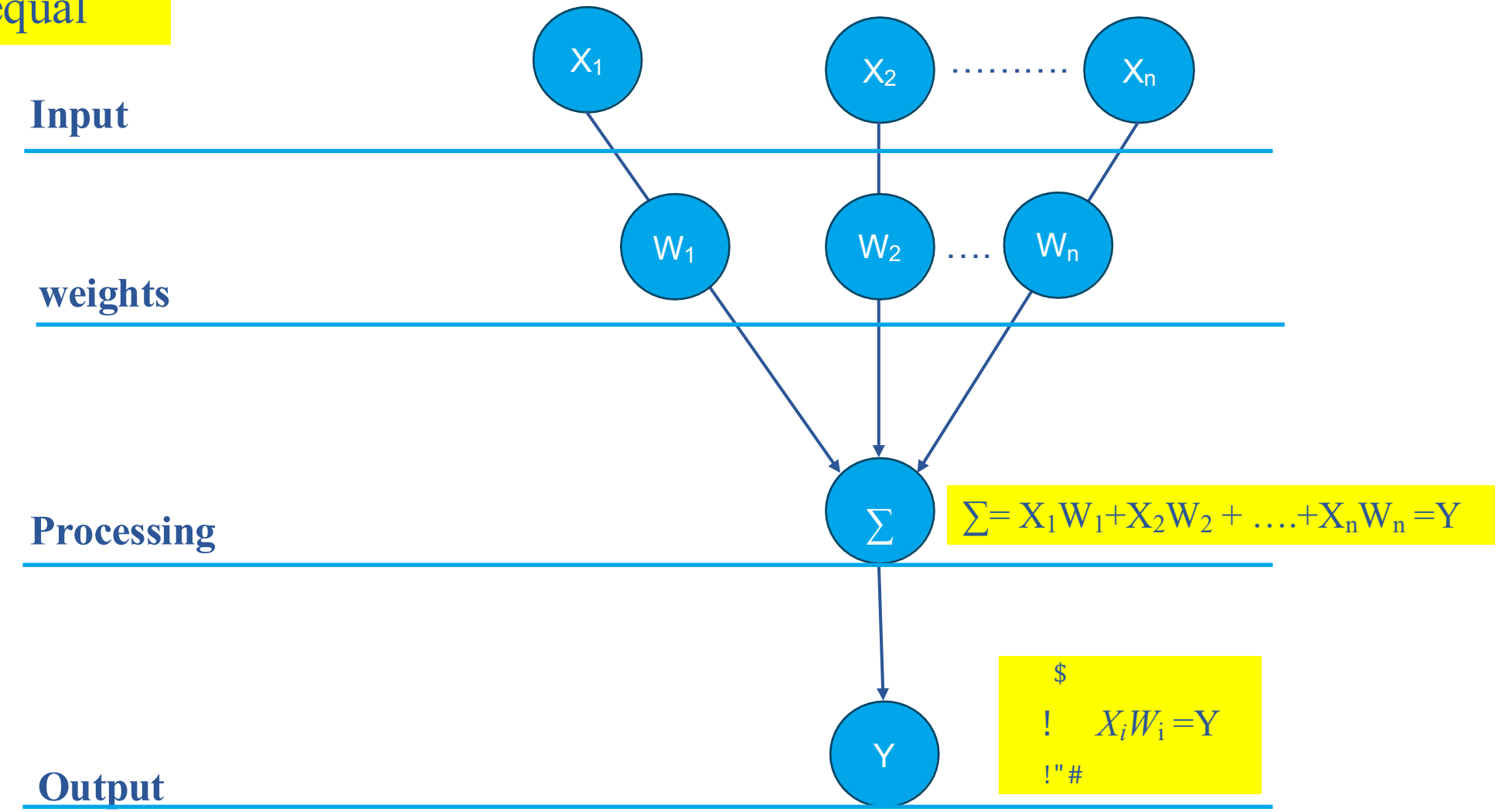


How do ANNs work?



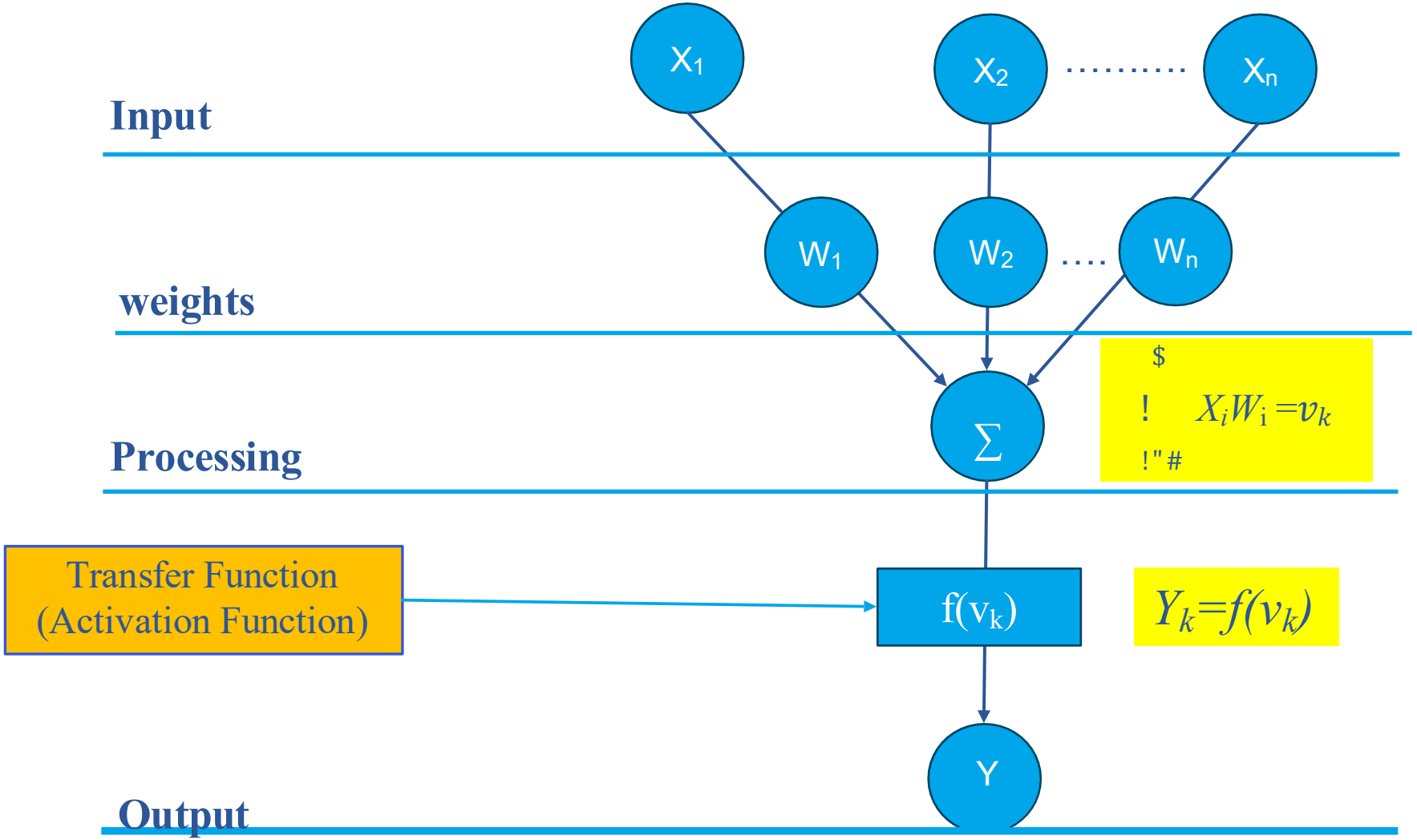
How do ANNs work?

Not all inputs are equal



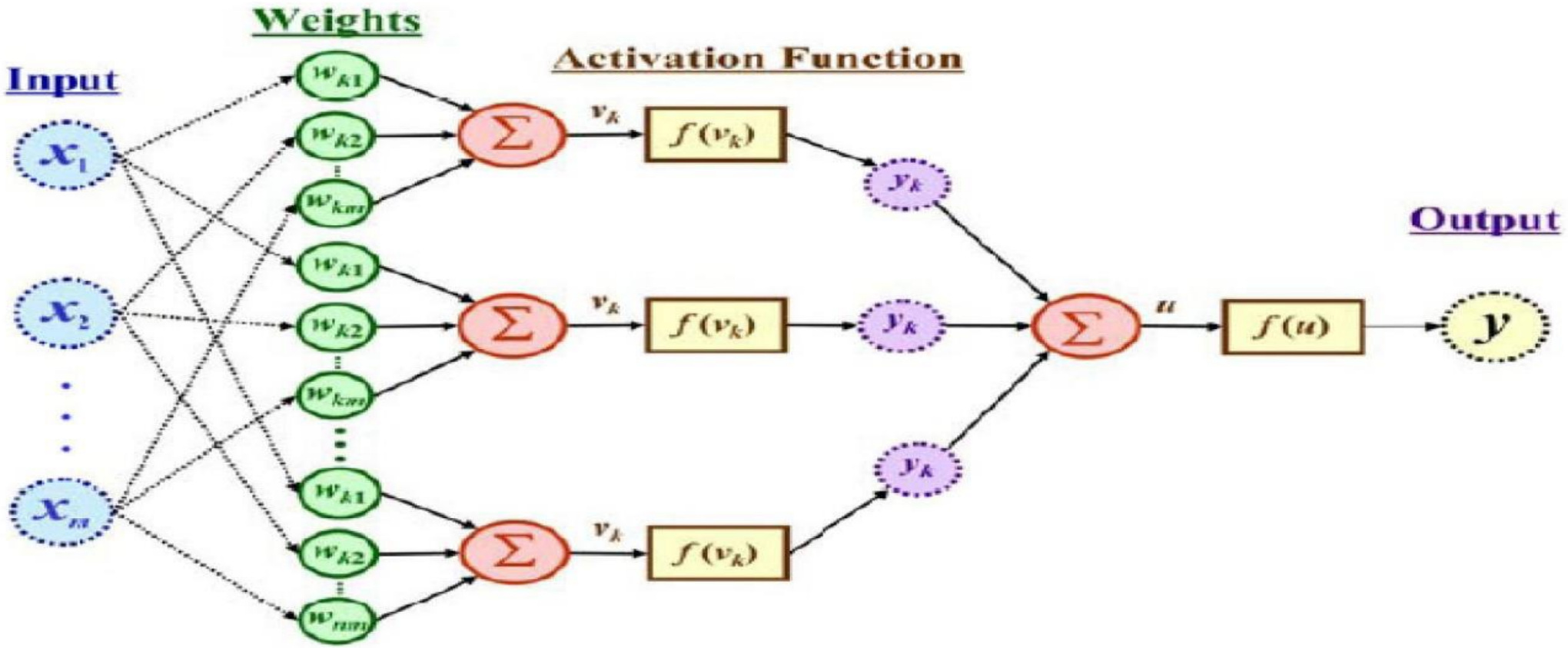
How do ANNs work?

The signal is not passed down to the next neuron verbatim



How do ANNs work?

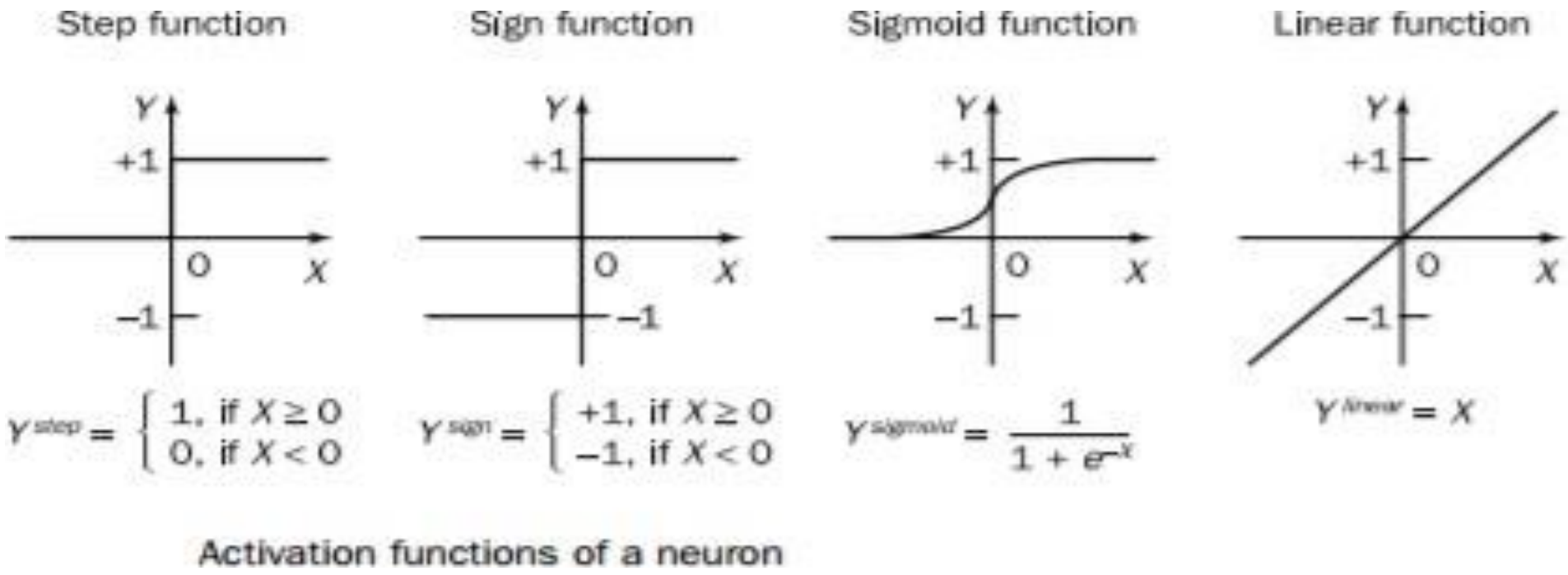
The output is a function of the input, that is affected by the weights, and the transfer functions



Activation Functions

- Many activation functions have been proposed, but only a few have found practical applications.
- Their main purpose is to convert a **input signal** of a node in a A-NN to **an output signal**.
- Activation functions are important to **avoid** using **large values** in NN (the unbounded linearly weighted sum from neurons).
- Help the network to use the useful information (**decrease the noise effect**).
- The step and sign activation functions, also called hard limit functions **(0 or 1)**.
- Linear and sigmoid activation functions are soft limit functions **(transition is gradual)**.

Activation Functions



Activation functions of a neuron

Example: Find y (output) for x=5 (input) and use four activation functions

y = +1 (Step function)

y = +1 (Sign function)

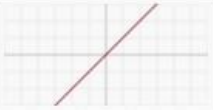
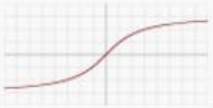
$y = 1/1+e^{-5} = 0.9933$ (Sigmoid function)

y = 5 (Linear function)

Activation Functions

Why derivative/differentiation is used?

When updating the curve, to know in which direction and how much to change or update the curve depending upon the slope.That is why we use differentiation in almost every part of Machine Learning and Deep Learning.

Name	Plot	Equation	Derivative
Identity		$f(x) = x$	$f'(x) = 1$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1+e^{-x}}$	$f'(x) = f(x)(1-f(x))$
TanH		$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2+1}$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parameteric Rectified Linear Unit (PReLU)[2]		$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Exponential Linear Unit (ELU)[3]		$f(x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
SoftPlus		$f(x) = \log_e(1+e^x)$	$f'(x) = \frac{1}{1+e^{-x}}$

https://towardsdatascience.com/activation-functions-neural-networks-1cbd9f8d91d6

Activation Functions

How to choose the right Activation Function?

Thumb rule, start using the ReLU activation function if ReLU doesn't provide optimum results, check another activation functions.

1. **ReLU** activation function should **only** be used in **the hidden layers**.
2. **Sigmoid/Logistic** and **Tanh functions** should **not** be used in **hidden layers** as they make the model more susceptible to problems during training (due to vanishing gradients).
3. **Swish** function is used in neural networks having a depth greater than **40 layers**.

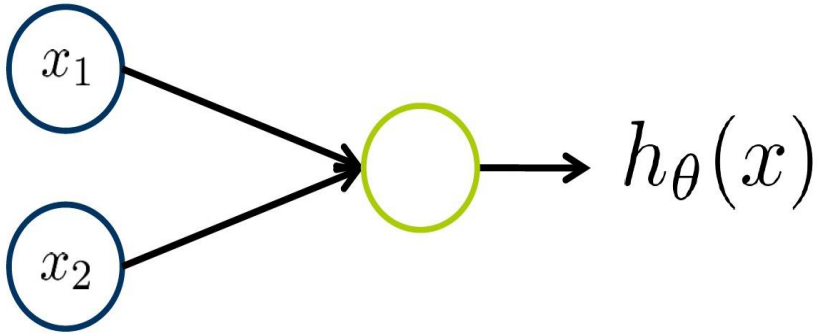
Rules for choosing the activation function for your output layer based on the task:

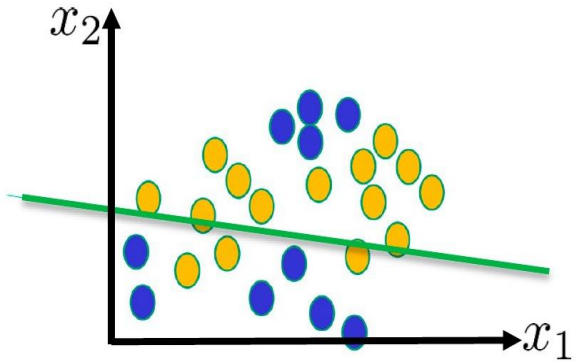
1. **Regression** - Linear Activation Function
2. **Binary** Classification—Sigmoid/Logistic Activation Function
3. **Multiclass** Classification—Softmax
4. **Multilabel** Classification—Sigmoid
5. **Convolutional Neural Network (CNN):** ReLU activation function.
6. **Recurrent Neural Network**: Tanh and/or Sigmoid activation function.

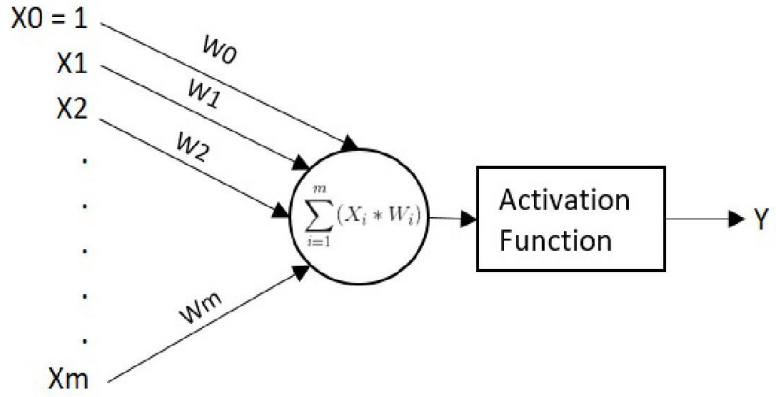
https://towardsdatascience.com/activation-functions-neural-networks-1cbd9f8d91d6

Neural Network Representation

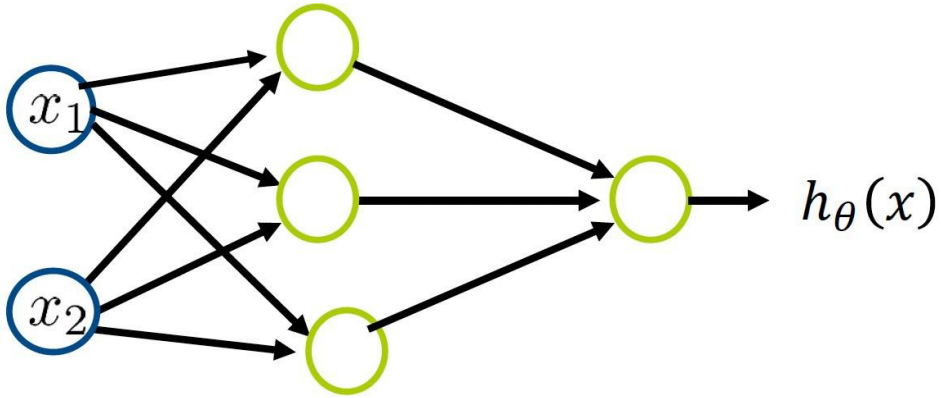
With 0 hidden layers, we get a linear model (Perceptron = logistic regression algorithm)

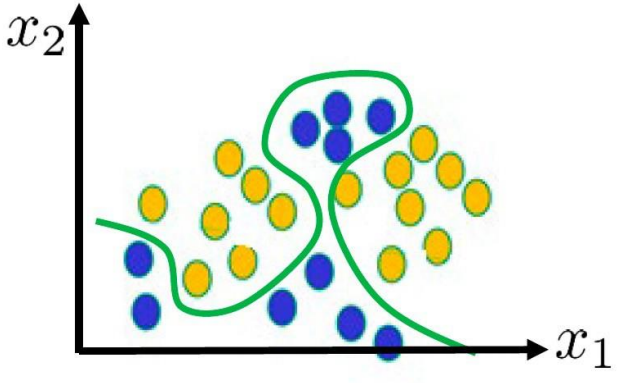




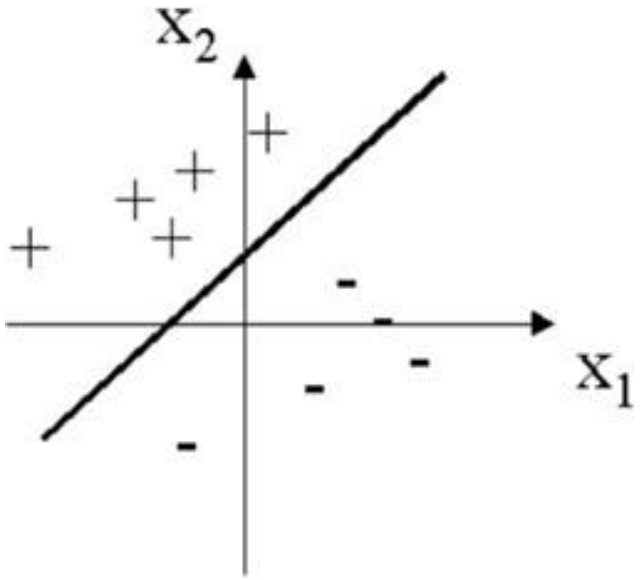


With at least one hidden layer (neural network) we can get a nonlinear model

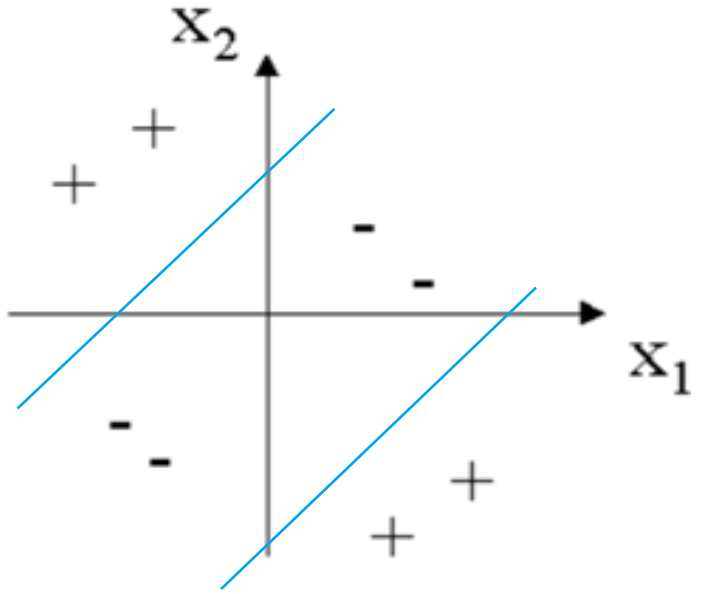




Perceptron

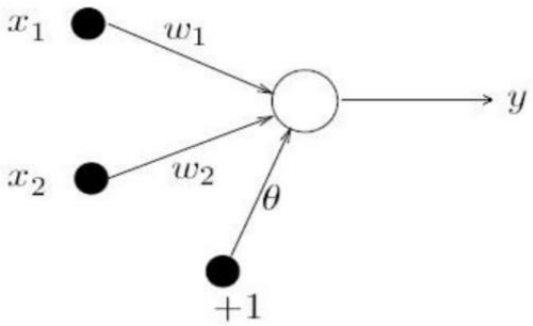


Classification with one Perceptron



Classification with more than one Perceptron

$$w_1 x_1 + w_2 x_2 + \theta = 0$$



Single-layer perceptron with threshold units fails if the problem is not linearly separable

- Example: XOR

Gradient Descents

- Gradient descent is an algorithm that minimizes functions.
- Iteratively moves toward a set of parameter values that minimize the function.
- This iterative minimization is achieved using calculus, taking steps in the negative direction of the function gradient.
- The optimization formula is given as follows:

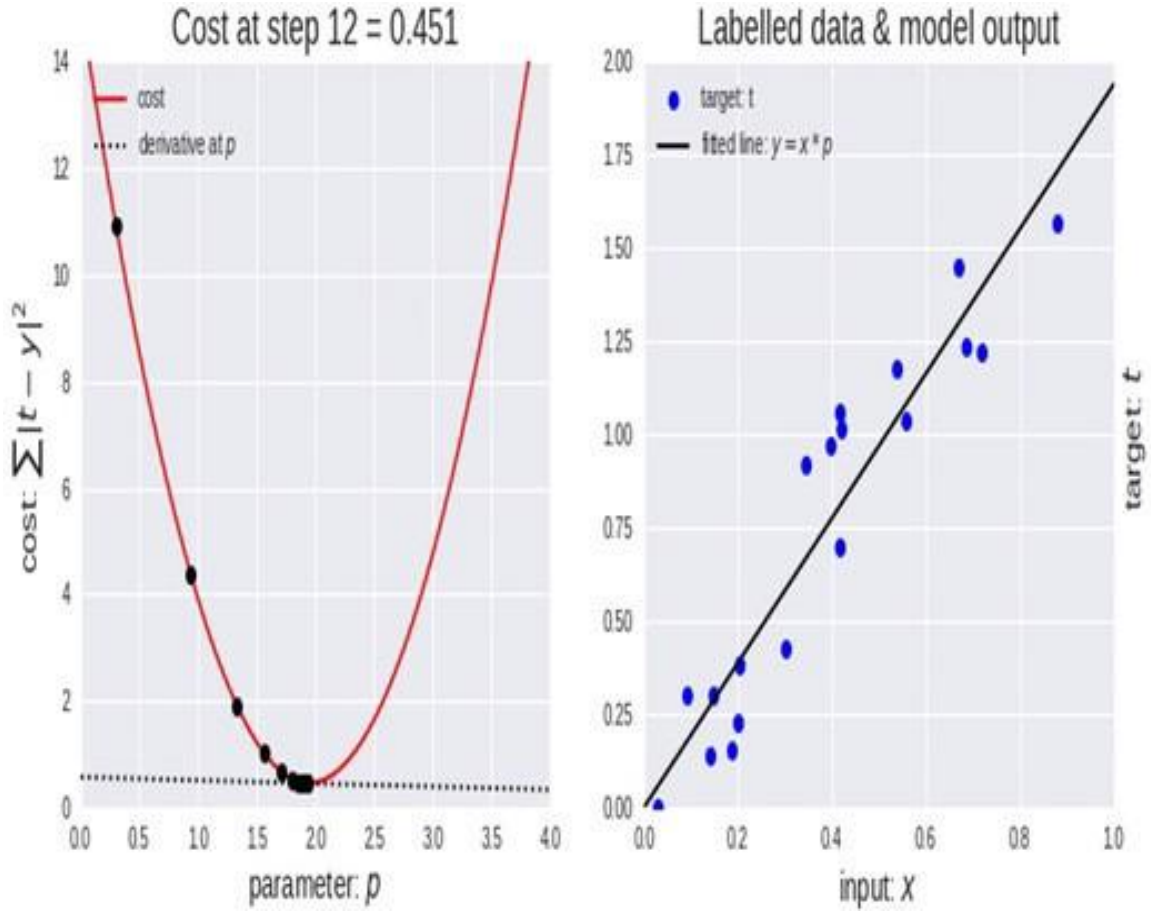
$$X = X - lr * \frac{d}{dX} f(X)$$

$Where,$

$X \quad = input$

$F(X) \; = output \;\; based \; on \; X$

$lr \quad = learning \; rate$



https://www.mql5.com/en/articles/11200
https://pub.towardsai.net/gradient-descent-algorithm-explained-2fe9da0de9a2

Gradient Descents

Example:

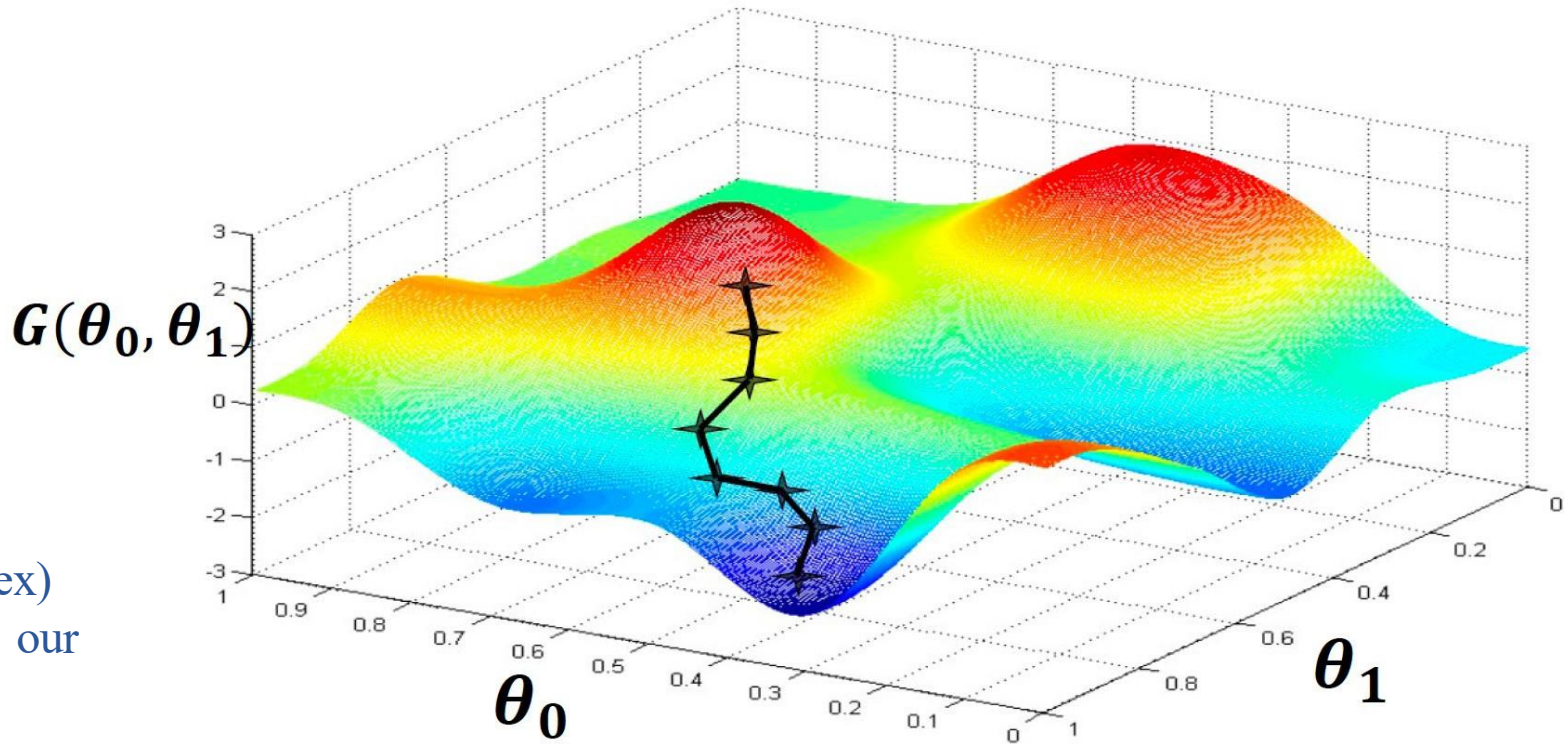
- Minimize $f(x) = x^2$ using Gradient Descent method.
  - Answer: **Derivative** of $f(x)$: $\frac{\partial f}{\partial x} = f'(x) = 2x$

Assume that: learning_rate=0.1, initial_x=11

x= x – learning_rate * 2*x		
zero iteration:	x=11,	$f(x^2)= 121$
1st iteration:	x=11 – 0.1*2 * 11 = 8.8	$f(x^2)=77.44$
2ed Iteration :	x= 8.8 – 0.1 * 2* 8.8 = 7.04	$f(x^2)= 49.56$
3rd Iteration:	x= 7.04 – 0.1 * 2* 7.04 = 5.63	$f(x^2)= 31.71$
4th Iteration:	x= 5.63– 0.1 * 2* 5.63 = 4.50	$f(x^2)= 20.30$
…….		
50th Iteration:	x= 0.000245– 0.1 * 2* 0.000245 = 0.000196	$f(x^2)= 0.0000000385$

Gradient Descents – Basic idea

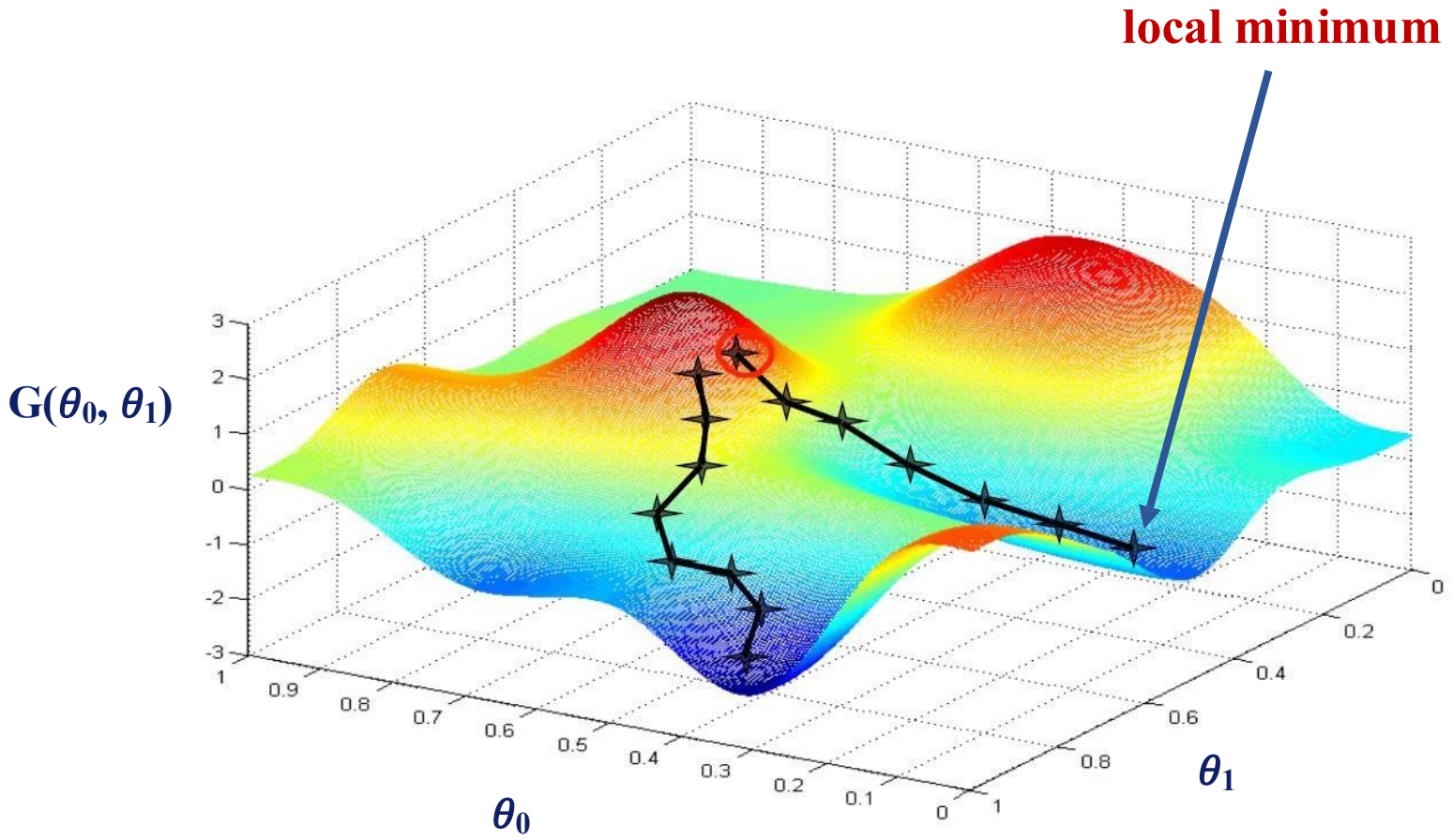
1. Start with some values for the parameters θ_0, θ_1
2. Keep updating θ_0, θ_1 to reduce $E(\theta_0, \theta_1)$ until we end up at a minimum.



Note:
G(…) is just some arbitrary (non-convex) function used for this example. It's not our previous (convex) cost function E(…).

Gradient Descents – Basic idea

1. Start with some values for the parameters θ_0, θ_1
2. Keep updating θ_0, θ_1 to reduce $E(\theta_0, \theta_1)$ until we end up at a minimum.
   - Depending on initial parameters values, we might end-up at a different (local) minimum.

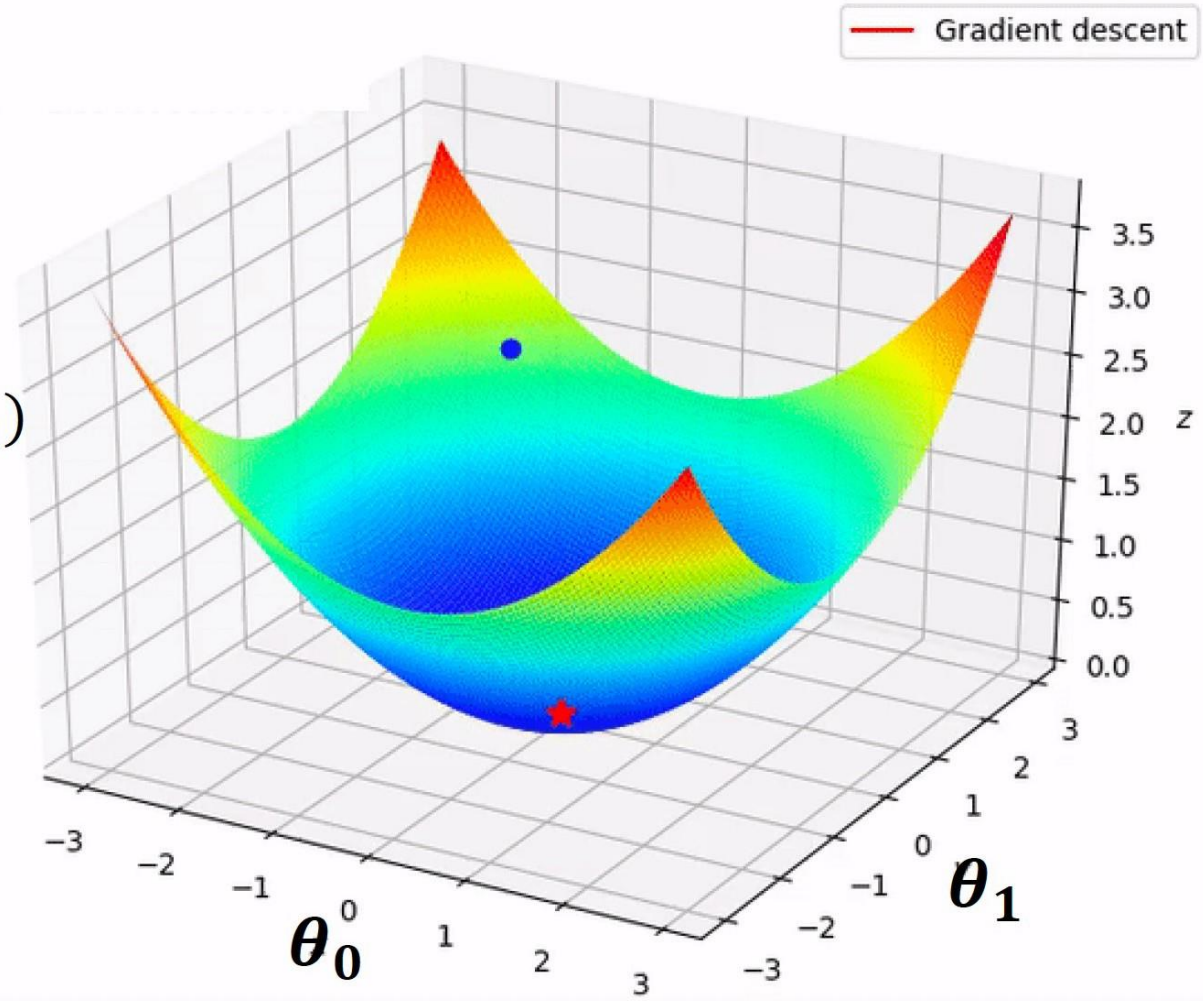


Note:
G(…) is just some arbitrary (non-convex) function used for this example. It's not our previous (convex) cost function E(…).

Gradient Descents

Example with a convex error function (MSE).
Only one (global) minimum.

$E(\theta_0, \theta_1)$
MSE



At each update step, how does gradient descent decide if we should increase or decrease each of the parameters θ_0 and θ_1 ?

Gradient Descents - Algorithm

Repeat until convergence {

$$\theta \leftarrow \theta - \underbrace{\alpha}_{\text{Learning rate } \alpha > 0} \underbrace{\nabla E(\theta)}_{\text{Gradient of } E}$$

}

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}$$

- The gradient of the cost function $E(\theta)$ is simply a vector containing the derivative of $E(\theta)$ with respect to each parameter θ_j

$$\nabla E(\theta) = \begin{bmatrix} \frac{\partial E(\theta)}{\partial \theta_0} \\ \frac{\partial E(\theta)}{\partial \theta_1} \end{bmatrix}$$

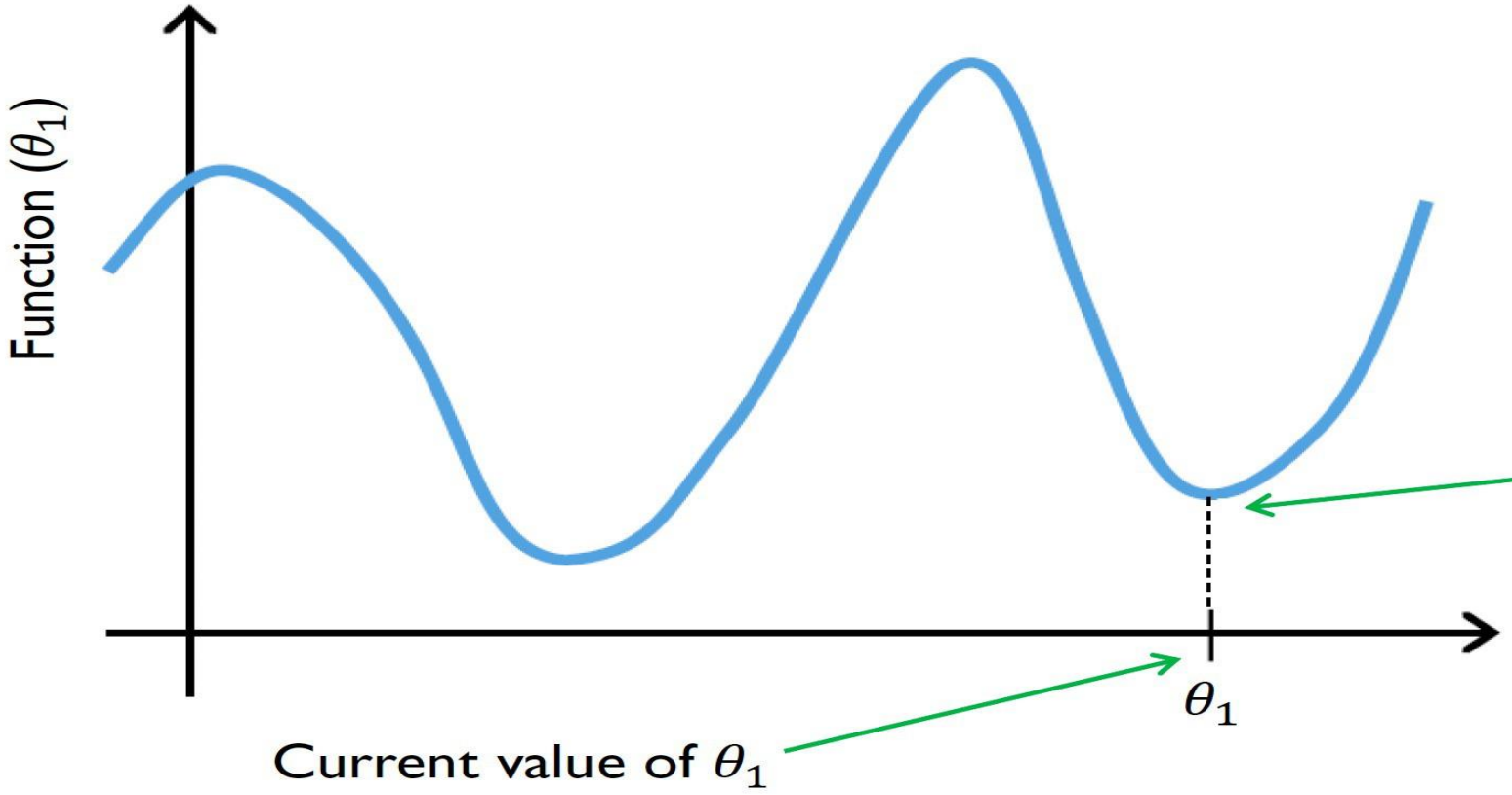
Gradient of E

Derivative of E with respect to θ_0

Derivative of E with respect to θ_1

Gradient Descents - Algorithm

Assume for now that we have only one parameter θ_1



Assume that we have reached a local optimum (local minimum here).

θ_1 at a local minimum

$$\theta_1 \leftarrow \theta_1 - \alpha \frac{\partial E}{\partial \theta_1}$$

The derivative will be equal to 0, so **convergence**. We get stuck at this local minimum.

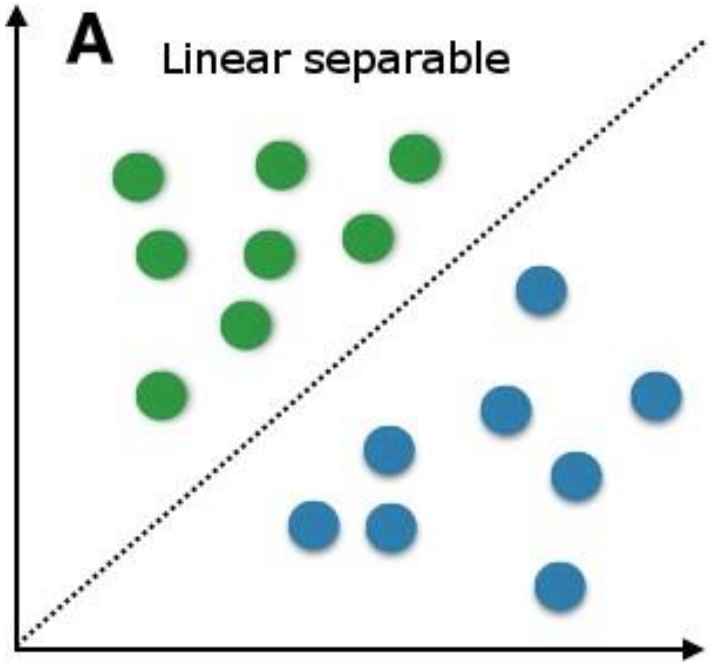
Gradient Descents Types

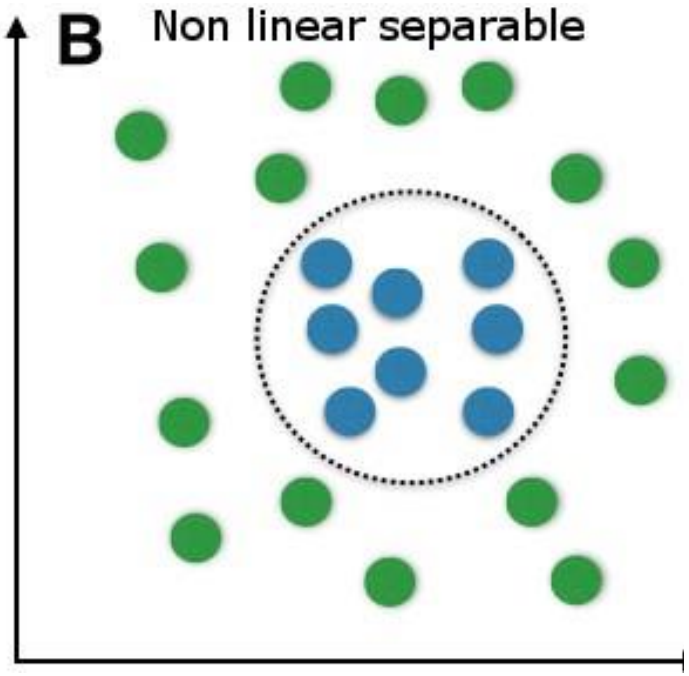
- **Batch Gradient Descent**: Train ***whole*** dataset and find the error then apply the gradient descent to the ANN to update the weights. It takes long time to update the model

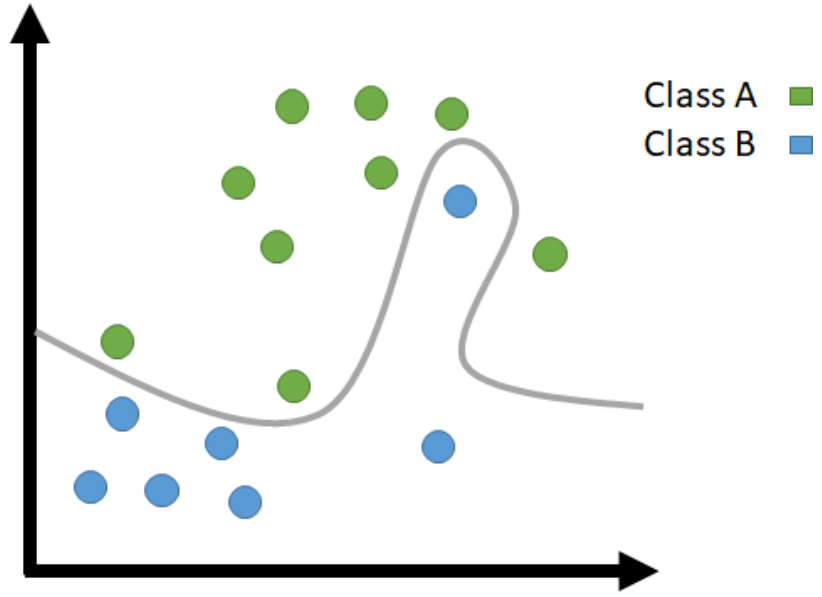
- **Stochatsic Gradient Descent (Online)**: Train ***each*** data point and find the error then apply the gradient descent to the ANN to update the weights. This approach is much faster than Classical Gradient Descent.
  - E.g. 1: you don't have all the training dataset beforehand. Your training examples arrive one by one over time, as a stream.
  - E.g. 2: your training dataset is very big (computationally expensive to use batch GD, or the dataset doesn't fit in memory)."

- **Mini-Batch Gradient Descent**: Train ***some*** data in the dataset and find the error then apply the gradient descent to the ANN to update the weights. This approach is achieves the optimal weigh result fast and efficiently.

Multi-layers neural networks

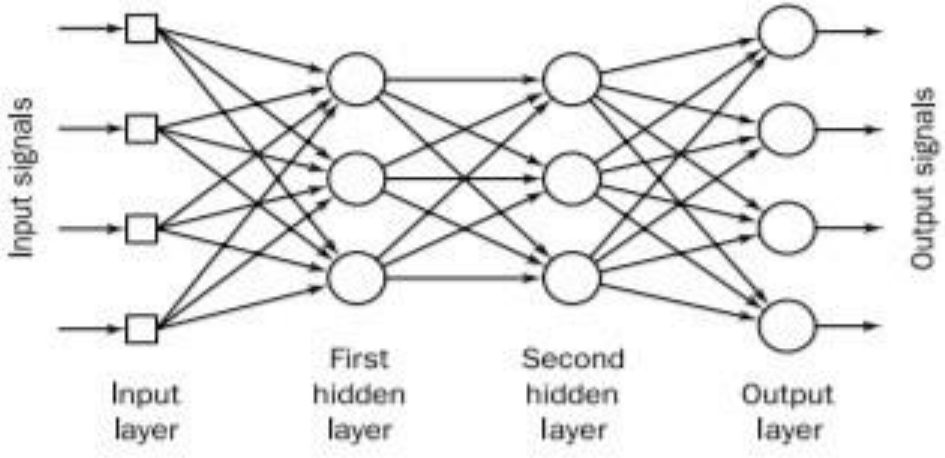






Multi-layers neural networks

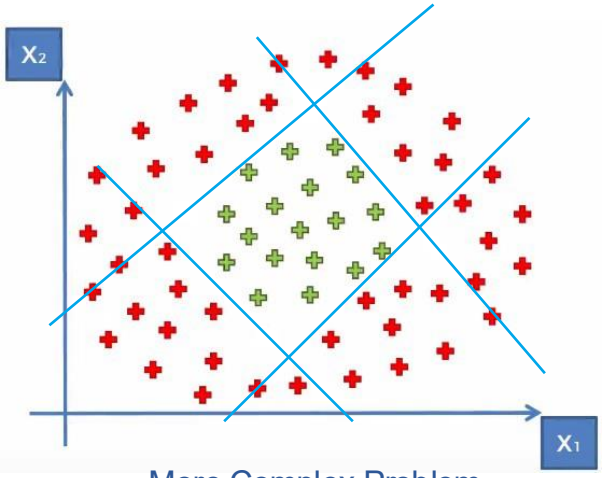
- Multi-layers NNs are more **complex models** than the single perceptron model.
- These network models consist of **number of perceptrons**.
- They have an **input layer**, **hidden layer(s)** and **output layer**.
- The learning algorithm is based on
  - **Feed-forward**
  - **Back-propagation**



Multilayer perceptron with two hidden layers

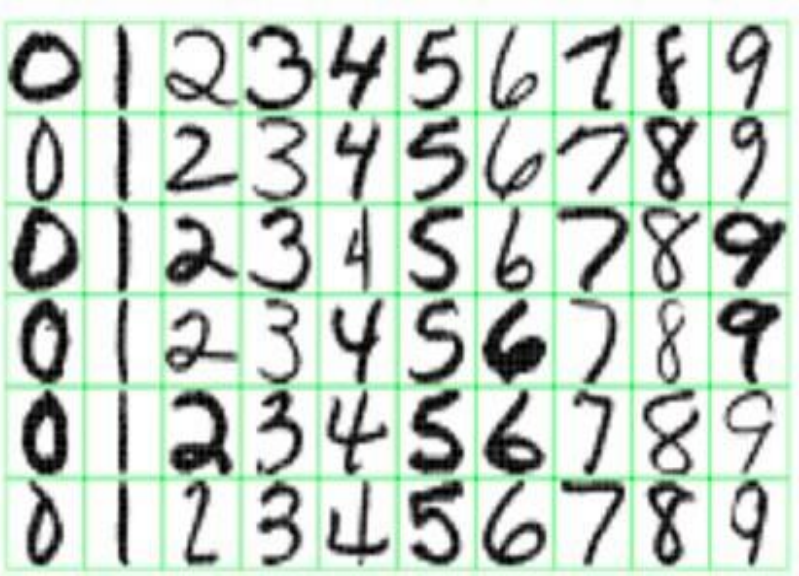
Why Multi-Layers?

- **Units** in hidden layers become **self-organised feature** detectors/**extractors**.
- These intermediate features are **important** for difficult tasks.
  - ► Nodes in **earlier** layers learn **low level** features that are **generic**.
  - ► Nodes **deeper** layers learn **higher level** features that are more **specialized**.

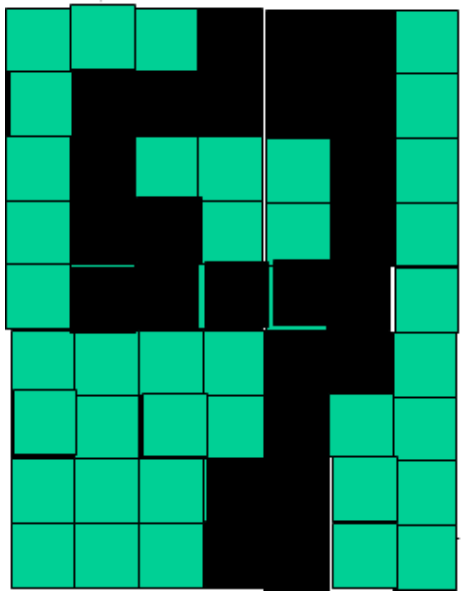


More Complex Problem

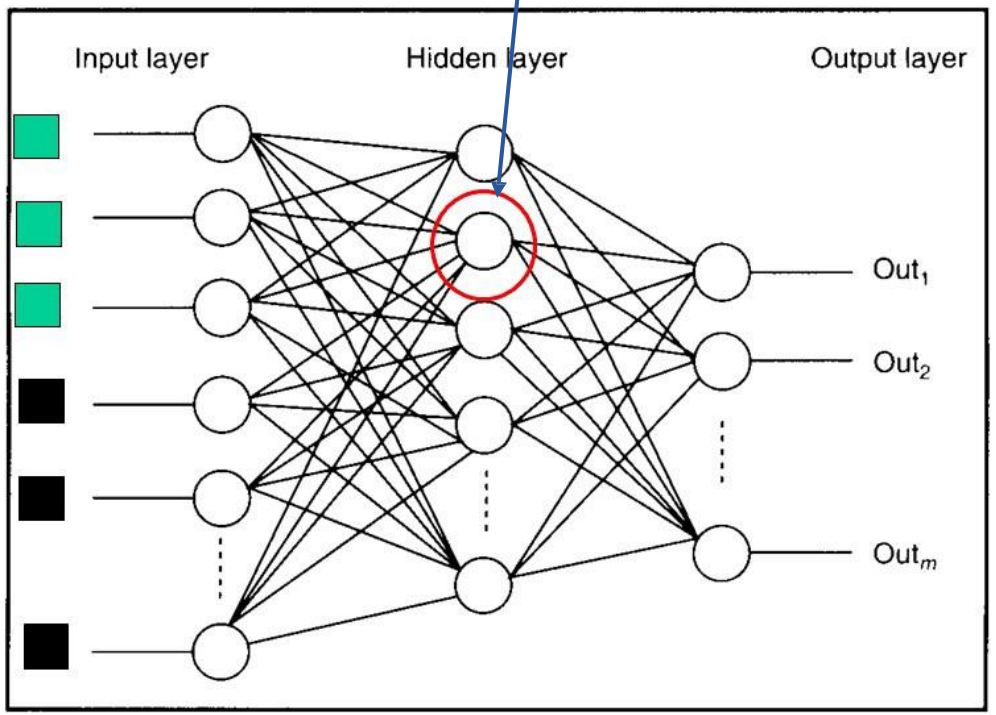
Why Multi-layers?



handwritten digits recognition

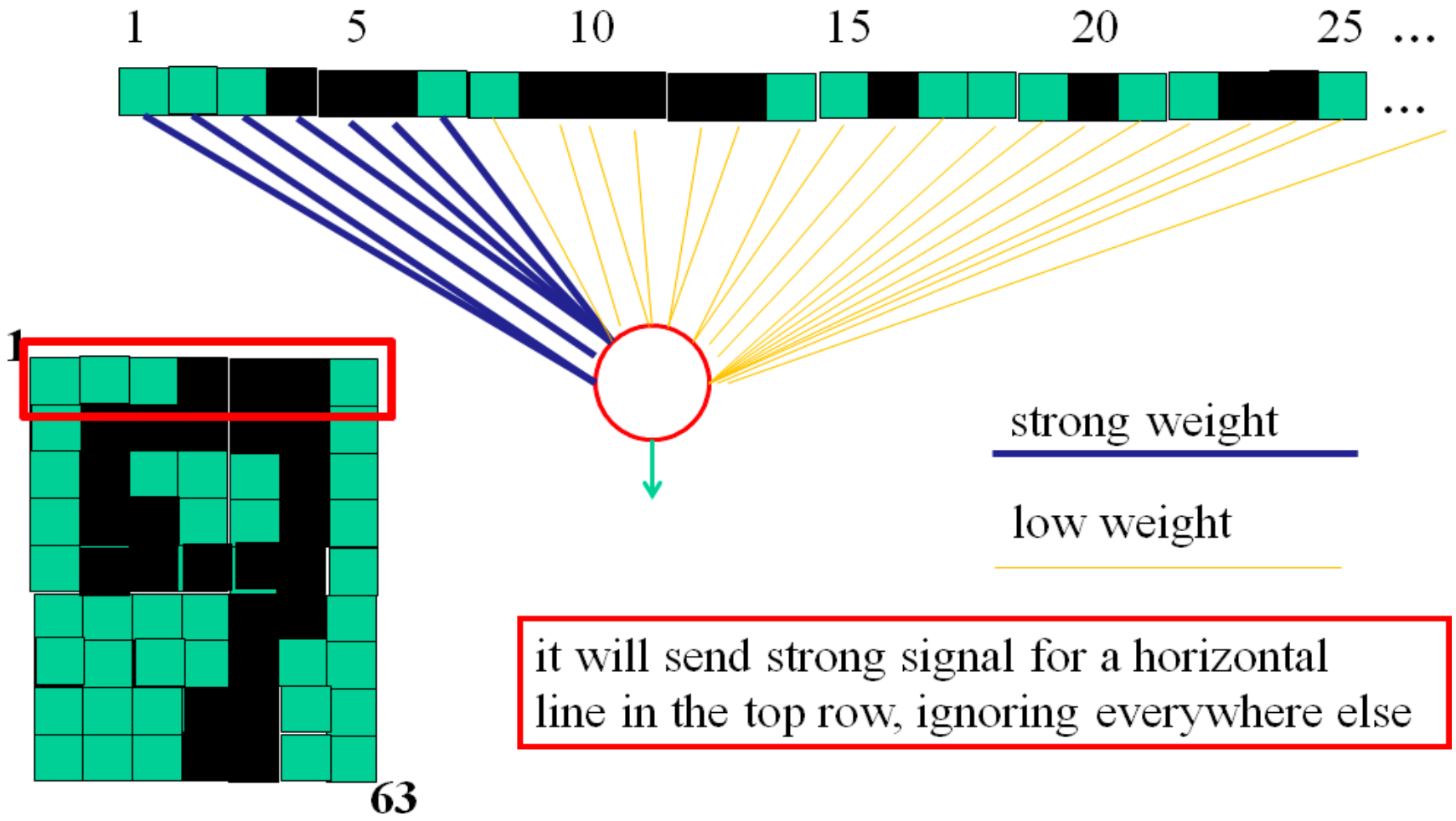


What is this unit doing?



Why Multi-layers?

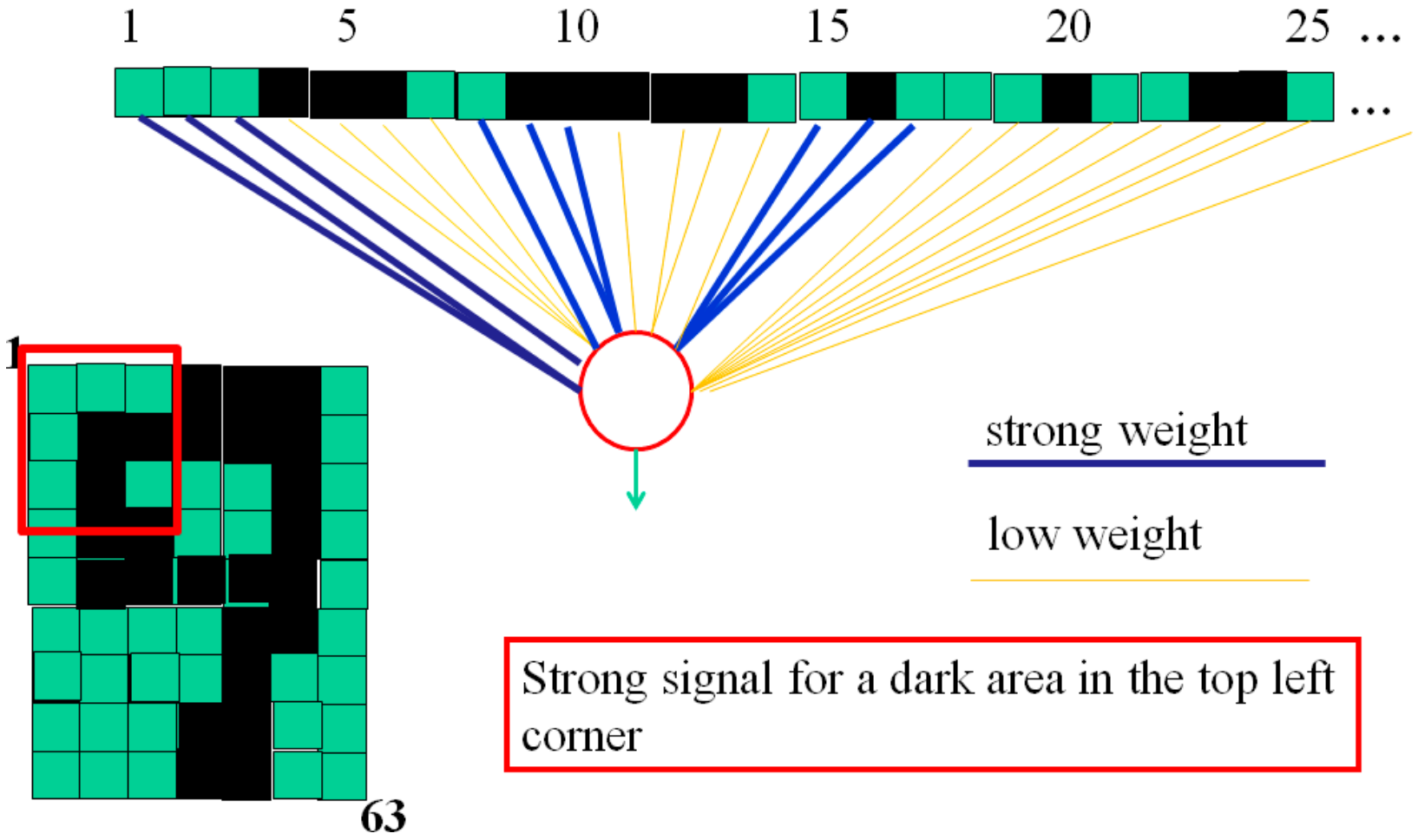
What does this unit detect?



it will send strong signal for a horizontal line in the top row, ignoring everywhere else

Why Multi-layers?

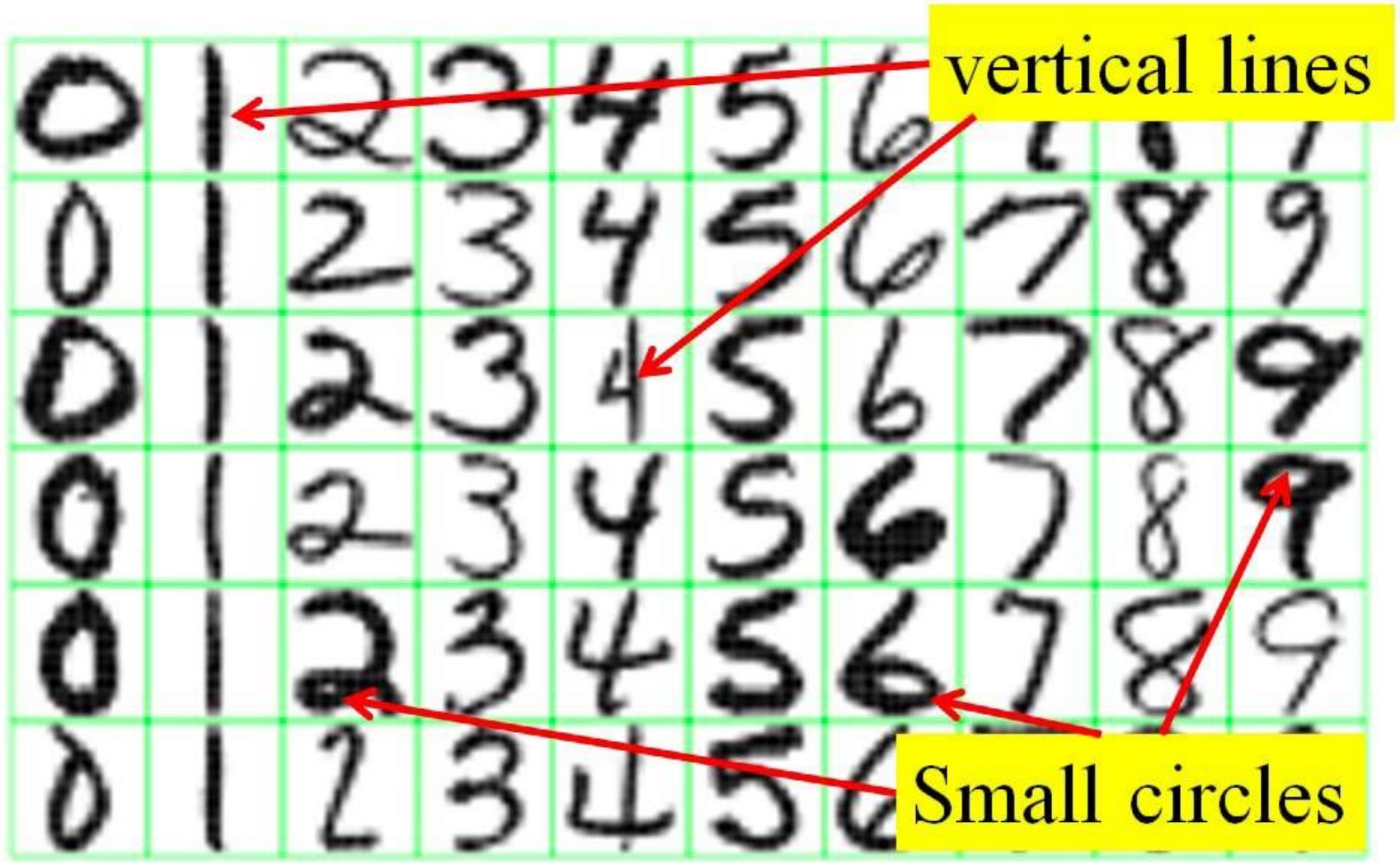
What does this unit detect?



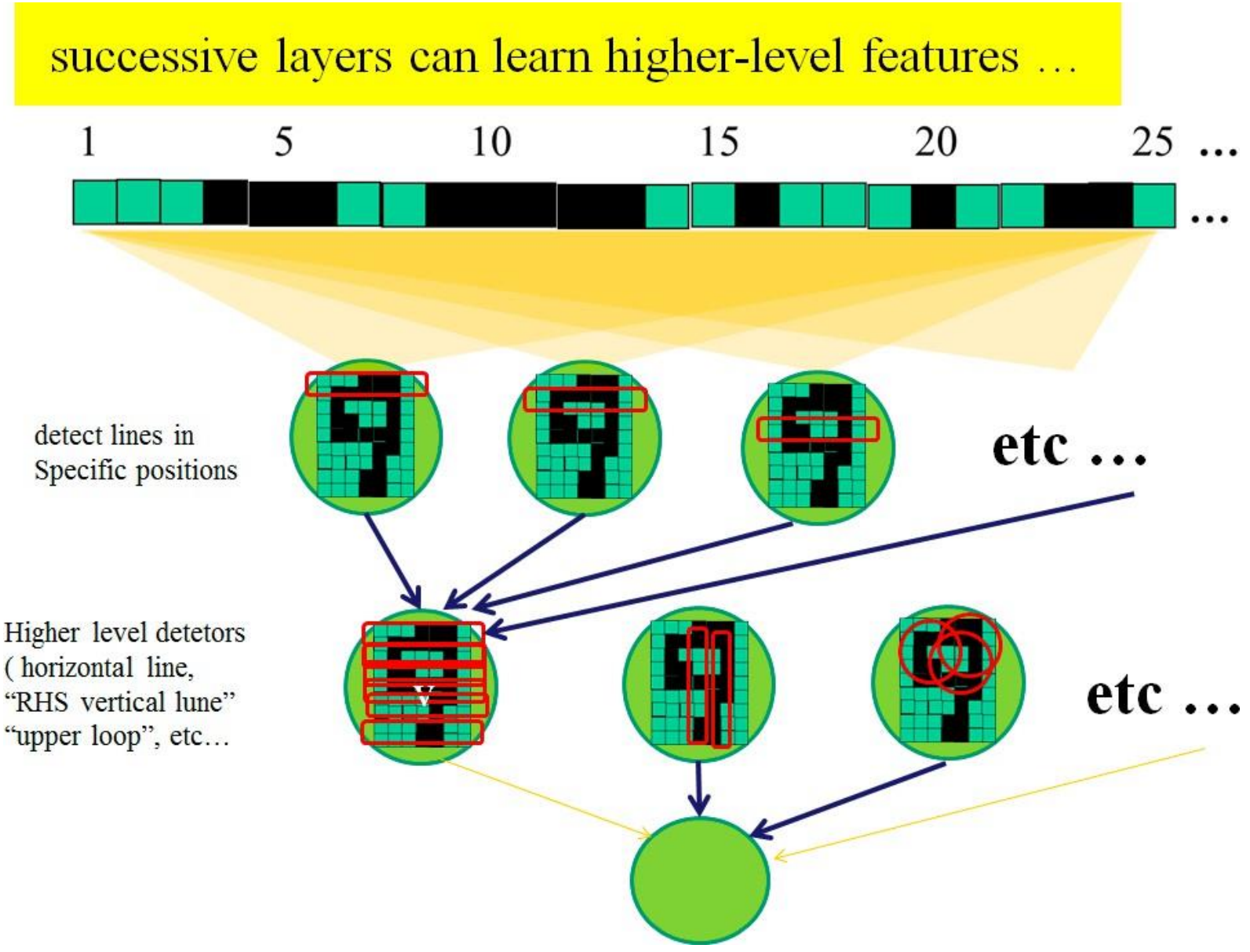
Strong signal for a dark area in the top left corner

Why Multi-layers?

These are some low-level features …



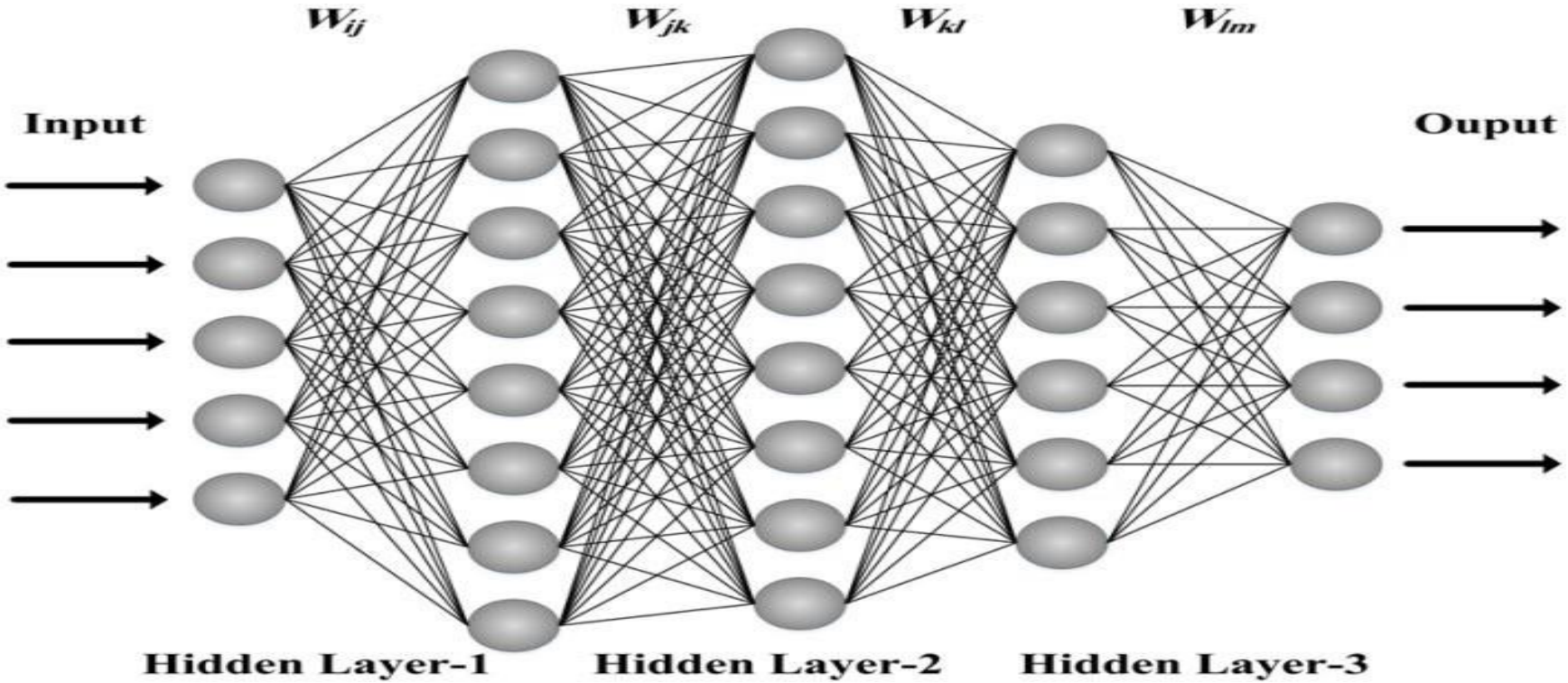
Why Multi-layers?



Multi-layers neural networks

So, multiple layers make sense …

- Many-layer neural network architectures are capable of learning higher and higher level features …



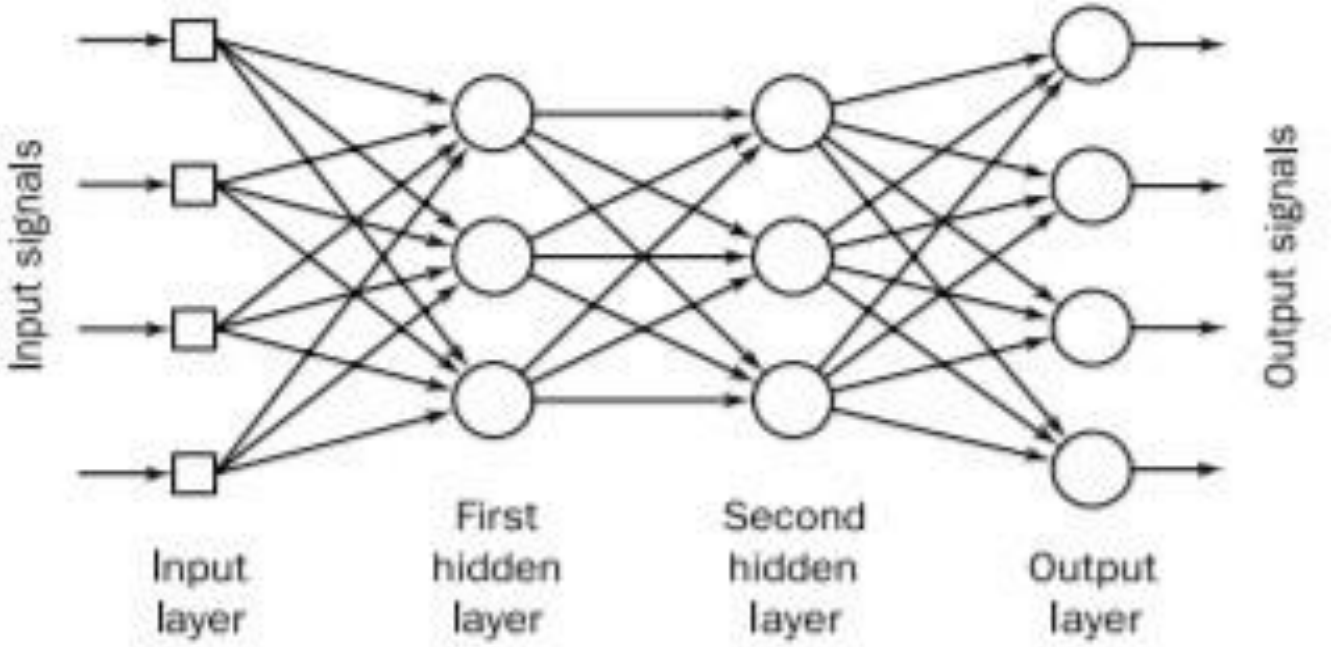
Multi-layers neural networks

Why is a middle layer in a multi-layer network called a 'hidden' layer? What do these layers hide?

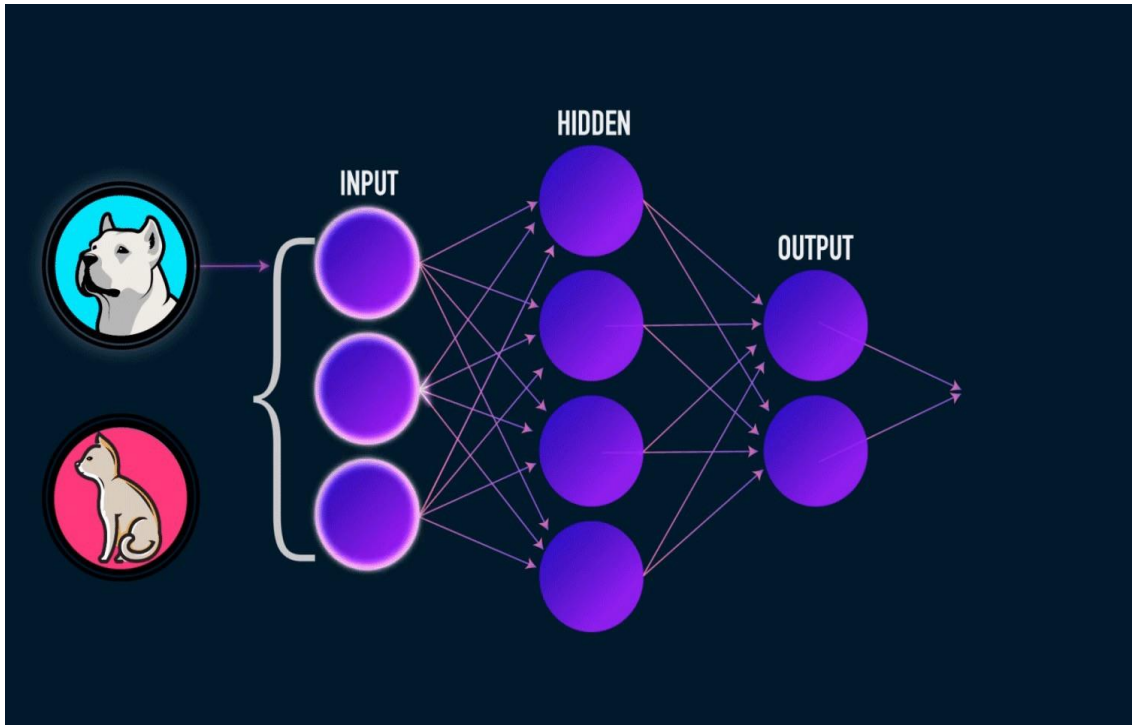
- A hidden layer '**hides**' **its desired output**.
- Neurons in the hidden layer **cannot** be **observed through the input**/output behaviour of the network.
- There is **no obvious** way to know **what the desired output** of the hidden layer **should be**.

Feedforward neural network

- A **feedforward NN**is an artificial neural network wherein connections between the units do *not* form a cycle.
- Data enters at the inputs and passes through the network, layer by layer, until it arrives at the outputs.
- During normal operation, that is when it acts as a classifier, there is no feedback between layers.
- This is why they are called *feedforward* NNs.

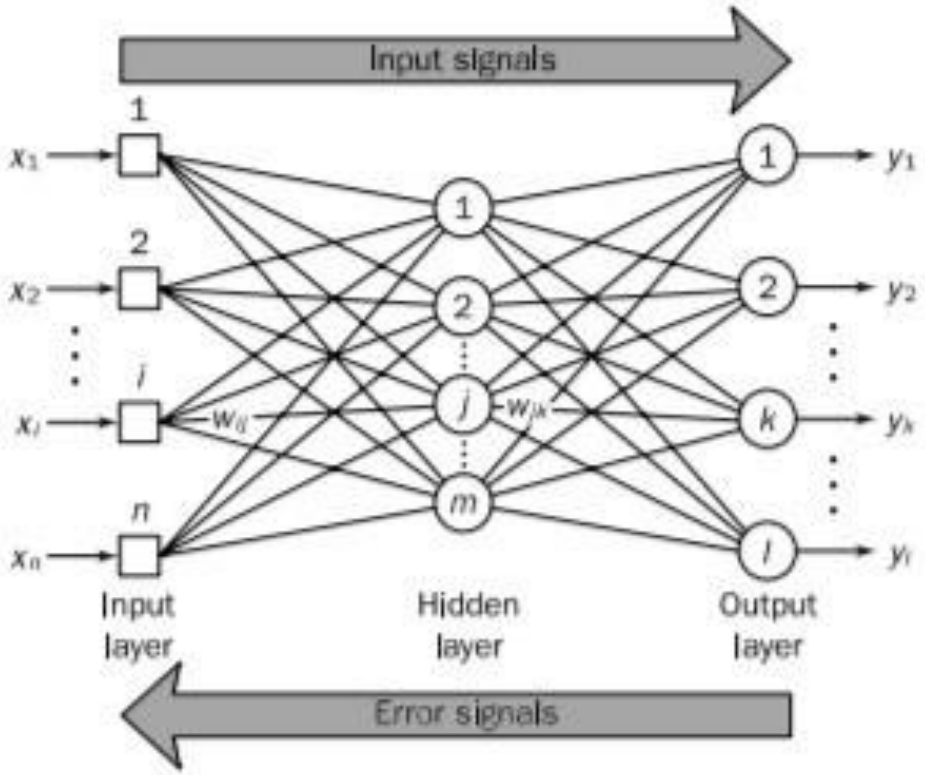


Multilayer perceptron with two hidden layers

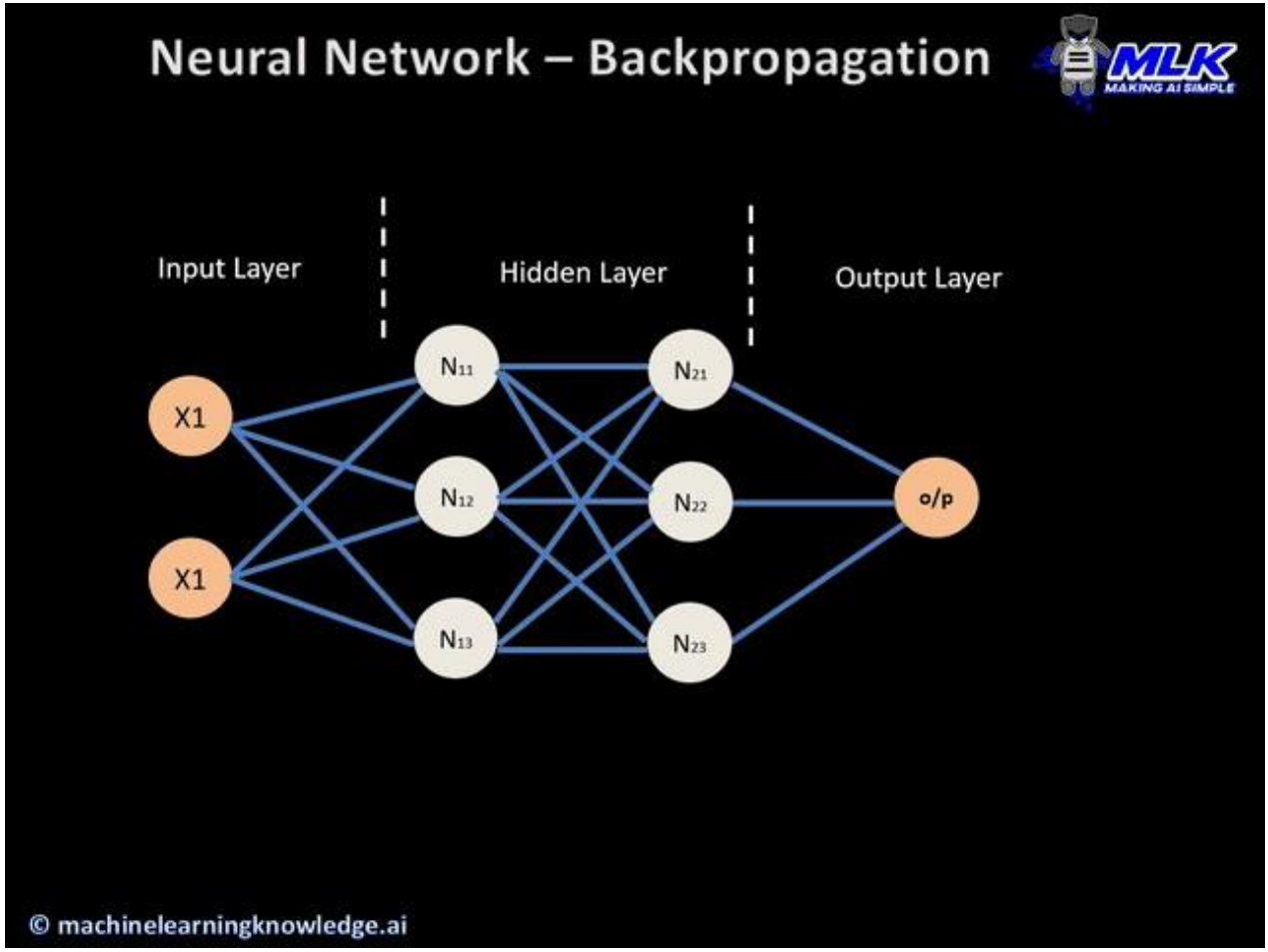


Back-propagation Learning

- To derive the back-propagation learning law, let us consider the three-layer network shown below.
- Input signals, x_1; x_2; ... ; x_n, are propagated through the network from left to right, and error signals, e_1; e_2; ... ; e_n, from right to left.



Three-layer back-propagation neural network



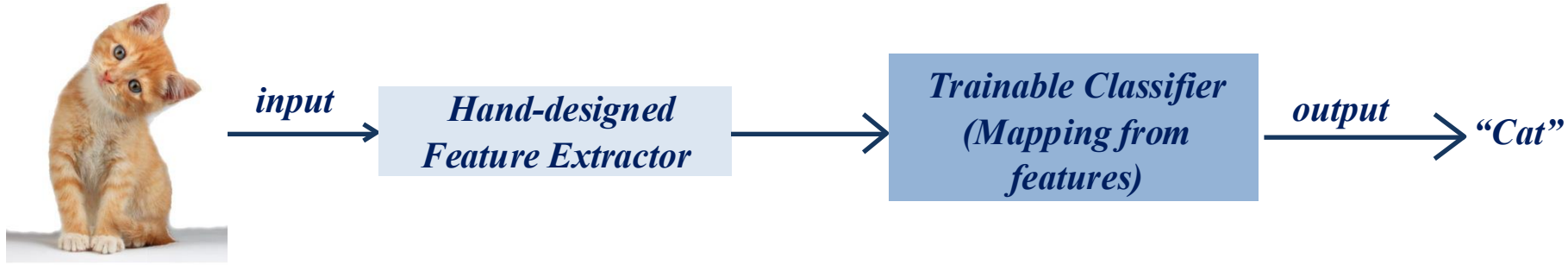
https://machinelearningknowledge.ai/wp-content/uploads/2019/10/Backpropagation.gif

Back-propagation Learning

1. To propagate error signals, we start at the output layer and work backward to the hidden layer.
2. Error is estimated by subtracting desired and actual outputs.
3. Weights are updated.

Deep Learning

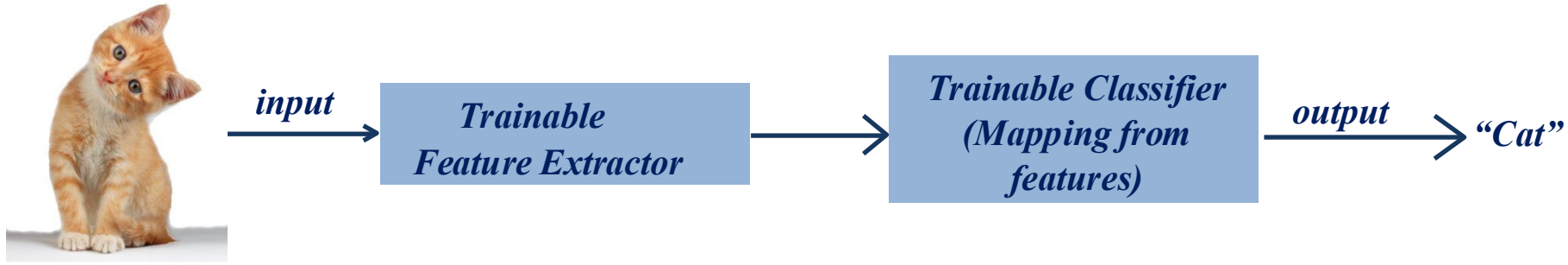
Artificial Neural Networks & Deep learning



- ML methods depend heavily on the representation of given data
  - Parameter tuning is needed
- **Simple ML methods cannot perform *complex recognition tasks, i.e. generalization problems!***

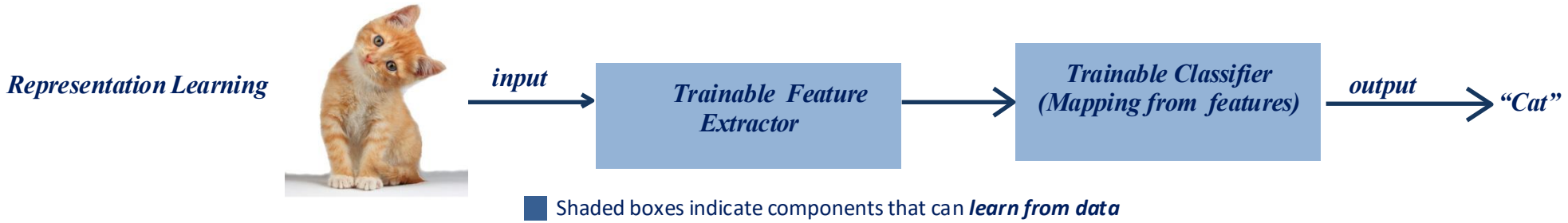
■ ***Solution:***

– Use ML to ***learn not only mapping*** from representation to output but ***representation itself (i.e. features).*** This is known as ***representation learning.***



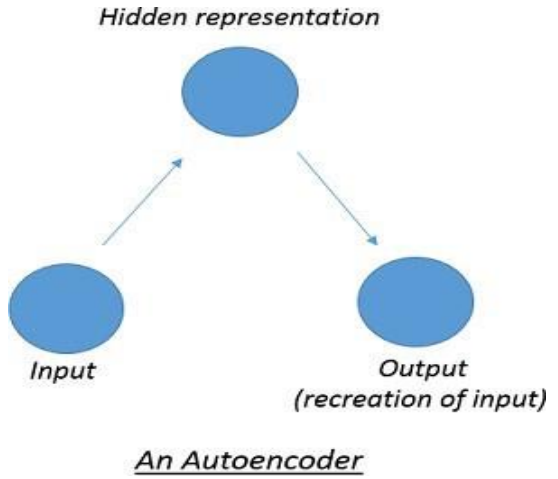
Shaded boxes indicate components that can ***learn from data***

Artificial Neural Networks & Deep learning

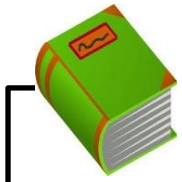


- This **replaces manual feature engineering in ML** and allows a machine to both learn the features and use them to perform a specific task.
- A representation learning algorithm can **discover** a good set of features for a task (feature learning)

- A classical example of representation learning is an **Autoencoder**, which involves an **encoder** to convert the input into a **latent representation**, from which a **decoder** re-creates the original input.



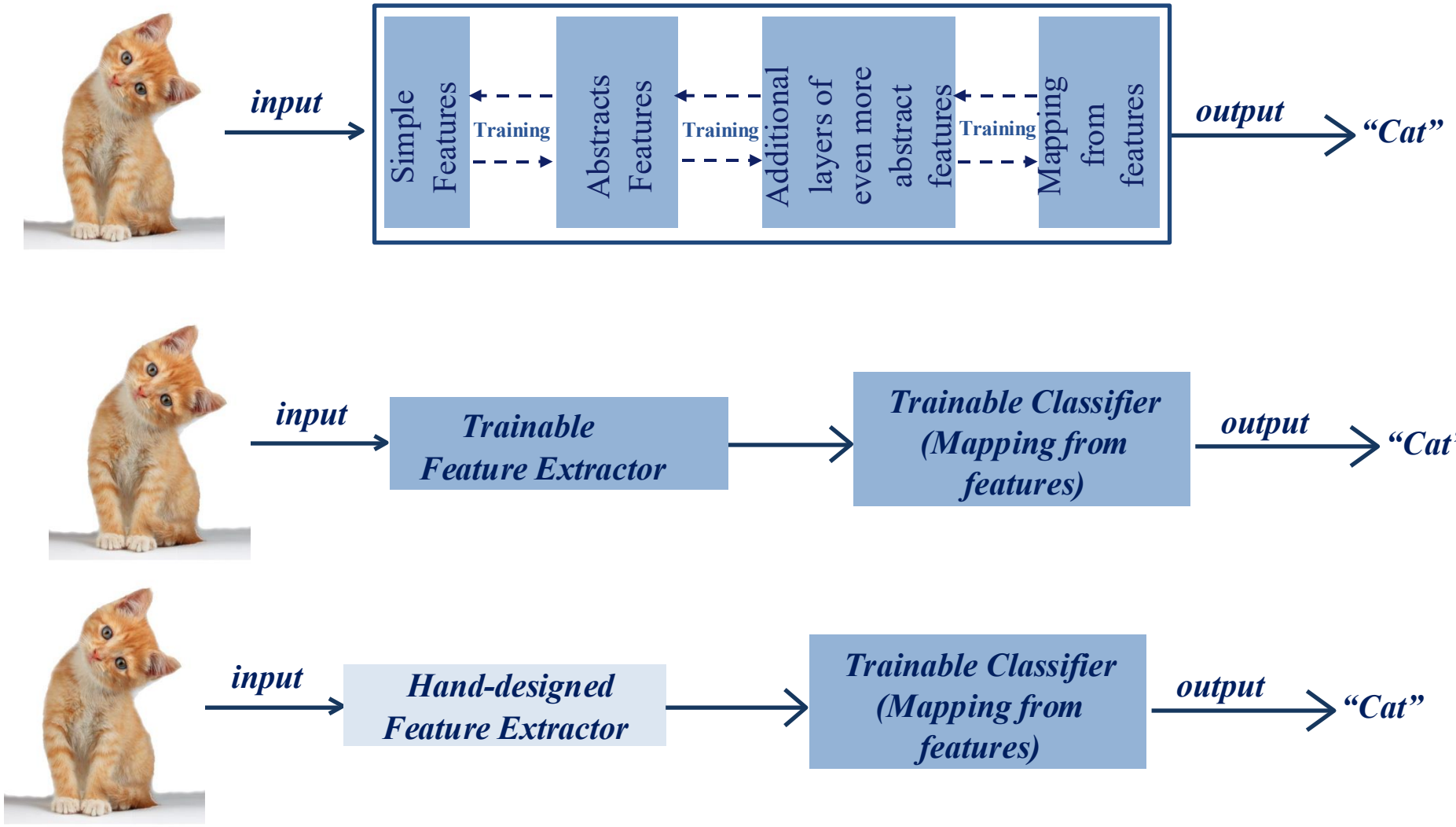
An Autoencoder



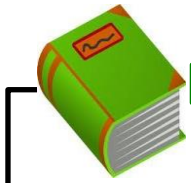
Definition: Representation Learning

(a.k.a. feature learning) allows a machine to be fed with ***raw data*** and **automatically discover** the **representations** needed for detection or classification tasks.

Artificial Neural Networks & Deep learning

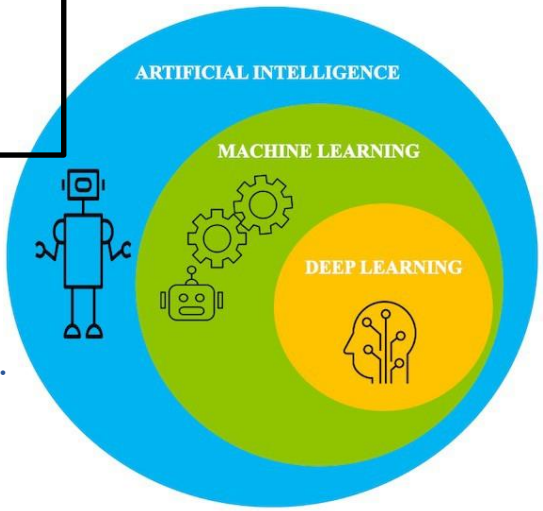


Artificial Neural Networks & Deep learning

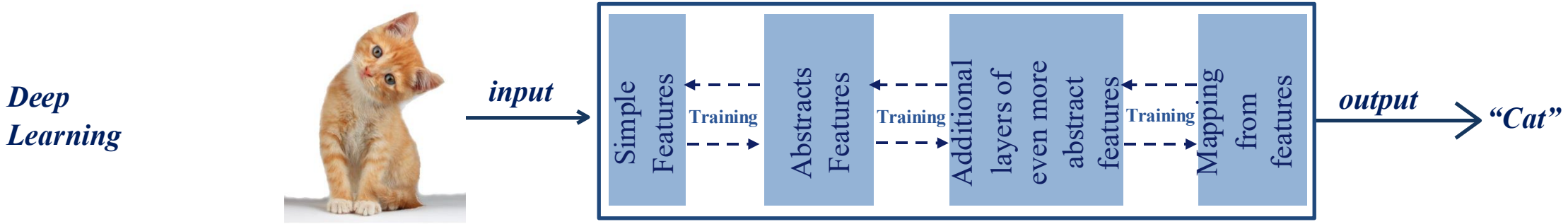


Definition: Deep Learning

- Is a type of **representation learning** where the learned features are **compositional or hierarchical**.
- Is an algorithmic tool that leverages ***Artificial Neural Networks (ANN)*** as ***nonlinear function approximators.***



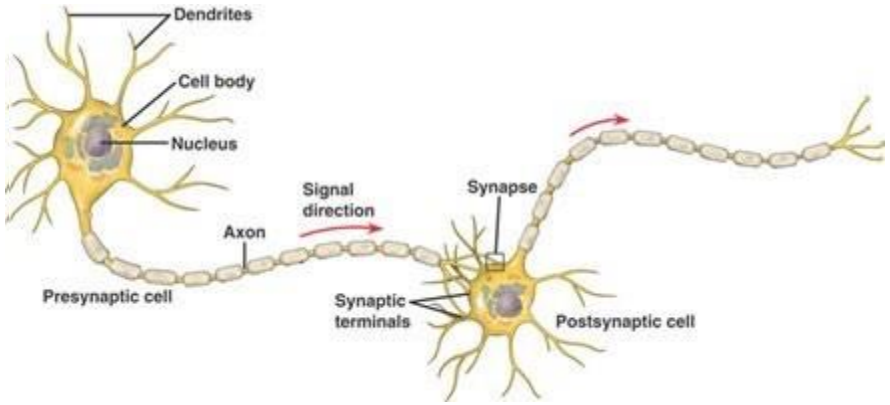
- Exceptional effective at **learning features directly from raw data** (no more feature engineering)
- Attempts to learn **(multiple levels of) representation** by using a **hierarchy** of multiple layers
  - The first layer represents low-level features, e.g. edges at particular orientations and locations in the image.
  - The next layer detects mid-level features, such as motifs, by spotting particular arrangements of edges.
  - The subsequent layer captures high-level class-specific features, e.g. object pars.



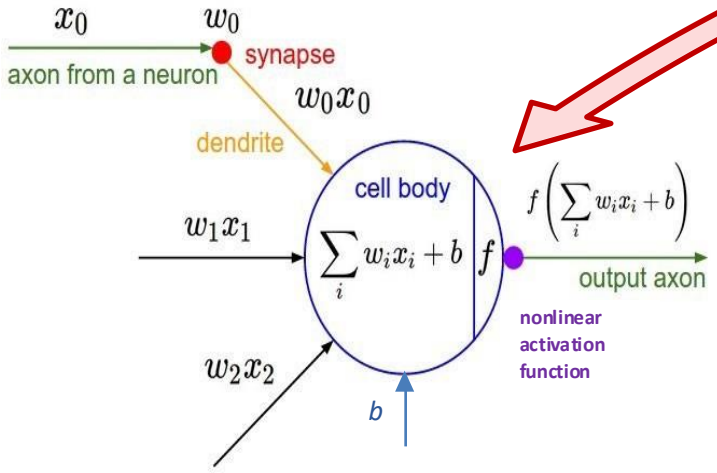
Artificial Neural Networks & Deep learning

- The brain has **~100 billion** neurons and **~1000 trillion** synaptic connections.
- A neuron can **send electrical** pulses, which then **excite** the connected neuron. If this neuron gets **sufficiently excited**, it will further **transfer** electrical pulses to the **next** one.

The term **DEEP** comes from the **scale of the network.**

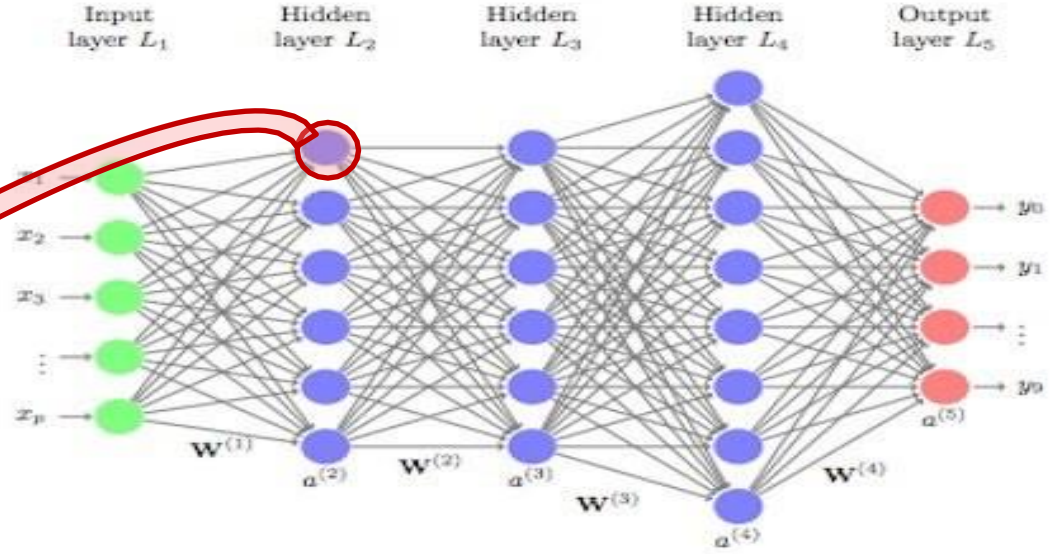


Biological Neuron



Artificial Neuron

Input can be in any arbitrary length

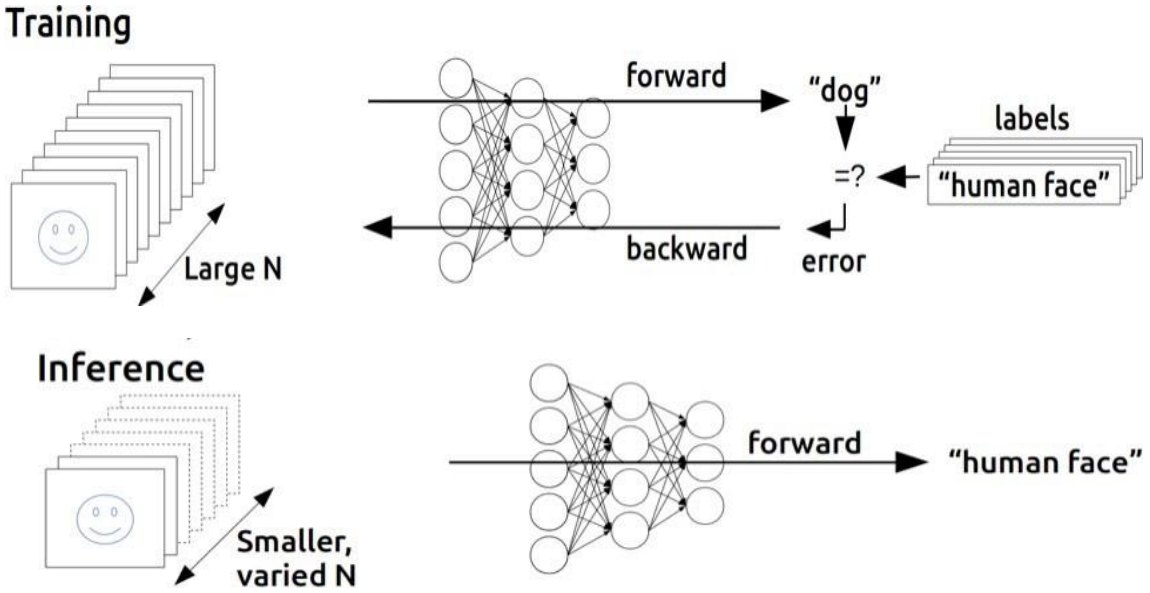


Artificial Neural Network

Deep learning

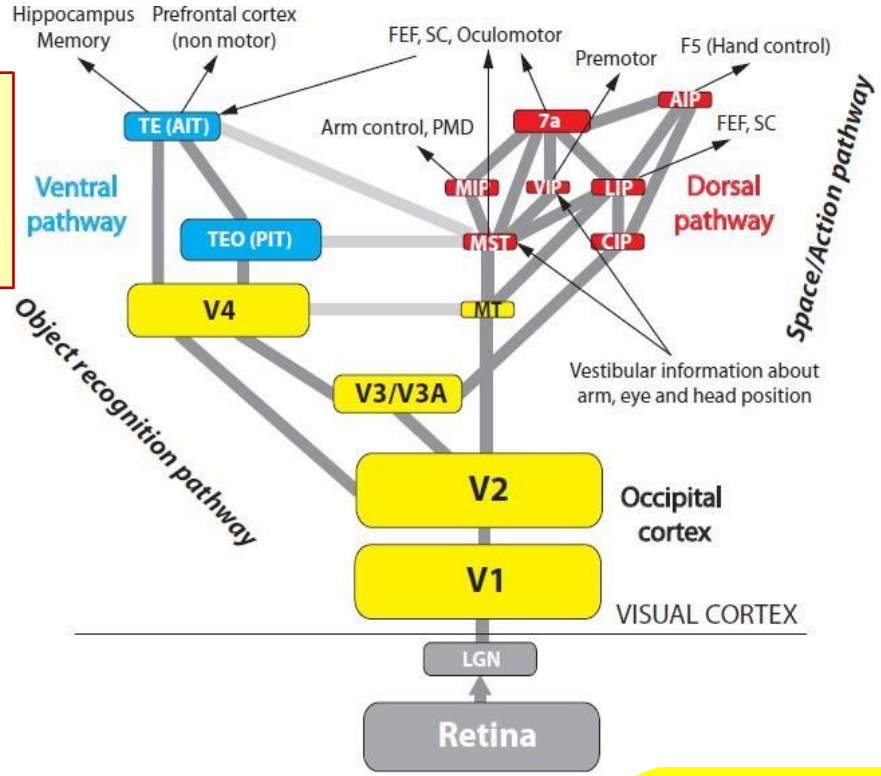
- Deep Learning means a mathematical model using a neural network with several layers, each computes ***non-linear input-output mappings***; thus, ***Deep Neural Networks (DNNs)*** refer to *ANNs*.
- In DNNs, ***weighted connections*** between input and output neurons are ***trained*** via ***error backpropagation,*** which propagates the error back through the network's layers and ***updates their weights using gradient descent*** in order to improve the network's performance.
- This way, DNNs ***encode a function*** that ***minimizes the disparity*** between prediction and truth by reaching the optimal weight values in the layers.
- A well-trained DNN holds ***latent hierarchical feature representations in the hidden layers***.

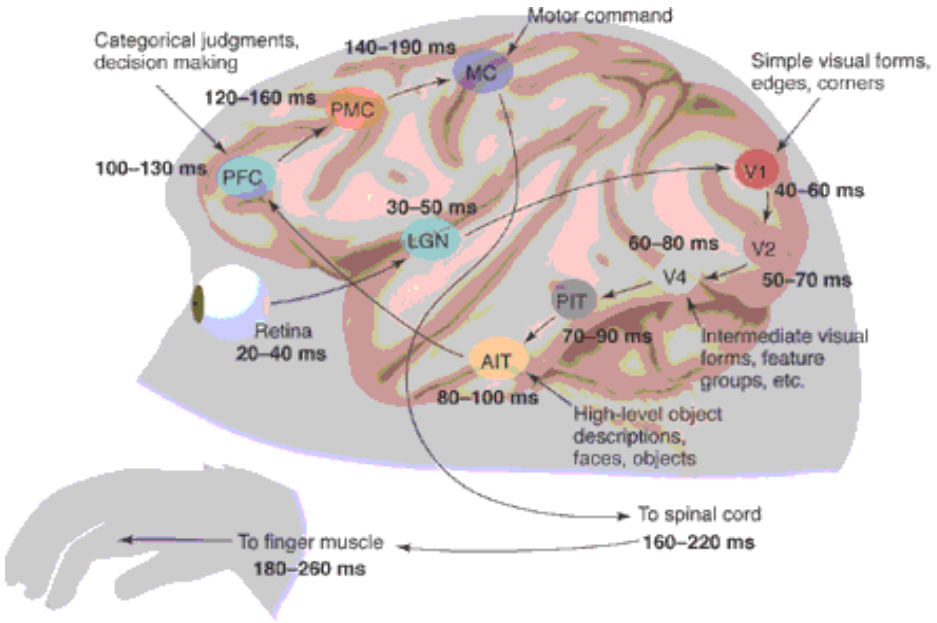
What is hierarchical feature representation?



Deep learning

- The Mammalian Visual Cortex is ***Hierarchical***
  - The ventral (recognition) pathway in the visual cortex





Sub-cortical processing

- Sensory input, contrast computation, etc.
- Contains photoreceptors, LGN, and Ganglion cells which have small receptive fields
- Center-surround receptive fields are sensitive to changes in luminance



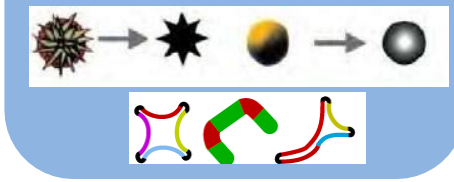
Occipital / Early Vision

- Generic feature processing
- 66% of the Visual Cortex
- Contains cells that respond to edges, bars, colors, contours, etc.
- Computes absolute disparity

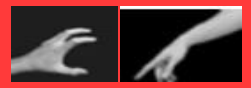


Ventral Pathway

- Object recognition and categorization
- 22% of the Visual Cortex
- Significantly larger receptive fields
- Features more complex than simple bars or spots are extracted



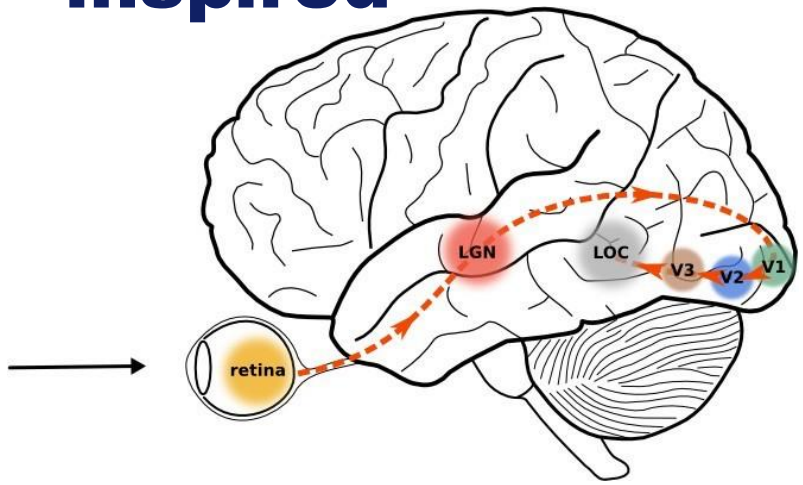
- Both motion and visual information is processed
- Bridges the visual and motor systems.
- Action planning is engaged
- 11% of the Visual Cortex
- Receptive field sizes as well as complexity of features increase

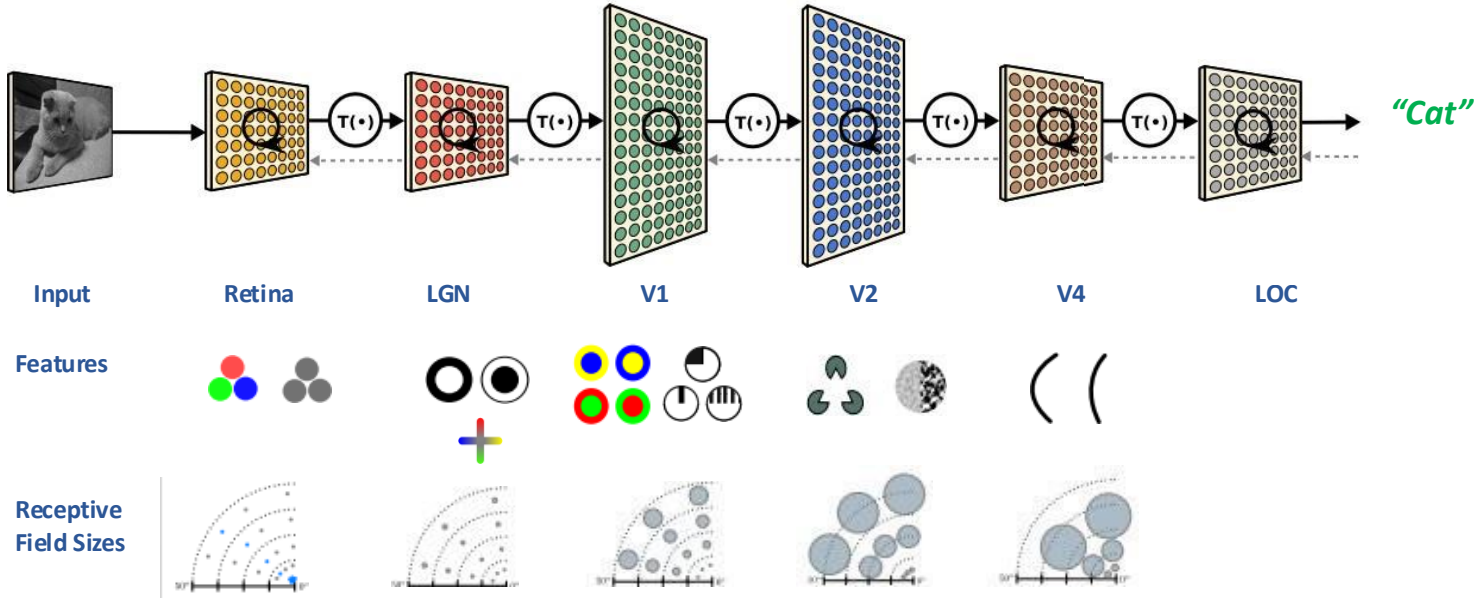


Kruger et al., 2013 "Deep Hierarchies in the Primate Visual Cortex: What Can We Learn for Computer Vision?". *IEEE PAMI*

Deep Learning = Brain "inspired"

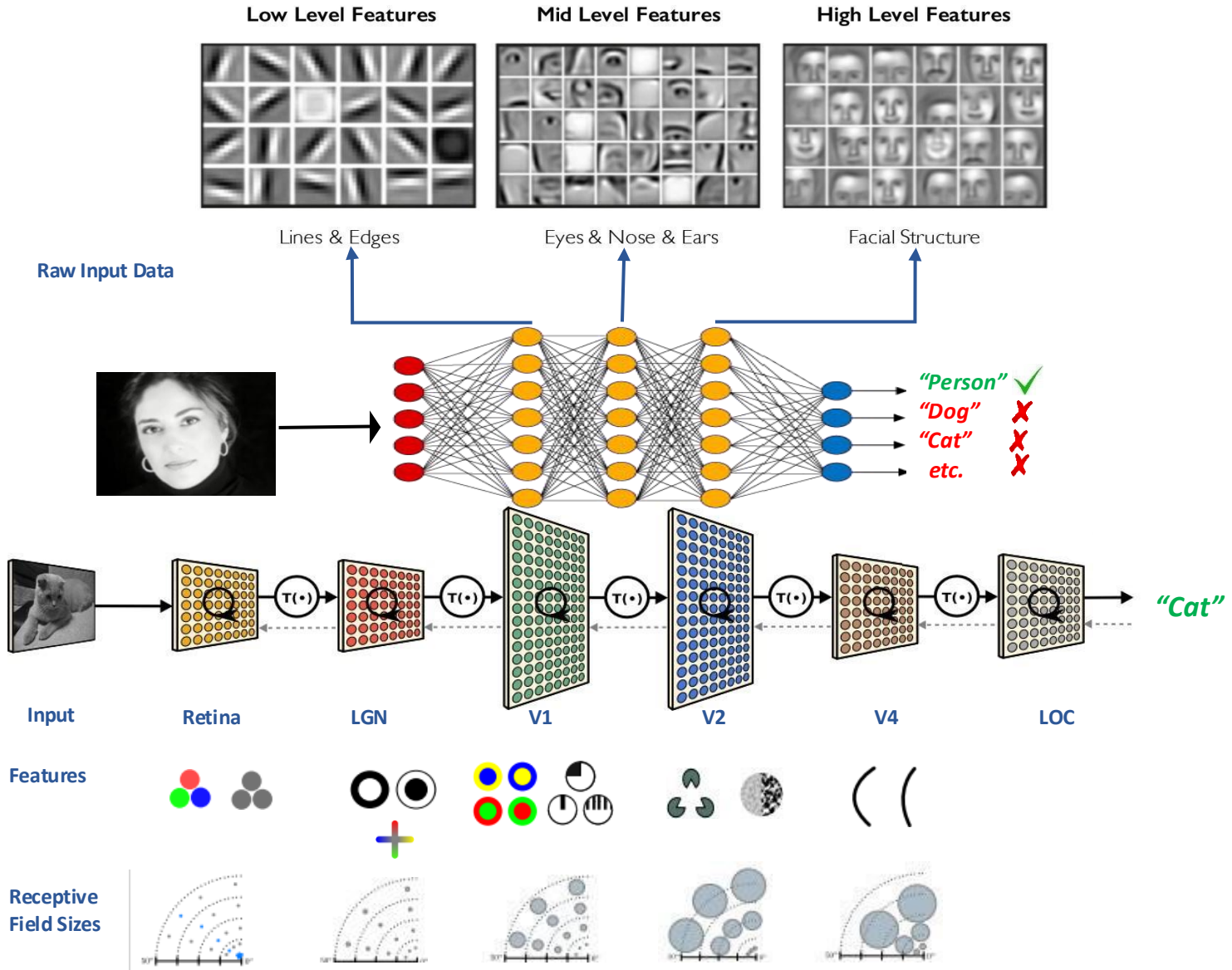
- The Mammalian Visual Cortex is ***Hierarchical***
  - The ventral (recognition) pathway in the visual cortex





Deep learning = Hierarchical Feature Representation

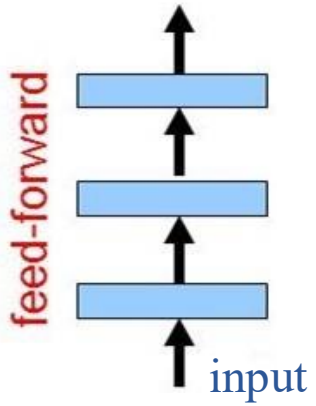
- The series of layers between the input and the output of the neural network do ***feature extraction in a series of stages***, just as our brains seem to do



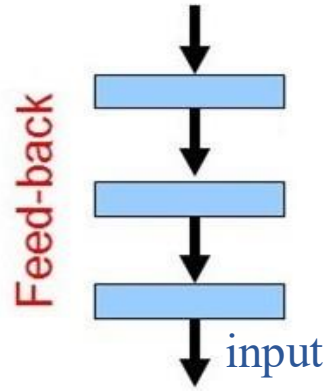
- Note that the receptive field size in each layer is different.
- In higher layers, it is bigger. Features in the higher levels are from bigger regions.
- Due to the sake of simplicity, features are shown in the same size, i.e. scale.

Deep Architectures

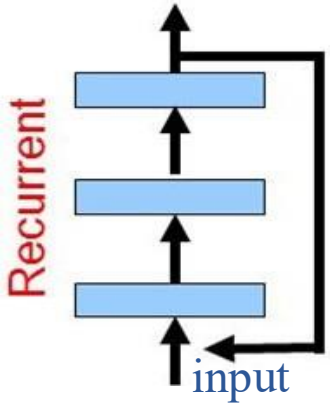
- Feed-forward
  - Neural nets
  - Convolutional Neural Networks



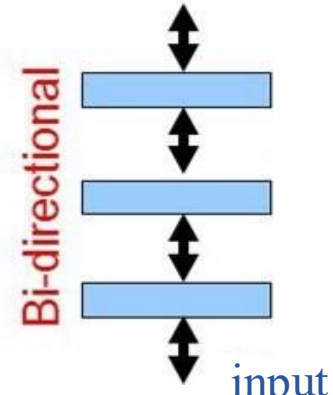
- Feed-back
  - Deconvolutional Networks

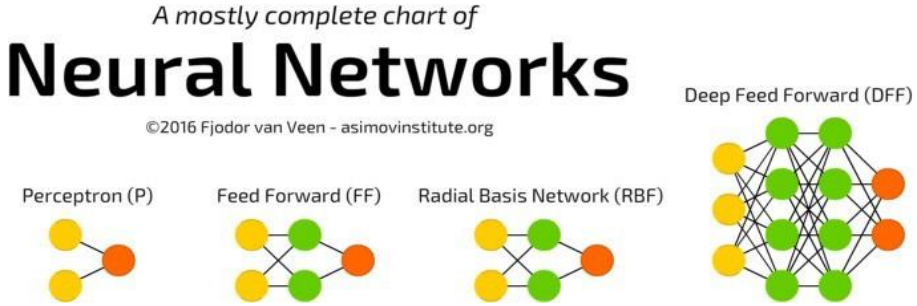


- Sequence based
  - Recurrent Neural Networks
  - Long-Short Term Memories



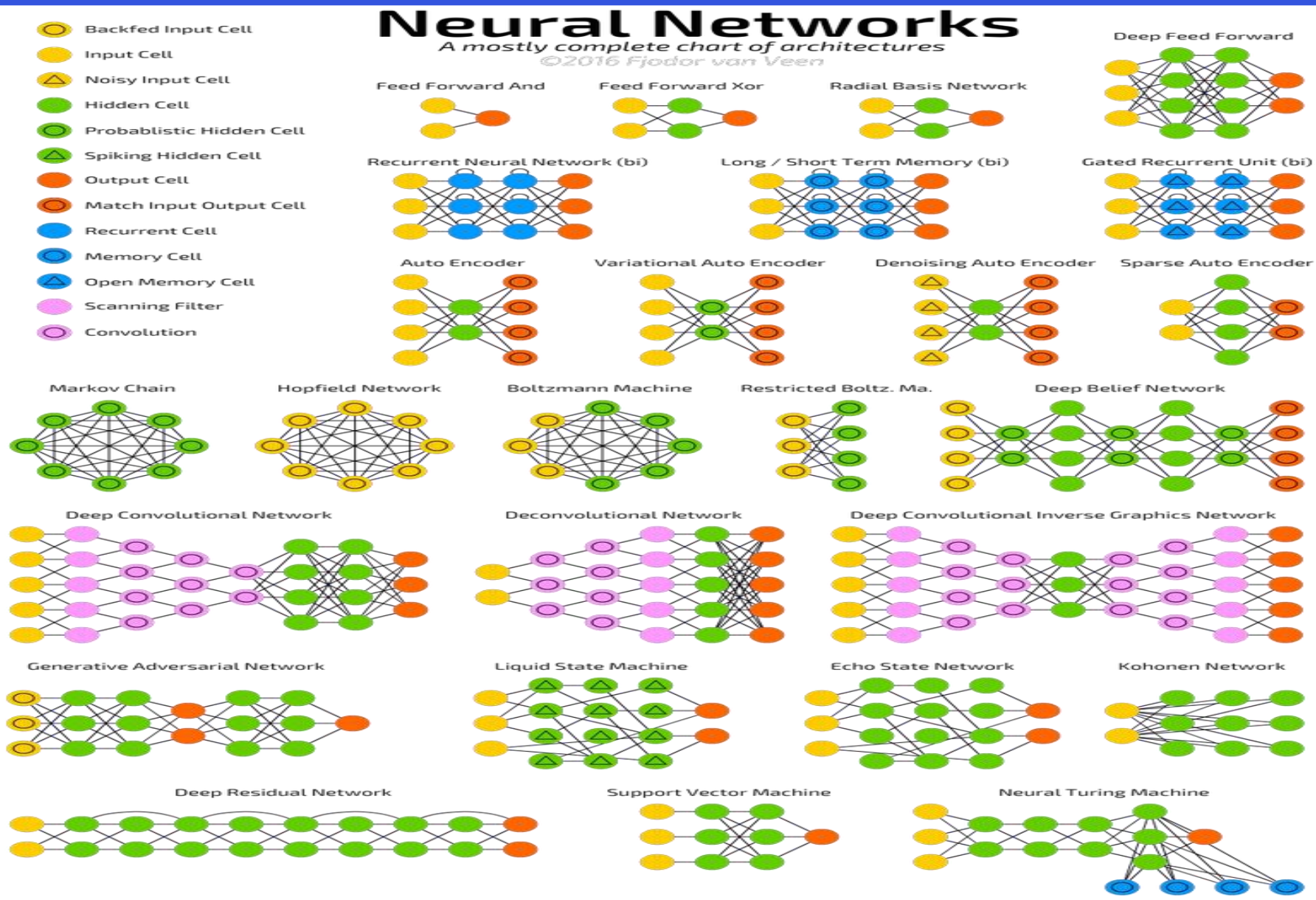
- Bi-directional
  - Deep Boltzmann Machines
  - Stacked Autoencoders





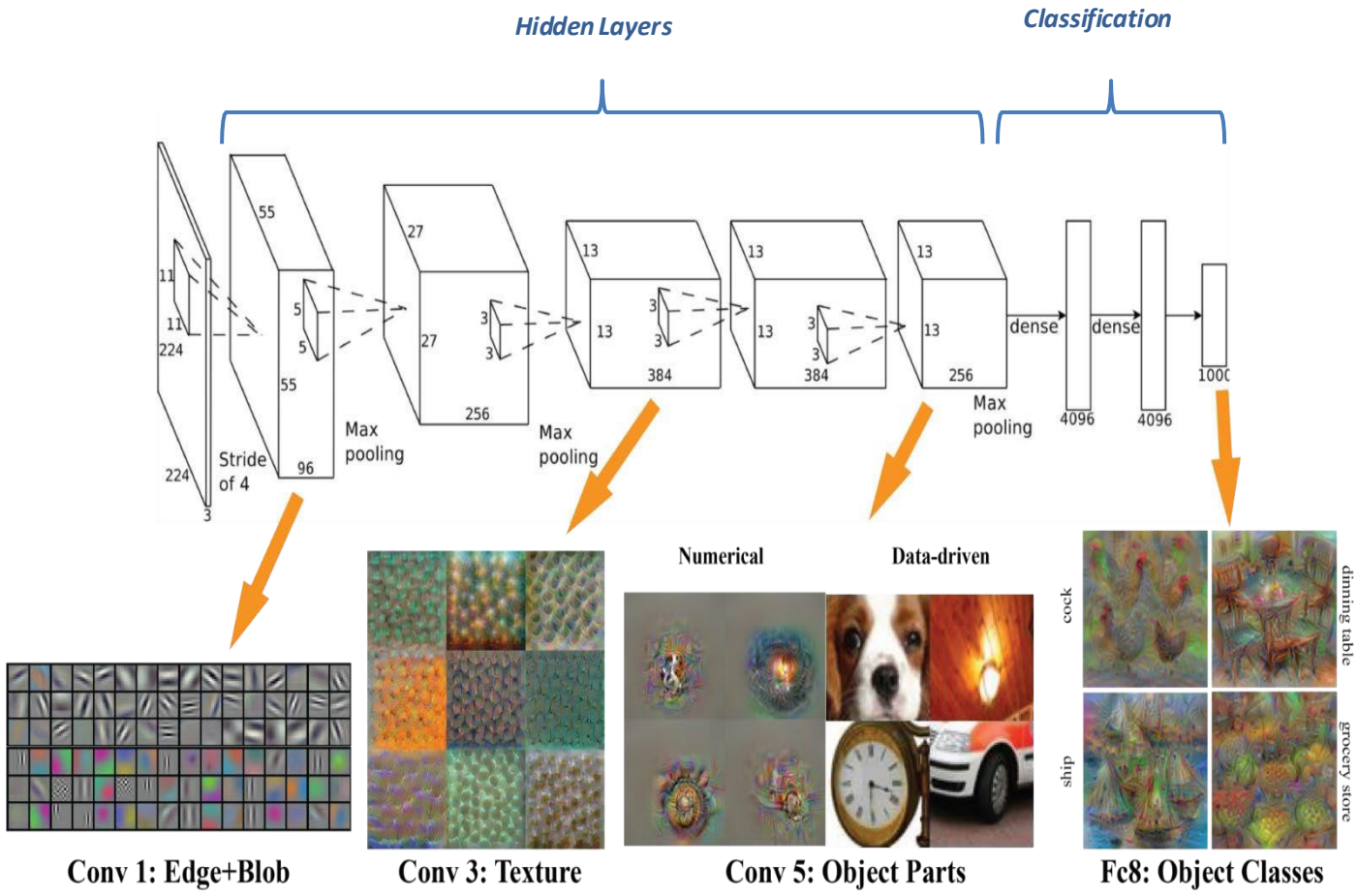
Follow this link to see ***the mostly complete chart*** of neural networks!

Deep Architectures



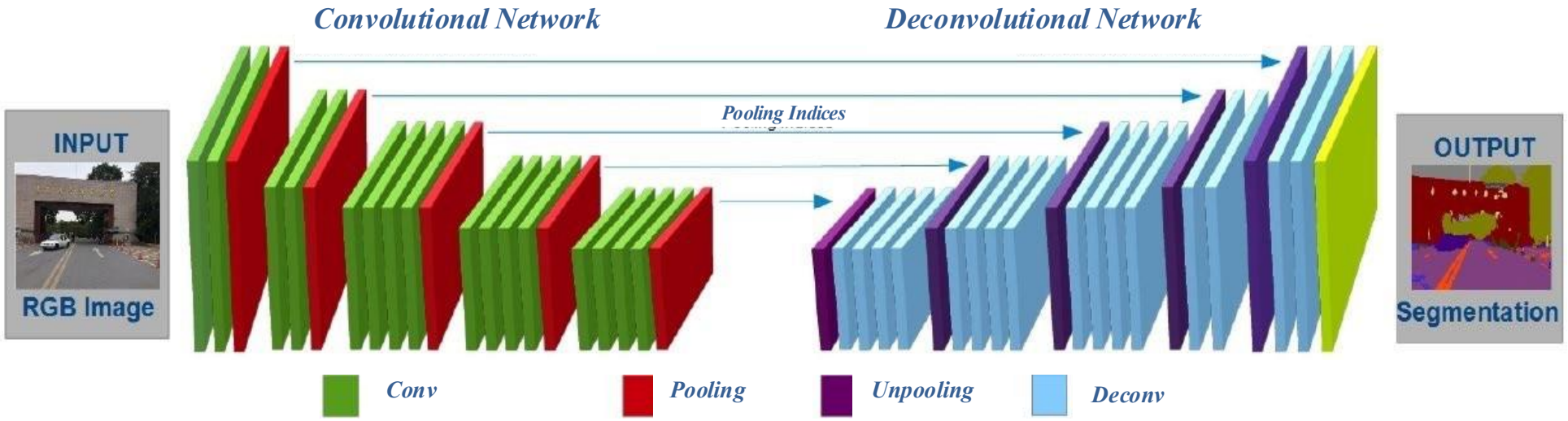
Convolutional Neural Networks (CNNs)

- Feed-forward network architecture
- Inspired by the human visual cortex and learns a hierarchy of visual features
- Local pixel-level features are scale and translation invariant
- CNNs (a.k.a. “ConvNets”) learn the *“essence”* of visual objects and generalizes well
- CNNs are designed to process data that come in the form of multiple layers (e.g. RGB images)
- Convolution layer is a feature detector that automatically learns to filter out irrelevant information from input by using a convolutional kernel
- Pooling layers compute the max value of a particular feature over a region of the input data. Also reduces memory size.



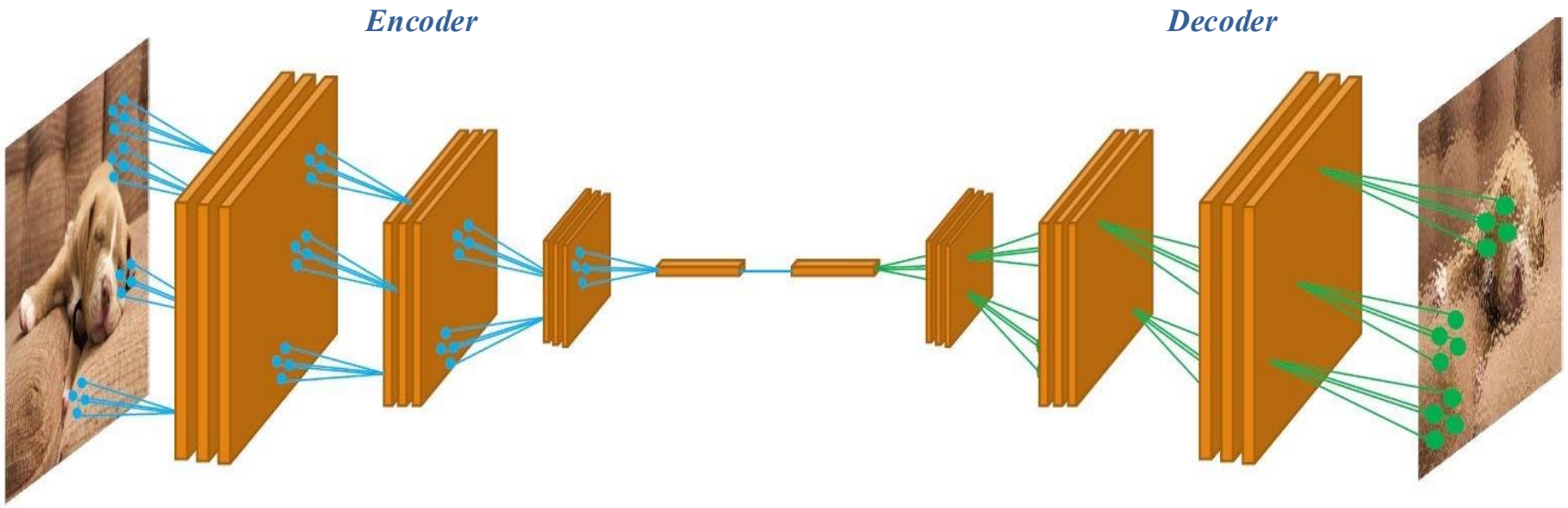
Deconvolutional Neural Networks

- Feed-back network architecture
- Deconvolutional networks (“DeConvNets” for short) map learned features back to, for instance, pixels when modeling images
  - Opposite of what a normal convolutional layer does.
- The unpooling operation is a way to approximate the inverse of the max pooling operation.
  - During the forward path, the locations of the maxima in the max pooling operation are saved to be used in the unpooling operation to place the reconstructions in the appropriate locations
- The filter operation in DeConvNets is used to inverse the filter operation of the forward path.
  - Therefore, it uses the transposed version of the filters of the forward path, which means simply flipping the filter horizontally and vertically.



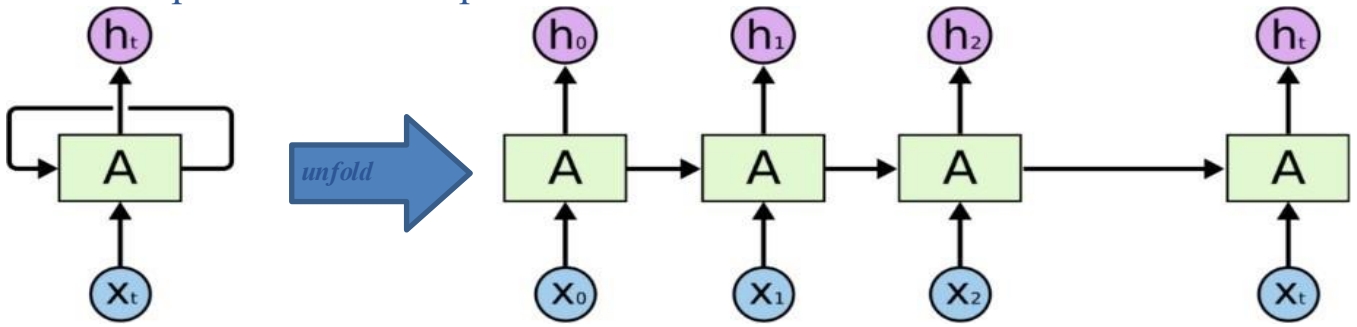
Autoencoder Networks

- Bi-directional network architecture
- Takes a set of typically unlabeled inputs, and after encoding them, tries to reconstruct them as accurately as possible
  - Has encoder-decoder structure
  - Predicts at the output the same input data
- Dimensionality reduction:
  - Uses hidden layer as a feature extractor of the desired size
  - Acts as a feature extraction engine
  - You can omit the output layer after training and use the hidden layer as input features for classification

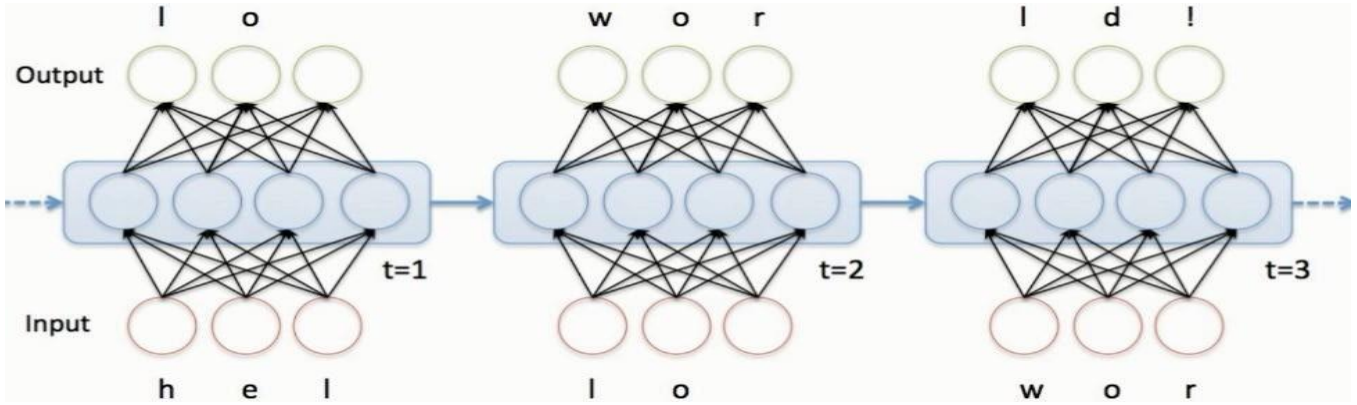


Recurrent Neural Networks (RNNs)

- RNNs are for modelling time series, i.e. sequence of data.
  - They can learn the pattern that frequently appears in data and also can learn to predict what comes next if they partially see that pattern learned before.
  - RNNs can learn to map input sequences to output sequences (flexible-sized vectors).
- The structure of an RNN is similar to feedforward neural networks, with the distinction that it allows a recurrent hidden state whose activation at each time depends on that of the previous time step.



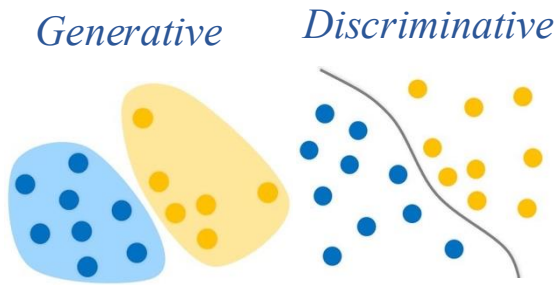
- In this example, three artificial neural network side by side represent the same neural network which has a recurrent relationship from middle hidden layer back to itself. The network takes three characters and predicts the next upcoming three characters.



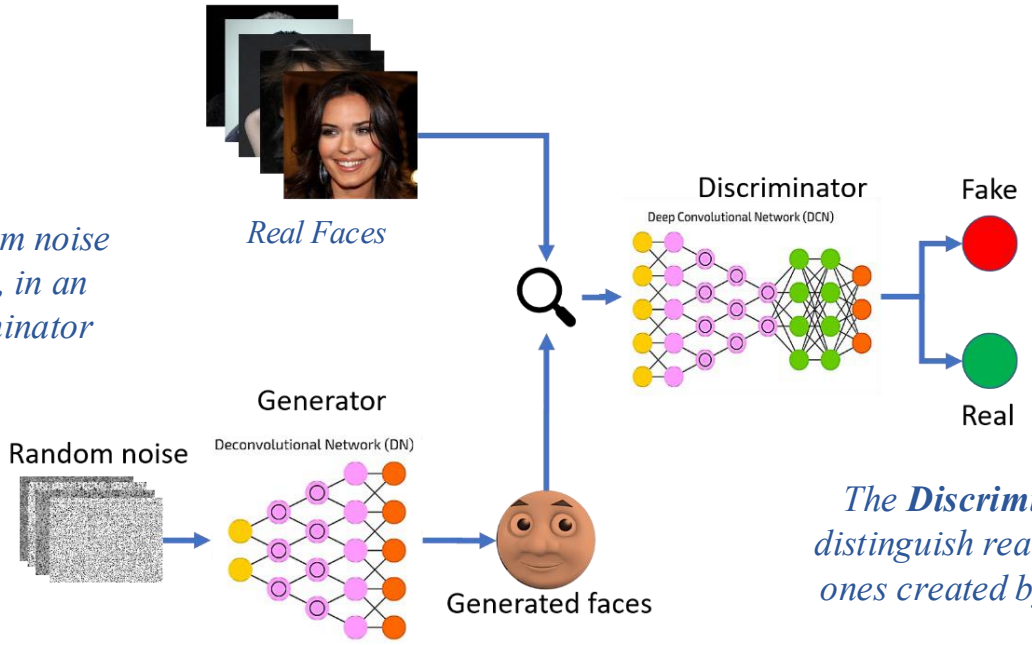
Generative versus Discriminative Networks

- **Generative**
  - Models a more general problem: how the data was generated
  - Estimates the distribution of data
  - Natural use of unlabeled data
  - Very expensive to get: needs lots of data

- **Discriminative**
  - Focus on the decision boundary
  - Can classify but not generate data
  - More powerful with lots of data
  - Only supervised task
  - Example: CNNs



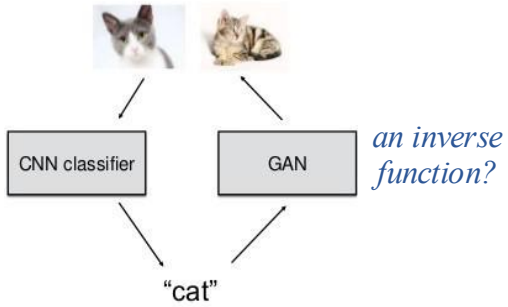
Generative Adversarial Networks (GANs)



*The **Generator** turns random noise into imitations of the data, in an attempt to fool the discriminator*

*The **Discriminator** tries to distinguish real data from fake ones created by the generator*

Discriminative vs. Generative



an inverse function?

When should we use DNNs or classical ML approaches?



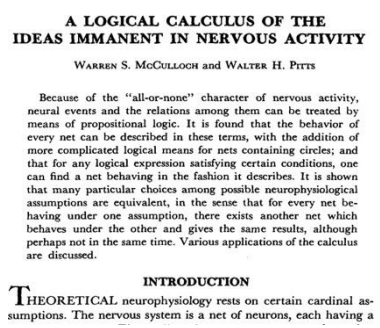
- It ***depends heavily on the problem*** you are trying to solve.
- The ***"no free lunch" theorem***:
  - There is no single technique that will always do best
- ***DNNs show state-of-the-art results*** since they are
  - ***Robust*:**
    - No need to design the features ahead of time, features are automatically learned to be optimal for the task
    - Robustness to natural variations in the data is automatically learned
  - ***Generalizable***:
    - The same neural net approach can be used for different applications and data types
  - ***Scalable:***
    - Performance improves with more data, method is massively parallelizable
- Although ***DNNs show state-of-the-art results***, classical ML approaches ***are still alive*** since
  - **They work better on small data:** To achieve high performance, deep networks require extremely large datasets (e.g. *1.2 million images*). For smaller datasets, classical ML algorithms often outperform deep networks.
  - They are financially and computationally cheap: Deep networks require high-end GPUs to be trained in a reasonable amount of time with big data. Classical ML algorithms can be trained just fine with just a decent CPU, without requiring the best of the best hardware.
  - They are easier to interpret: Due to the direct feature engineering involved in classical ML, these algorithms are quite easy to interpret and understand.

Deep Learning Timeline

- ***First computational model*** and was based on ***threshold logic***
- Each neuron is in a ***binary state*** of either "on" or "off"
- A neuron model that sums binary inputs and outputs a 1 if the sum exceeds a certain threshold value, and otherwise outputs a 0
- They also modeled a simple neural network with ***electrical circuits***
- One of the difficulties with the McCulloch-Pitts neuron was its simplicity:
  - It only allowed for ***binary inputs and outputs,***
  - it only used a simple ***'threshold step function'*** as the activation function,
  - it ***did not*** incorporate ***weighting*** the different inputs, and
  - their model also ***lacked*** a mechanism for ***learning***

Electronic Brain
McCulloch & Pitts

1943



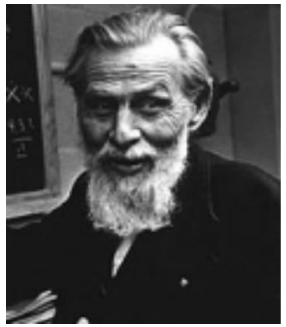
A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY

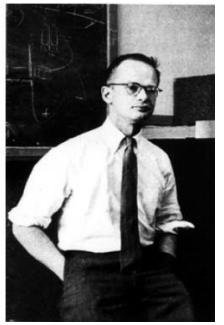
WARREN S. MCCULLOCH and WALTER H. PITTS

Because of the "all-or-none" character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms, with the addition of more complicated logical means for nets containing circles; and that for any logical expression satisfying certain conditions, one can find a net behaving in the fashion it describes. It is shown that many particular choices among possible neurophysiological assumptions are equivalent, in the sense that for every net behaving under one assumption, there exists another net which behaves under the other and gives the same results, although perhaps not in the same time. Various applications of the calculus are discussed.

INTRODUCTION

THEORETICAL neurophysiology rests on certain cardinal assumptions. The nervous system is a net of neurons, each having a





Deep Learning Timeline

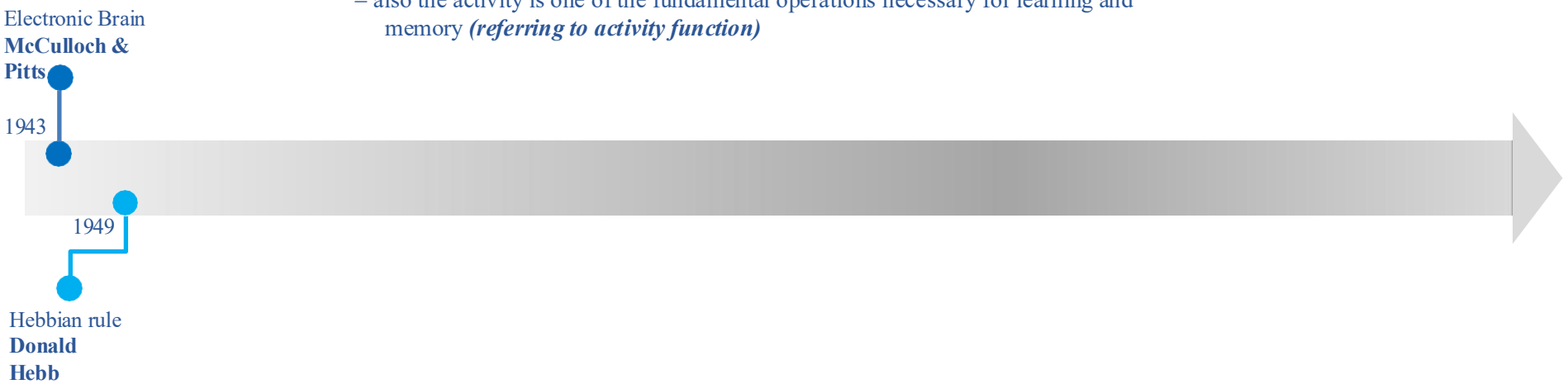
Donald Hebb (1949) – "The Organization of Behavior":

- In his book, he ***revolutionized*** the way that artificial neurons are perceived
- He proposed what has come to be known as Hebbian rule:

  "Cells that fire together wire together"

- Hebb had ***two proposals***:
  - when two neurons fire together simultaneously, the connection between the neurons strengthens ***(referring to weighting)***,
  - also the activity is one of the fundamental operations necessary for learning and memory ***(referring to activity function)***





Deep Learning Timeline

- Alan Turing, a British mathematician, is perhaps most well-known for his involvement in ***code-breaking during World War II***
- In 1947, he ***predicted the development of machine learning***, even going so far as to describe the impact it could have on jobs
- Computer scientist Alan Turing ***proposes a test for machine intelligence***
  - If a machine can trick humans into thinking it is human, then it has intelligence



Electronic Brain
McCulloch & Pitts

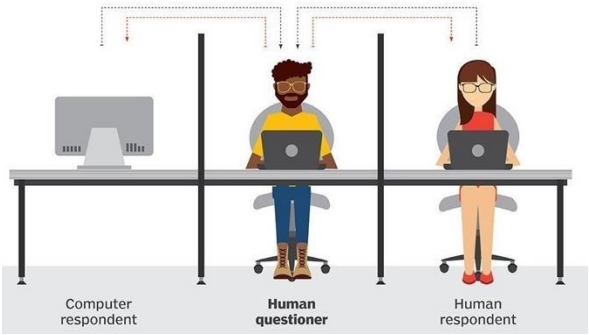
1943

1949

Hebbian rule
Donald Hebb

1950

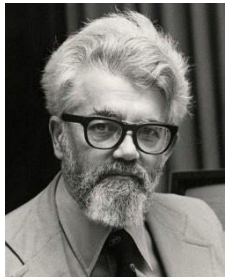
Computing Machinery & Intelligence
Alan Turing



Deep Learning Timeline

- Term ***"artificial intelligence"*** *is* ***coined*** by computer scientist, ***John McCarthy***
  to describe

  "the science and engineering of making intelligence machines"

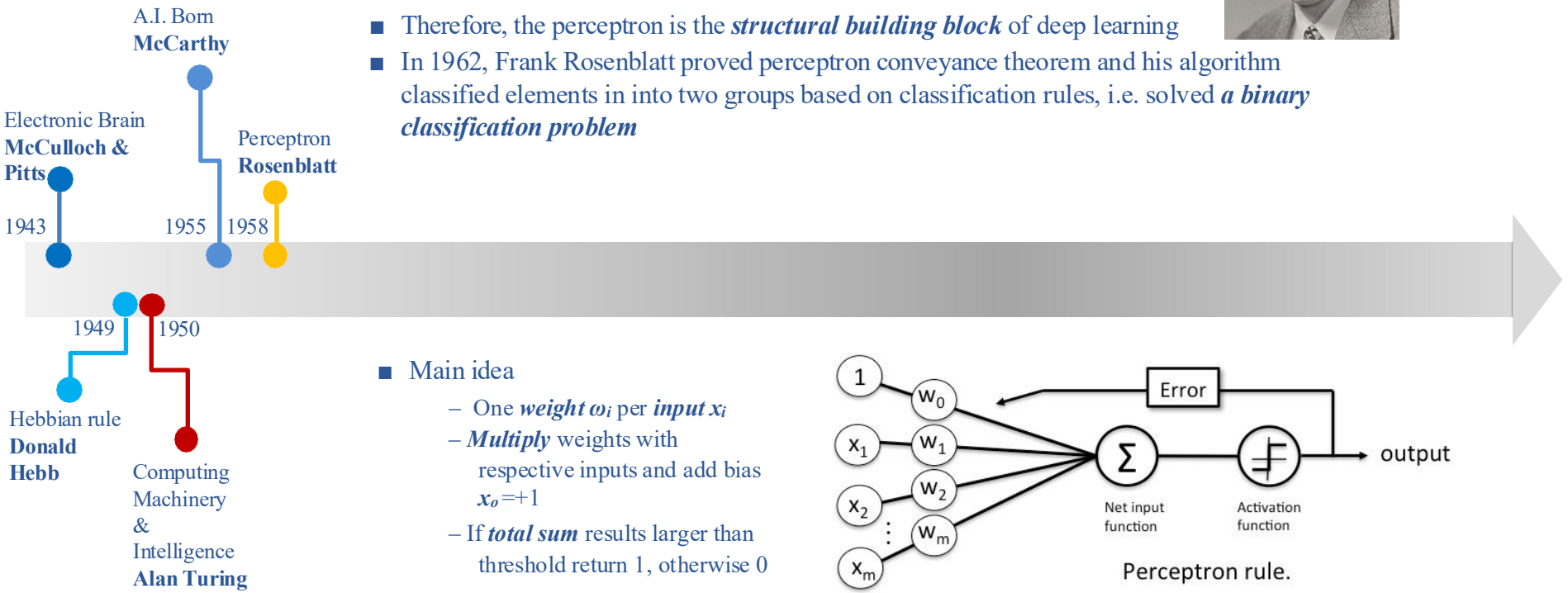




Deep Learning Timeline



- Frank Rosenblatt, using the ***McCulloch and Pitt's model*** of a neuron and the findings of ***Hebb***, went on to develop and introduced ***the first Perceptron*** in 1957
- A perceptron is a ***single neuron*** in a neural network and ***could learn*** in the Hebbian sense through the weighting of inputs
- Therefore, the perceptron is the ***structural building block*** of deep learning
- In 1962, Frank Rosenblatt proved perceptron conveyance theorem and his algorithm classified elements in into two groups based on classification rules, i.e. solved ***a binary classification problem***

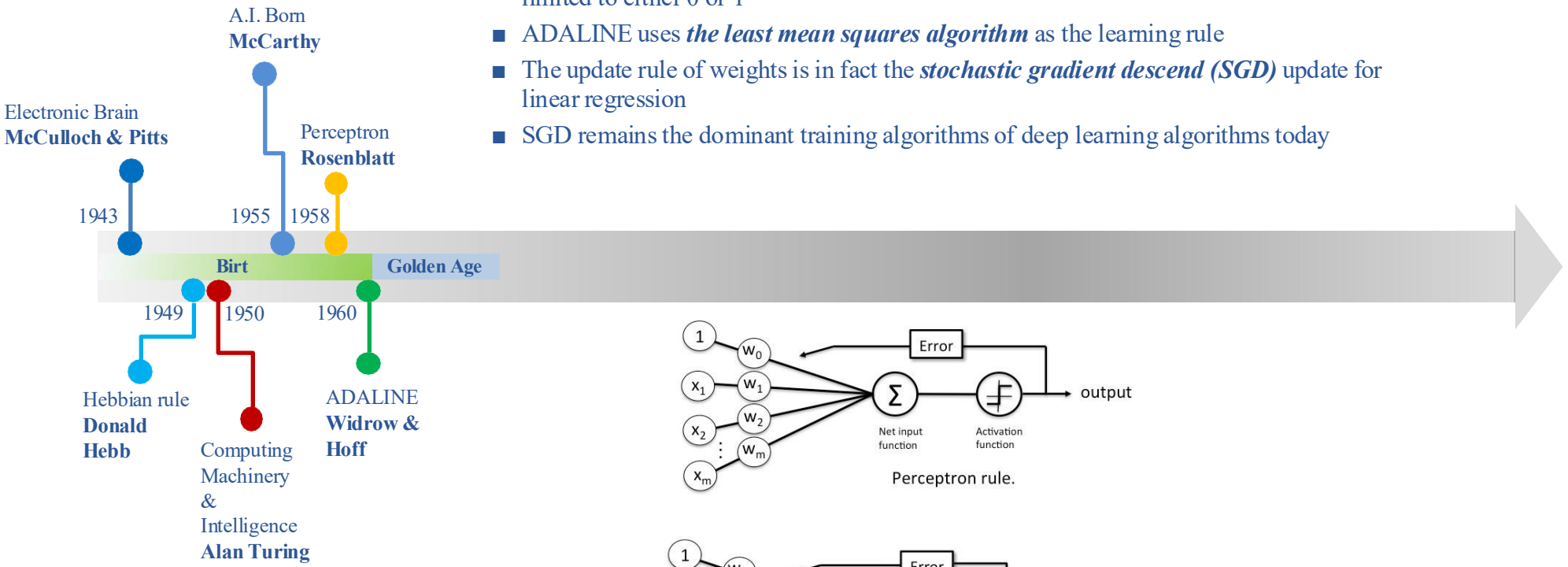


- Main idea
  - One ***weight*** ω_i per ***input*** x_i
  - ***Multiply*** weights with respective inputs and add bias x_o=+1
  - If ***total sum*** results larger than threshold return 1, otherwise 0

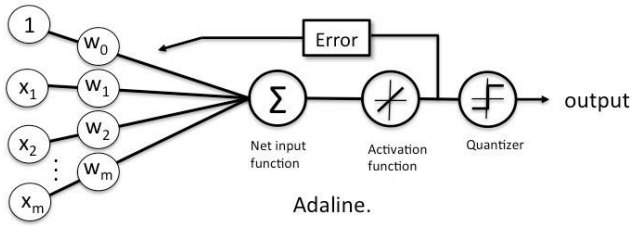
Perceptron rule.

Deep Learning Timeline

- ***ADALINE*** (ADAptive Linear Neuron) has the same architecture of perceptrons, but the ***transfer function is linear*** rather than thresholding
- This allows ADALINE outputs to ***take on any value***, whereas the perceptron output is limited to either 0 or 1
- ADALINE uses ***the least mean squares algorithm*** as the learning rule
- The update rule of weights is in fact the ***stochastic gradient descend (SGD)*** update for linear regression
- SGD remains the dominant training algorithms of deep learning algorithms today

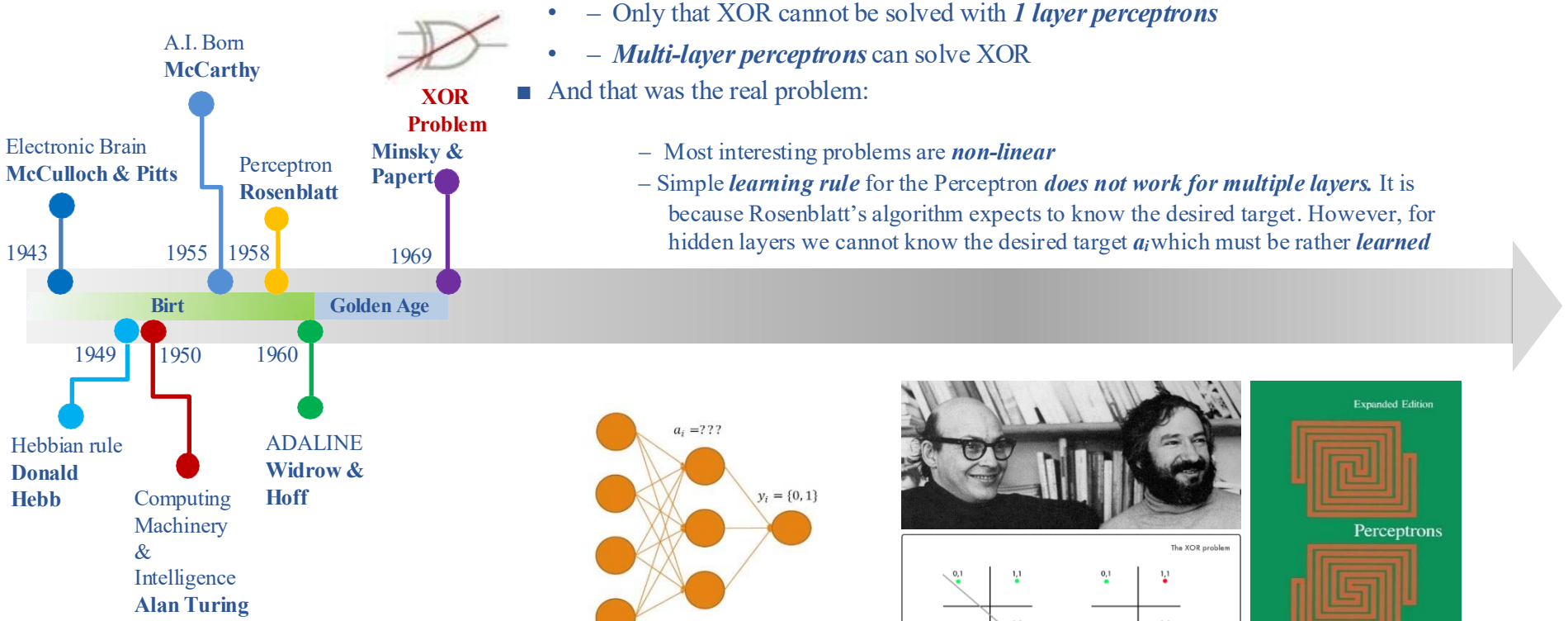


Perceptron rule.



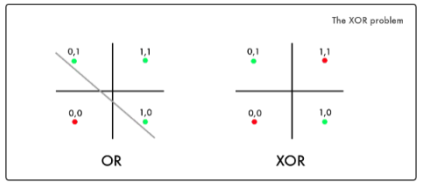
Adaline.

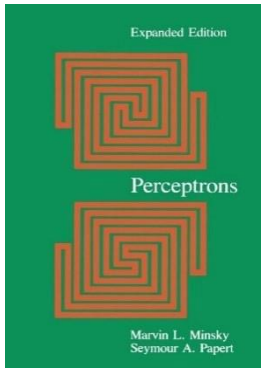
Deep Learning Timeline



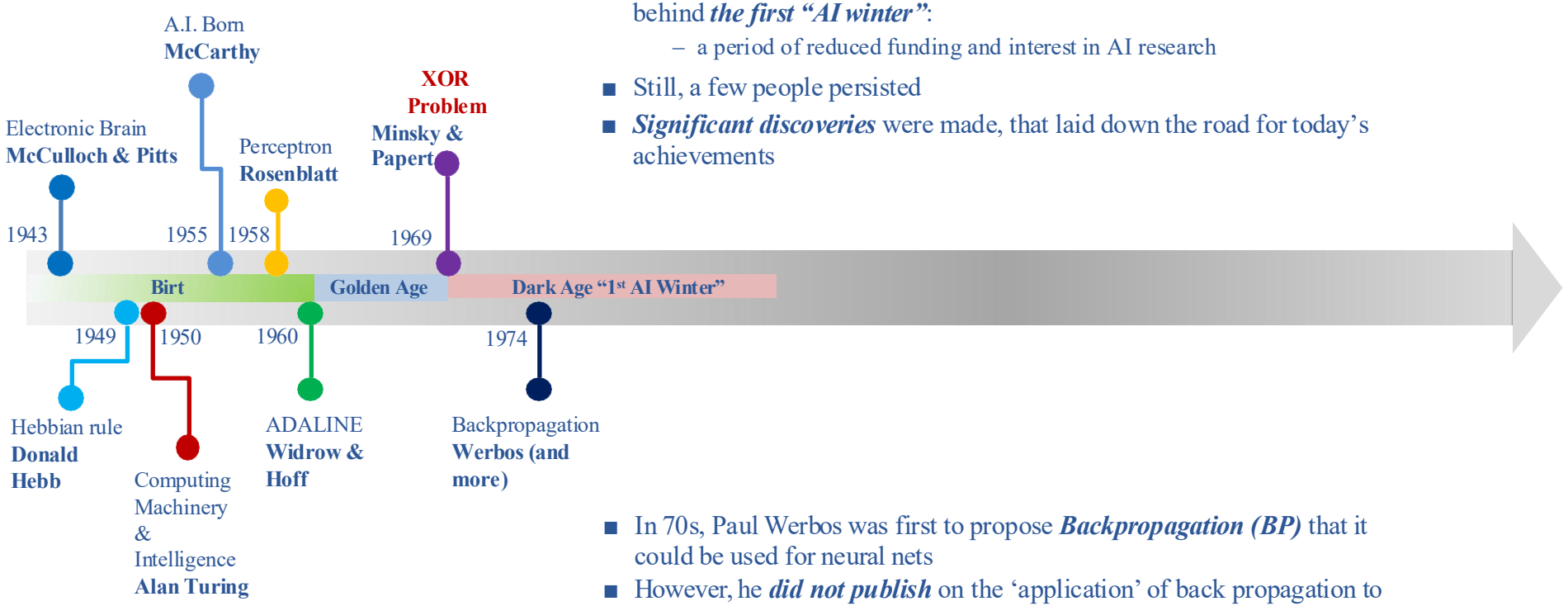
- In 1969, Minsky & Papert published "Perceptrons" ***stressing the limitations*** of perceptrons
  - – Linear networks like Perceptron and ADALINE can only solve ***linearly separable problems,*** but ***not learn*** the simple ***nonlinear function XOR***
- Minsky&Papert ***never*** said XOR cannot be solved by neural networks
  - – Only that XOR cannot be solved with ***1 layer perceptrons***
  - – ***Multi-layer perceptrons*** can solve XOR
- And that was the real problem:
  - – Most interesting problems are ***non-linear***
  - – Simple ***learning rule*** for the Perceptron ***does not work for multiple layers.*** It is because Rosenblatt's algorithm expects to know the desired target. However, for hidden layers we cannot know the desired target a_i which must be rather ***learned***

Multi-layer perceptron





Deep Learning Timeline

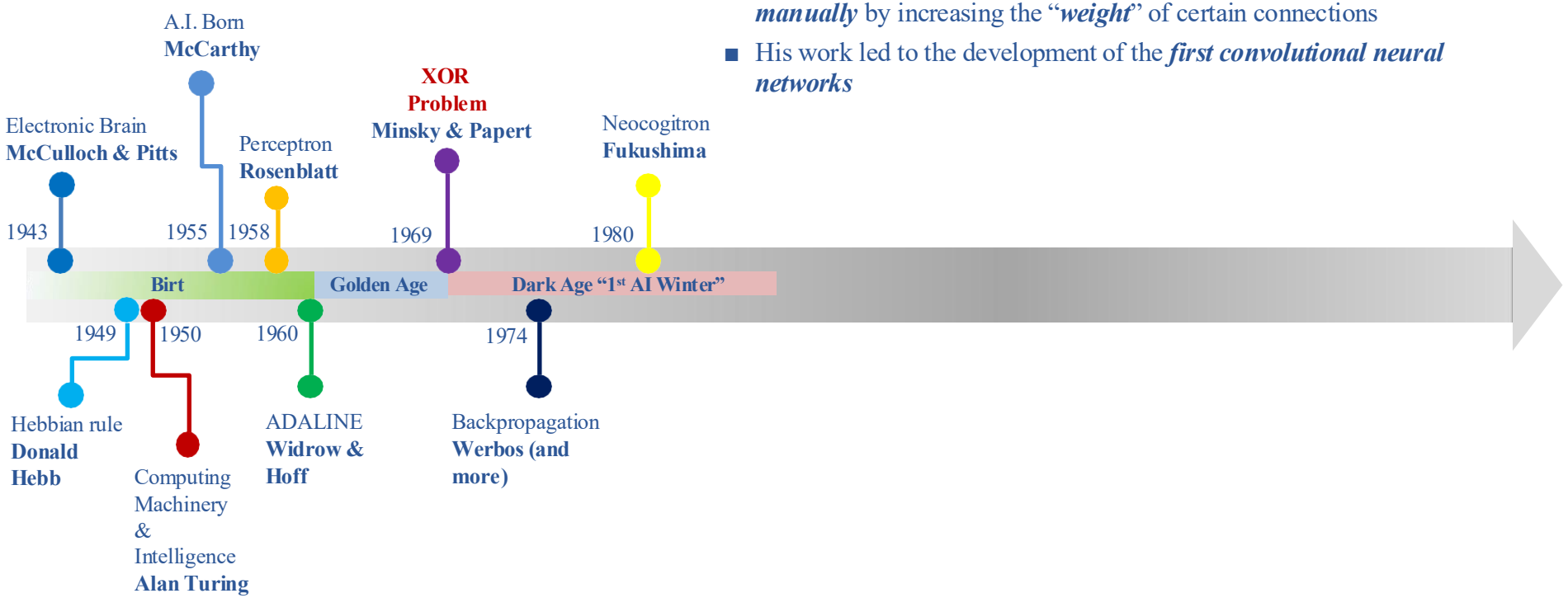


The first "AI Winter" started:

- Minsky & Papert's publication is widely believed to be the reason behind ***the first "AI winter"***:
  - a period of reduced funding and interest in AI research
- Still, a few people persisted
- ***Significant discoveries*** were made, that laid down the road for today's achievements

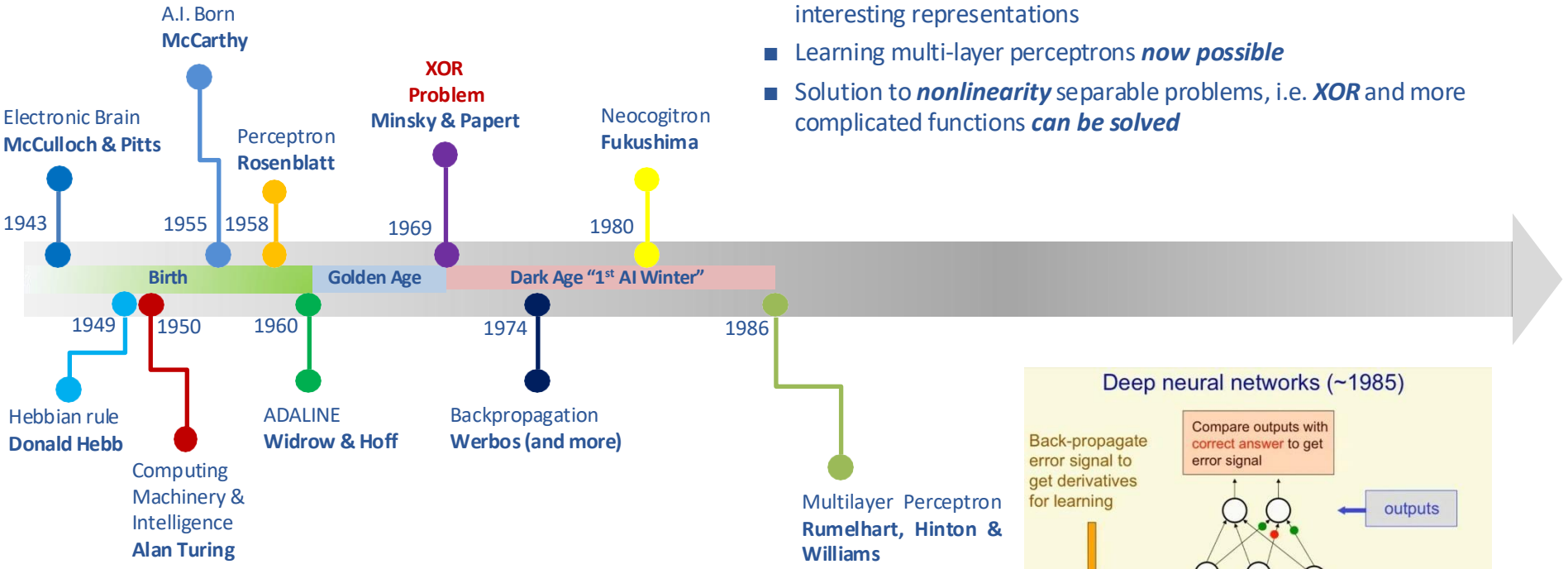
- In 70s, Paul Werbos was first to propose ***Backpropagation (BP)*** that it could be used for neural nets
- However, he ***did not publish*** on the 'application' of back propagation to neural nets until 1982 due to the chilling effects of ***AI winter***

Deep Learning Timeline

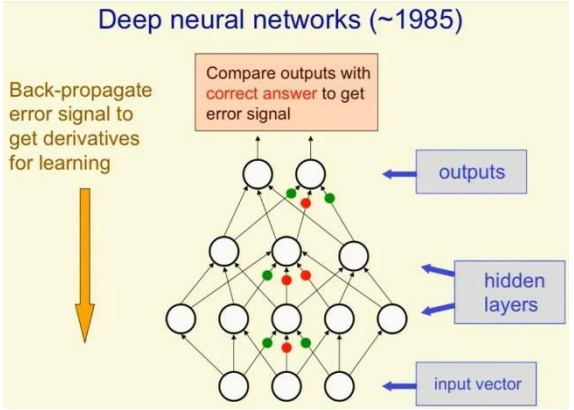


- Fukushima (1980) introduced the "***Neocognitron***" which is a ***hierarchical multilayered*** neural network capable of robust visual pattern recognition through learning
- The model was ***inspired*** by the ***visual nervous system*** of the vertebrate
- Fukushima's design allowed important features to be ***adjusted manually*** by increasing the "***weight***" of certain connections
- His work led to the development of the ***first convolutional neural networks***

Deep Learning Timeline

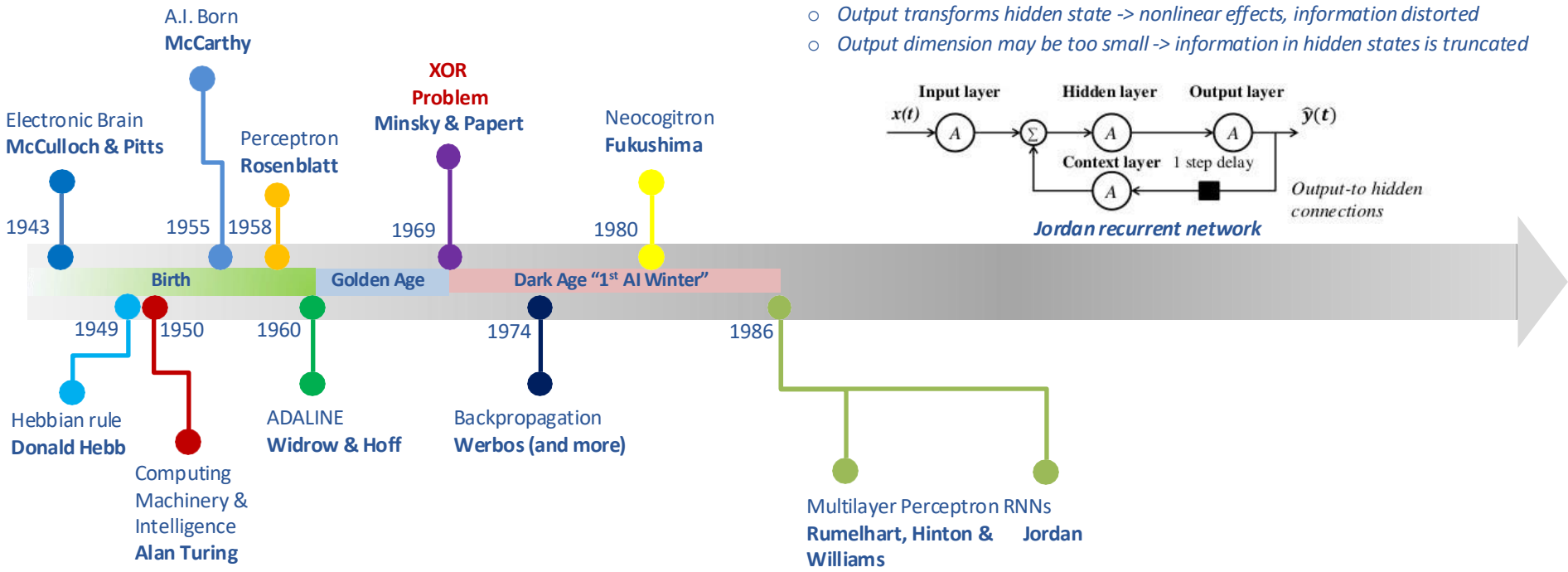


- In a 1986 paper entitled "***Learning Representations by Back-propagating Errors***," Rumelhart, Hinton, and Williams described in greater detail the ***process of backpropagation***.
- They showed that neural networks can ***automatically learn*** interesting representations
- Learning multi-layer perceptrons ***now possible***
- Solution to ***nonlinearity*** separable problems, i.e. ***XOR*** and more complicated functions ***can be solved***



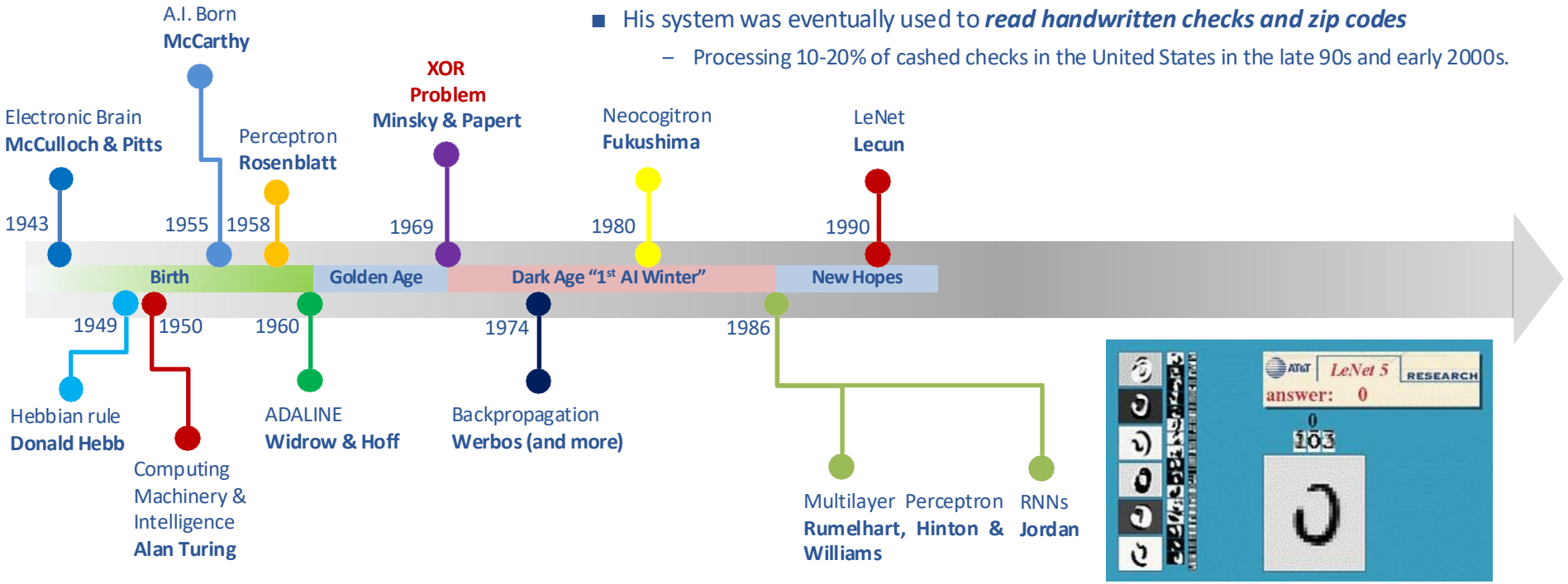
Deep Learning Timeline

- Traditional networks are "***too plain***",
  e.g. ***feed-forward networks:*** Static Input -> Processing -> Static Output
- What about ***sequences***, such as temporal data, language?
- Jordan introduced **recurrent feedback connections** to "***remember***" state changes
  - ***Pro:*** *fast to train since it can be parallelized*
  - ***Cons:***
    - *Output transforms hidden state -> nonlinear effects, information distorted*
    - *Output dimension may be too small -> information in hidden states is truncated*



Deep Learning Timeline

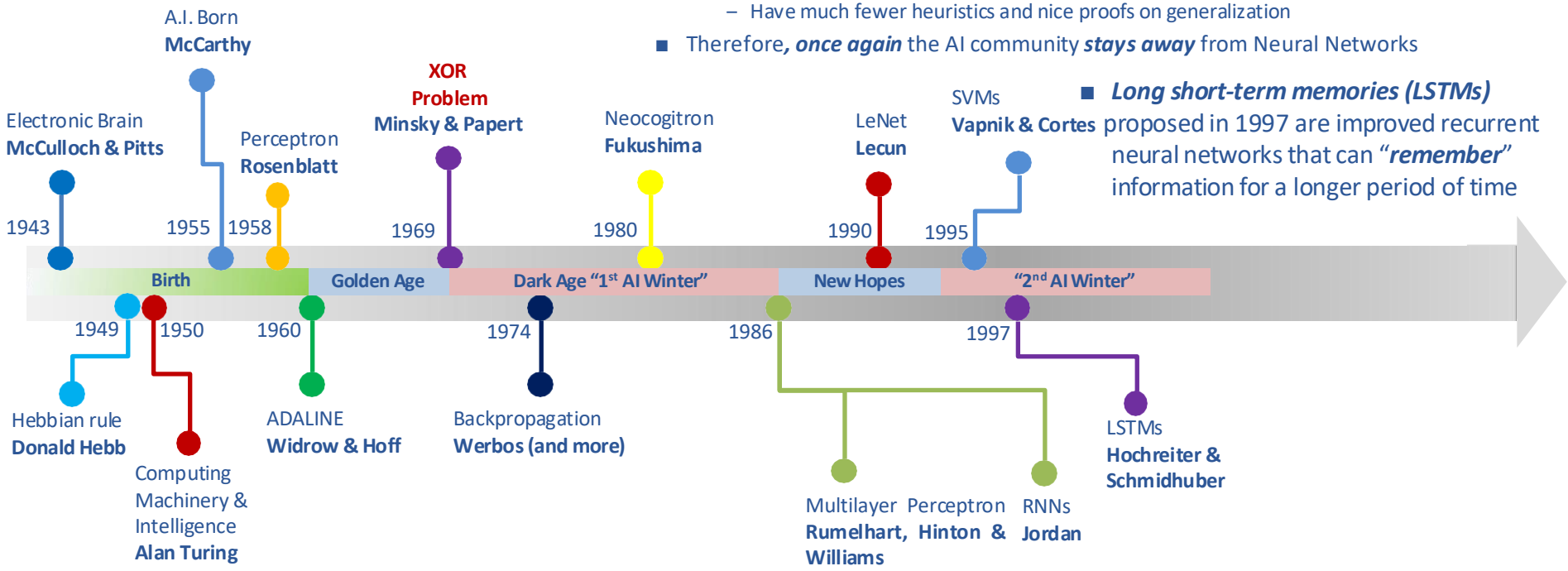
- ***Yann LeCun*** – another rock star in the AI and DL universe – ***combined convolutional neural networks with backpropagation theories*** to demonstrate a very significant real-world application: ***handwritten digits recognition***
- Concretely the first hidden layer of the neural network was '***convolutional***'
  - Instead of each neuron having a different weight for each pixel of the input image (40*60 = 2400 weights), the neurons only have a small set of weights (5*5=25) that were applied a whole bunch of small subsets of the image of the same size.
- His system was eventually used to ***read handwritten checks and zip codes***
  - Processing 10-20% of cashed checks in the United States in the late 90s and early 2000s.



Deep Learning Timeline

The second "AI Winter" started:

- Until early 2000 some ***nice algorithms*** were proposed
  - Backpropagation, Recurrent Long-Short Term Networks (LSTMs), LeNet with CNNs
- However, neural networks ***didn't scale to larger problems***
  - Too many parameters to learn from few labeled examples
  - Very slow and inefficient if networks have multiple hidden layers.
  - ***Vanishing gradient***: While learning NN, you need to multiply several weights. For example: if all are equal to 0.1, for 10 weights you have 0.0000000001, *too small for any learning*
- At the same time, ***Kernel Machines (e.g. SVMs)*** suddenly become very popular
  - Similar accuracies in the same tasks
  - Have much fewer heuristics and nice proofs on generalization
- Therefore***, once again*** the AI community ***stays away*** from Neural Networks

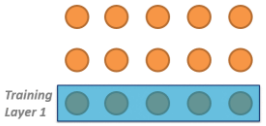


- ***Long short-term memories (LSTMs)*** proposed in 1997 are improved recurrent neural networks that can "***remember***" information for a longer period of time

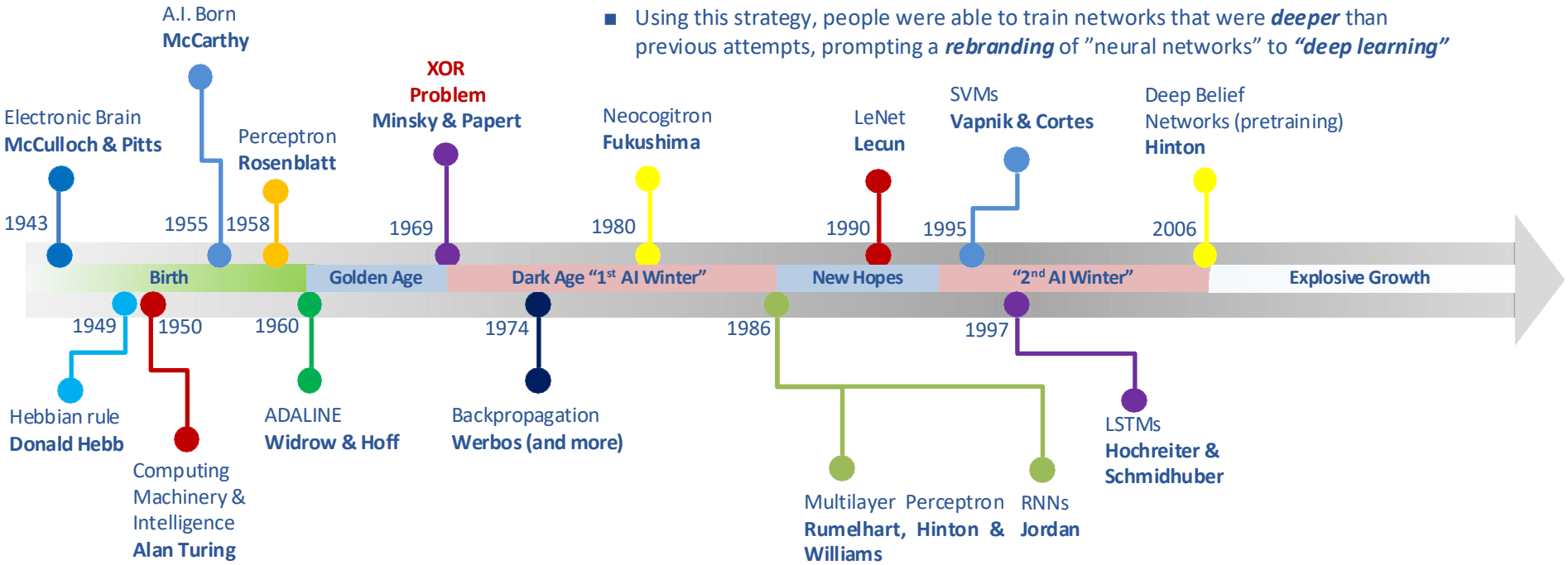
Deep Learning Timeline

Deep Learning Arrives:

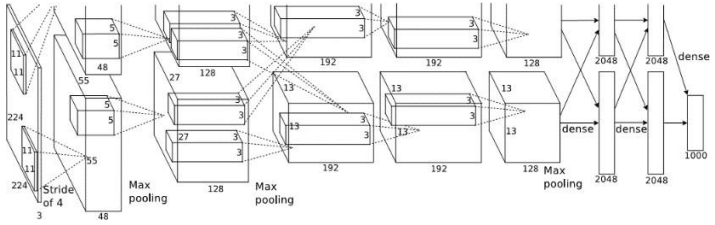
- In 2006, Hinton proposed ***greedy unsupervised layer-wise pretraining***:
  - Train one layer at a time starting from bottom (input) layer
  - Freeze the parameters of previous hidden layer
  - Repeat training for the next layer
  - Thus, each layer learns a higher level representation of the layer below
  - After pretraining, add output layer and train the whole network using backpropagation



- Using this strategy, people were able to train networks that were ***deeper*** than previous attempts, prompting a ***rebranding*** of "neural networks" to ***"deep learning"***

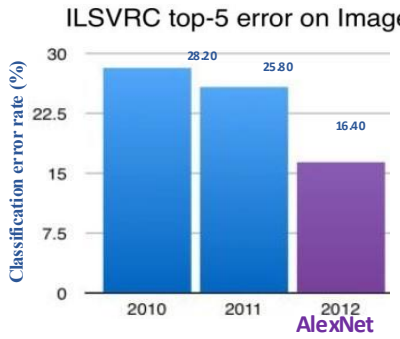


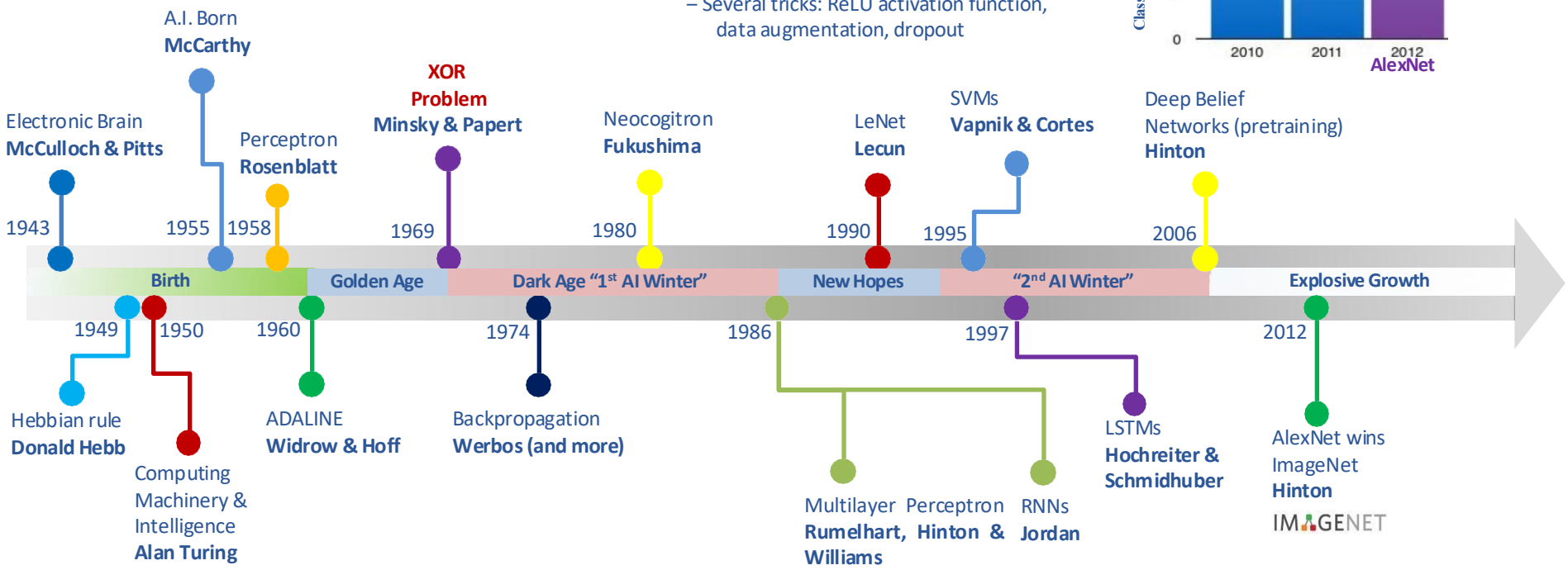
Deep Learning Timeline



- ***AlexNet*** was based on ***LeNet5*** which had 7 layers and introduced by Yann LeCun
- ***Winner of ImageNet 2012*** by a large margin
- The ***success of AlexNet***, a deep convolutional network with
  - 8 hidden layers
  - 60M parameters
  - Several tricks: ReLU activation function, data augmentation, dropout

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

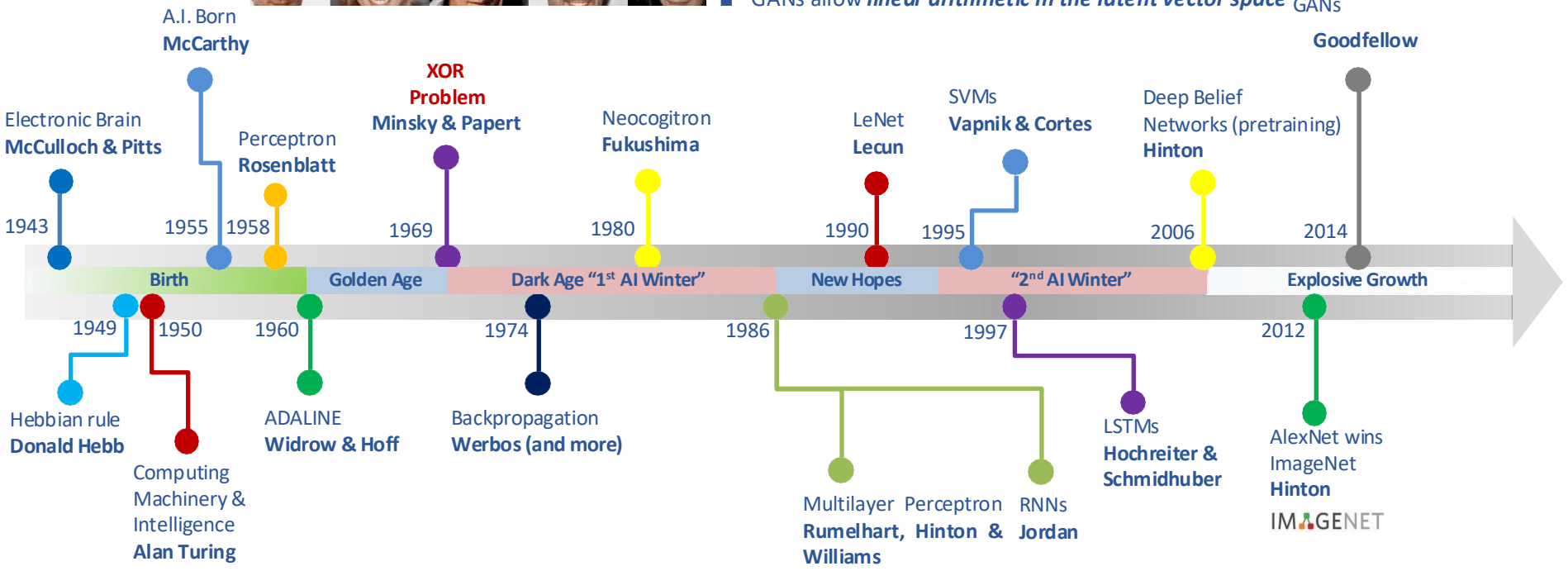




Deep Learning Timeline

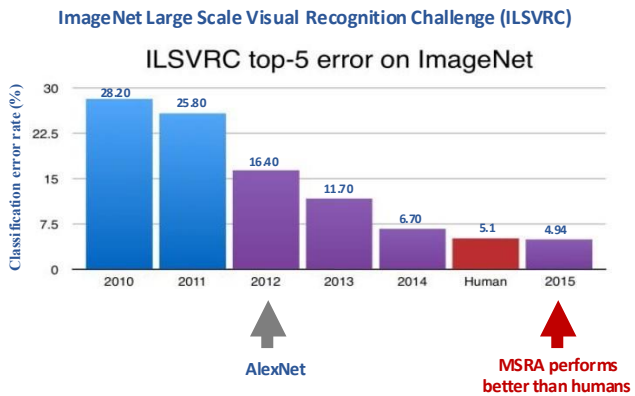


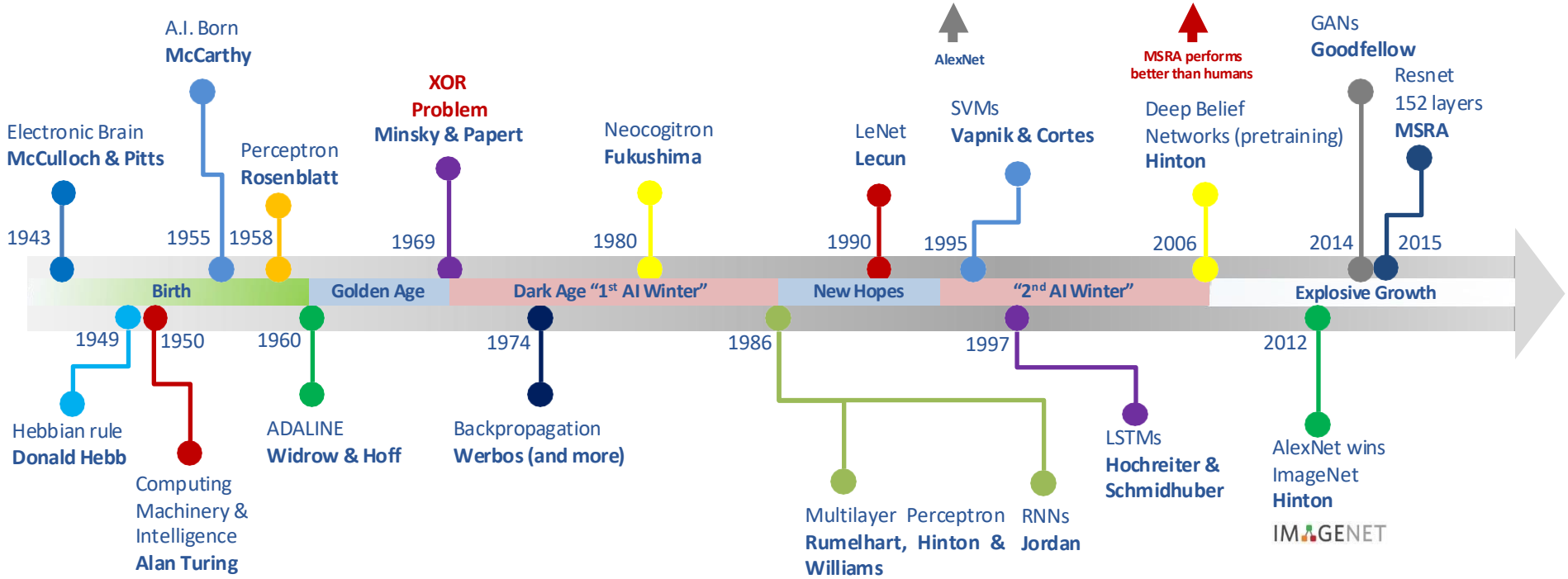
- *"**Generative Adversarial Network (GAN)** is the most interesting idea in the last ten years in machine learning"*, ***Yann Lecun***
- GANs introduced by ***Ian Goodfellow*** and have ***two different networks***, one against the other, hence they are ***adversarial***:
  - The ***Generator*** turns the input noise into fake data to try and fool the Discriminator
  - The ***Discriminator*** tries to distinguish between fake (generated) and real data
- GANs allow ***linear arithmetic in the latent vector space***



Deep Learning Timeline

Superhuman performance in image classification



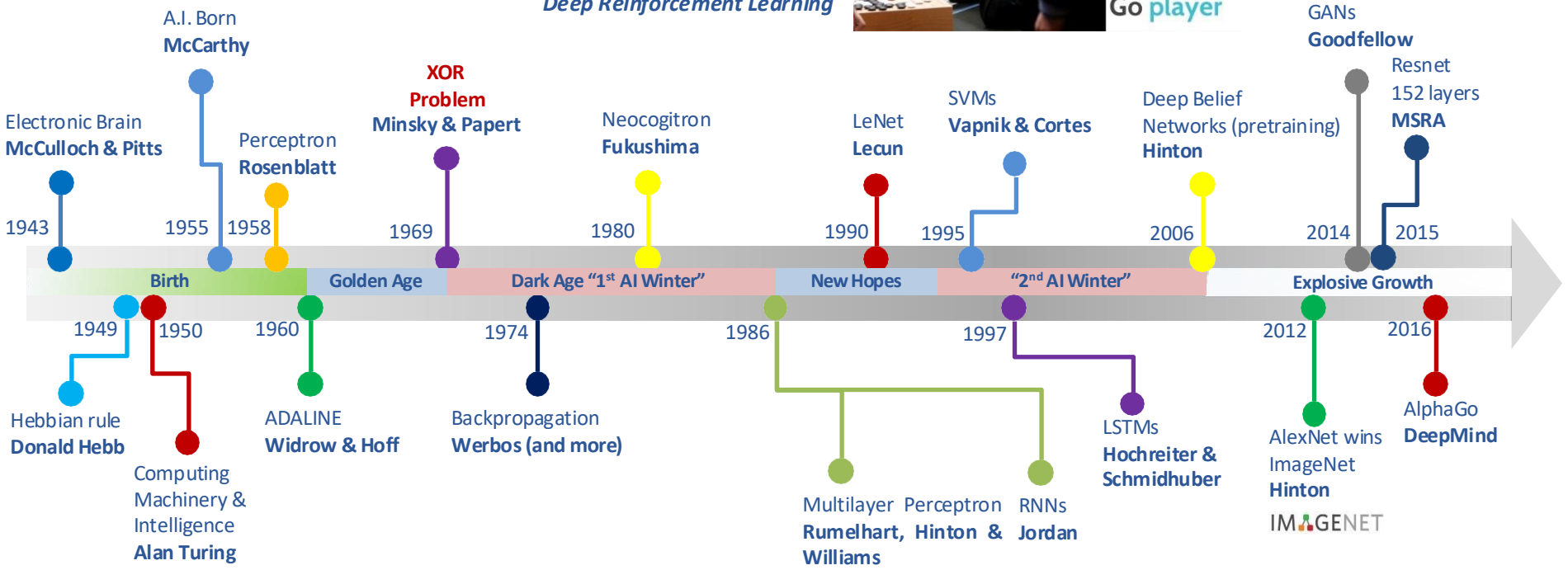


Deep Learning Timeline

AlphaGo could learn the game by ***examining*** thousands of human Go moves, and then it could master the game by ***playing itself*** over and over and over again
Deep Reinforcement Learning

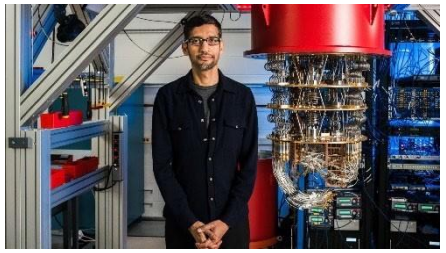


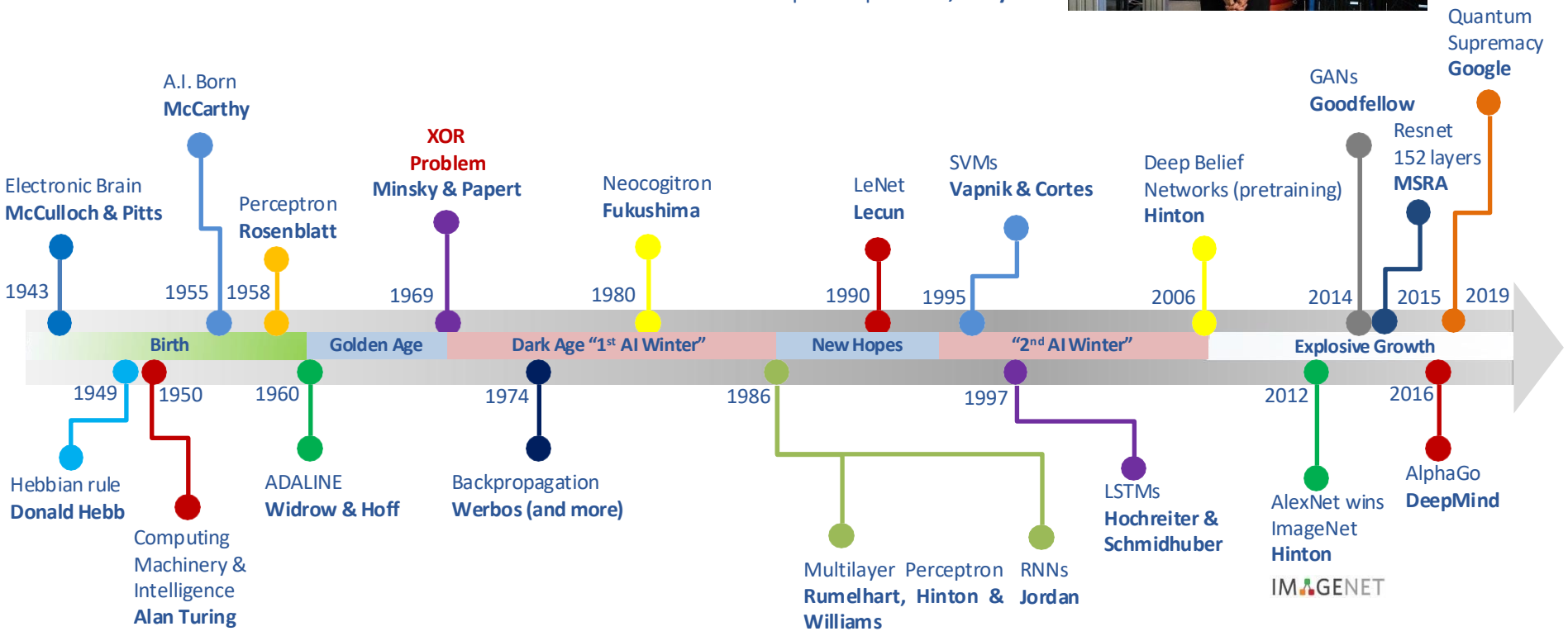
2016 DeepMind' AlphaGo defeated the best Go player



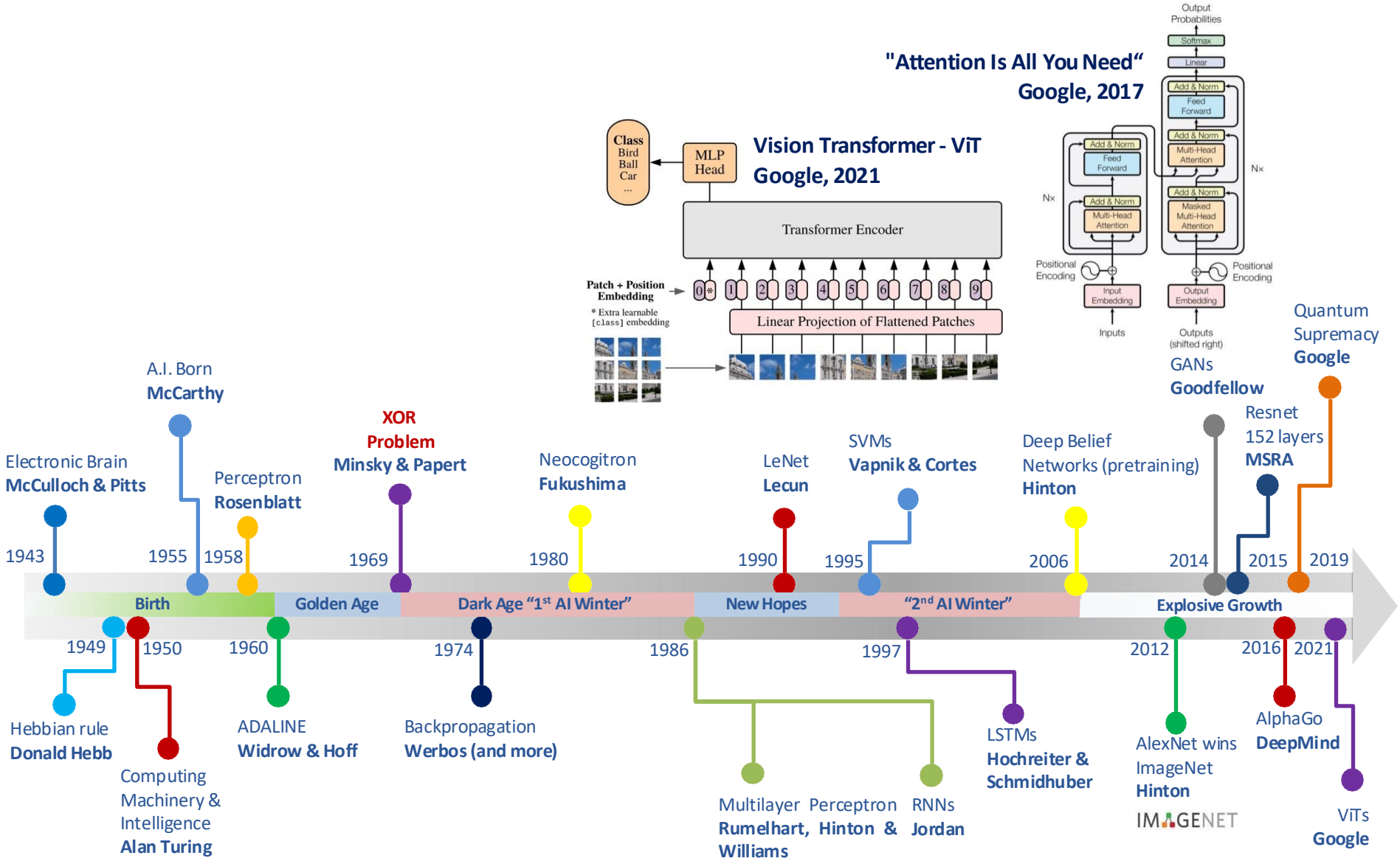
Deep Learning Timeline

Google claims to have attained "**quantum supremacy**" for the first time. Their 53-bit quantum computer, named Sycamore, took **200 seconds** to produce **one million random numbers** that, according to Google, would have taken the world's fastest supercomputer **10,000 years**.



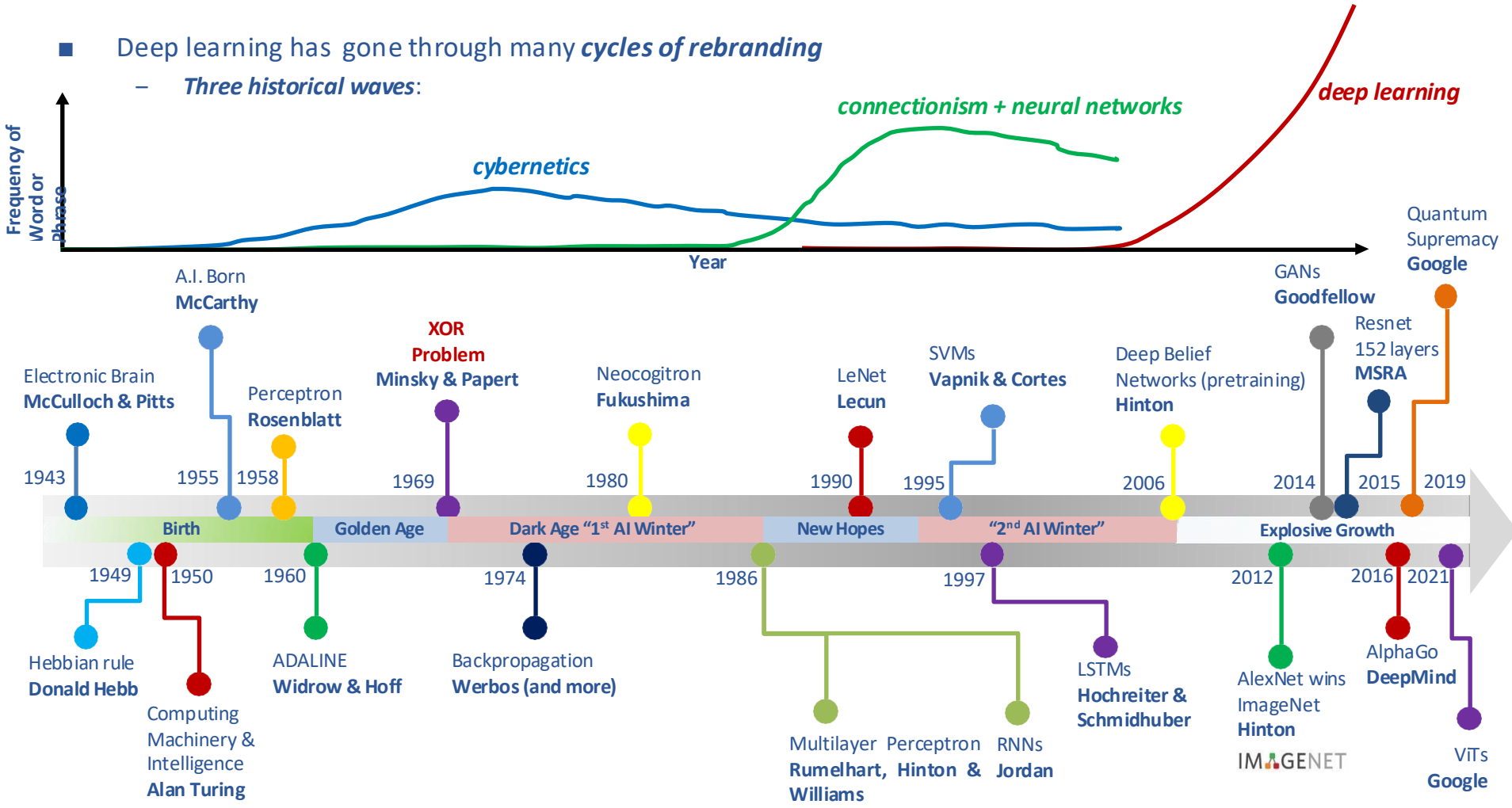


Deep Learning Timeline



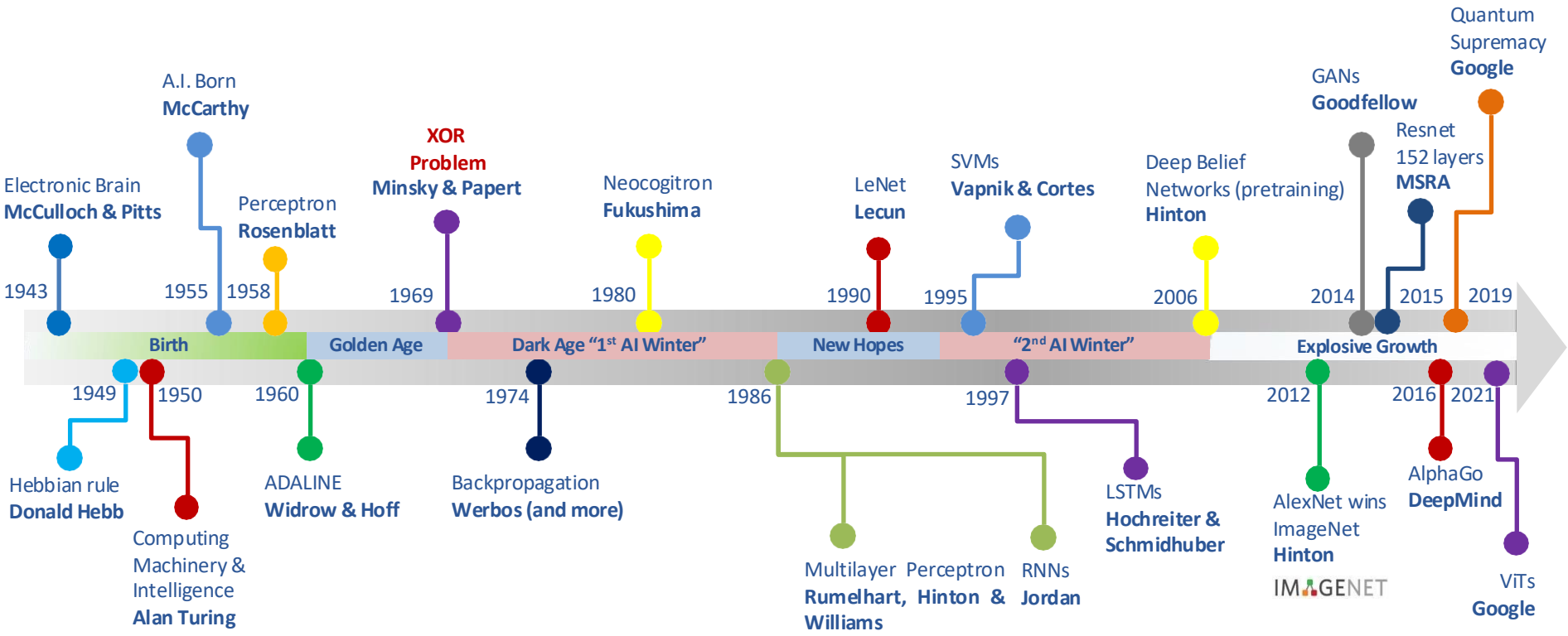
Deep Learning Timeline

- Deep learning has gone through many ***cycles of rebranding***
  - ***Three historical waves***:



Deep Learning Timeline

- Neural networks have been around ***for quite a while*** now. So why are NNs ***now taking off*** and ***working so well***?



Deep Learning Timeline

- Neural networks have been around ***for quite a while*** now. So why are NNs ***now taking off*** and ***working so well***?

Four Driving Factors…

Big Data Availability

- Larger Datasets
- Easier Collection & Annotation & Storage

IMAGENET 15 millions of labeled images

facebook 350 millions images uploaded per day

YouTube 100 hours of video uploaded every minute

Computational Power (Hardware)

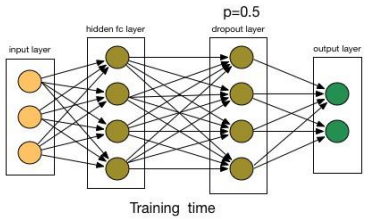
- Graphics Processing Units (GPUs)
- Massively Parallelizable
- Price decreases
- In every year, computational power is increasing exponentially.



Algorithms (Software)

- Improved regularization (e.g. dropout) and optimization techniques
- Novel Methods
- Toolboxes

TensorFlow



Dropout

Marketing

- Fancy naming
- New breakthroughs in robotics and AI





Thanks ©

And, once again:

Welcome!